

Документация Agent Platform

Table of contents:

Об Agent Platform

Версия движка Agent Platform

Тарификация

Импорт и экспорт проектов

Быстрый старт

Подходы к построению ИИ-решений

МСП-серверы в агентных системах

Обзор конструктора процессов

Подключение Git-репозитория

Управление сессией

Передача управления другому проекту

Вызов другого проекта

Тестирование процесса

Блок «Агент»

Работа агента в процессе

Управление контекстом агента

Встроенные функции

Пользовательские функции

Использование функций в процессе

Триггеры

Блок «Код»

Блок «Условие»

Подстановки {{ ... }}

Встроенные базы данных

Хранение данных внутри диалога

ВКонтакте

Чат-виджет

Chat API

Как подключить edna Chat Center

Mattermost

MAX

Prompter API

Telegram

WhatsApp

SIP-подключение

SIP-заголовки

Учетные данные для провайдеров ASR/TTS

Модели ASR и голоса TTS

Сравнение скорости ASR и TTS в телефонии

ASR и TTS в текстовых каналах

Перевод звонка на оператора

Подключение LLM

Подключение баз знаний (RAG)

Подключение MCP-серверов

Подключение IMAP-сервера

Подключение SMTP-сервера

Учетные данные для интеграций

Клиенты

Сессии

Приглашение пользователей в аккаунт

Роли и доступы

Управление тарифом и балансом

Техническая поддержка

Об Agent Platform

Agent Platform от Just AI — это единая платформа для создания AI-агентов. С ее помощью вы сможете:

- **Создавать умных помощников**, которые общаются с клиентами в чате и голосом.
- **Автоматизировать бизнес-процессы**, доверяя рутинные задачи AI-агентам.
- **Строить мультиагентные системы** для решения масштабных задач.

Обучающие материалы

Видеокурс: AI-агенты от А до Я

Изучите создание AI-агентов: от выбора LLM и интеграций до построения мультиагентных систем

Быстрый старт

Начните работу с Agent Platform и создайте свой первый проект

Версия движка Agent Platform

Версия движка влияет на написание и выполнение JavaScript-кода в проекте.

В Agent Platform работают две версии движка:

- Версия 2 — обеспечивает высокую производительность и позволяет создавать более сложные и отказоустойчивые процессы. В этой версии уже доступны все **встроенные функции и коллекции**.
- Версия 1 — поддерживается для обратной совместимости с ранее созданными проектами.

Ключевые отличия

Настройка поведения функций

Движок версии 2 позволяет гибко управлять поведением функций. В настройках блока *Функция* появилась новая вкладка *Поведение*, где вы можете настроить:

- *Время ожидания ответа*: до 60 минут. Если выполнение функции не завершится за это время, будет зафиксирована ошибка.
- *Повторные попытки при ошибке*: до 10 повторных вызовов.
Например, если указать таймаут 20 секунд и количество повторных попыток 3, то суммарное время ожидания может составить до 80 секунд: $20 \times (1 + 3)$.
- *Возвращать указанное значение вместо ошибки*: если функция не выполнится успешно после всех попыток, процесс не прервется, а на выход блока будет передано заданное значение (число, строка, JSON-объект, `true/false` или `null`).
- *Запрашивать подтверждение перед вызовом*: агент запросит у пользователя подтверждение перед запуском функции. Доступно только в **режиме инструмента**.

Синхронное выполнение системных функций

Встроенные функции в версии 2 выполняются синхронно. Это означает, что процесс дождетя завершения функции (например, ответа на HTTP-запрос), прежде чем перейти к следующей инструкции в коде. Такой подход делает код более предсказуемым.

В версии 1:



```
// Ожидание асинхронного ответа
Http.get("https://api.example.com/data")
  .then((response) => {
    // Код обработки ответа
  });
```

В версии 2:

```
// Процесс автоматически дождется результата
try {
  const response = Http.get({url: "https://api.example.com/data"});
  const body = JSON.parse(response.body);
  // Код обработки ответа
} catch (e) {
  // Обработка ошибок
}
```

⚠ ПРЕДУПРЕЖДЕНИЕ

Подключение прт-пакетов делает вызовы встроенных функций асинхронными. Обязательно учитывайте это при добавлении или удалении пакетов.

Подстановки {{ ... }}

- В движке версии 2 нельзя вызывать произвольные функции в подстановках {{ ... }}. Например, в блоке функции `Reactions.sendText()` нужно в качестве аргумента указывать не `{{ JSON.stringify(Context.getLastFunctionResult()) }}`, а `{{ previousResult }}` или `{{ previousResult | json }}`.

Подробнее в разделе [Подстановки](#).

The screenshot shows the configuration interface for the `Reactions.sendText` function. At the top, there's a title bar with a function icon, the name `Reactions.sendText`, and an edit icon. Below it are two buttons: "Отмена" (Cancel) and "Сохранить" (Save). The main description says "Sends a text message as a response". There are three tabs: "Параметры" (Parameters), "Контекст" (Context), and "Поведение" (Behavior). The "Параметры" tab is active. Under this tab, there's a toggle switch for "Режим инструмента. Функцию вызывает агент. Ответ не отправляется напрямую пользователю" (Tool mode. The function is called by the agent. The response is not sent directly to the user). Below this is a section titled "Параметры функции" (Function parameters). It contains a single parameter: `text: string *` with an information icon. The value field for this parameter contains the template `{{ previousResult | json }}`.

- Подстановки доступны для параметров с типом `object` в блоках функций.

Режим отладки

В версии 2 доступен **режим отладки** — пошаговое выполнение процесса с брейкпоинтами.

Тестирование функций в редакторе

Изолированный запуск пользовательской функции из ее редактора доступен только в движке версии 2 и только для JavaScript-функций. Подробнее в разделе **Тестирование функции в редакторе**.

Особенности и ограничения

Параметры системных функций

При вызове системных функций из кода (в пользовательских функциях, блоках *Код* и *Условие*) необходимо передавать аргументы в виде объекта. Передача примитивных типов (строк, чисел) не поддерживается.

В версии 1:

```
Reactions.sendText("Привет, мир!");
```

В версии 2:

```
Reactions.sendText({text: "Привет, мир!"});
```

Как переключить проект на новый движок

❗ К СВЕДЕНИЮ

Чтобы получить доступ к версии 2, свяжитесь со своим аккаунт-менеджером или напишите нам на contact@just-ai.com.

Чтобы переключить версию движка в проекте:

1. В левом меню перейдите в раздел *Настройки*.
2. На вкладке *Движок и агенты* в поле *Версия движка* выберите значение *Версия 2*.
3. Перейдите в конструктор:

- Отредактируйте вызовы системных функций в коде: в пользовательских функциях, блоках *Код* и *Условие*.
- Проверьте использование подстановок в параметрах блоков, внесите необходимые изменения.
- Если требуется, настройте **таймауты и обработку ошибок** в блоках функций.

4. Протестируйте процесс.

Тарификация

Для расчетов в Agent Platform используются **Ж-токены** (JT) — единица тарификации продуктов Just AI. Как посмотреть свой тариф, сменить его и пополнить баланс, читайте в статье [Управление тарифом и балансом](#).

❗ К СВЕДЕНИЮ

Баланс Ж-токенов в Agent Platform и в Jay Copilot не совпадает. Подробнее — в разделе про [общий баланс](#).

Тарифы

Бесплатный

На бесплатном тарифе вы платите только за то, что используете (Pay as you go) — работайте на платформе без ежемесячной абонентской платы.

Особенности тарифа

- Быстрый старт — начните работу сразу после регистрации.
- Ограниченные пакеты пополнения — на бесплатном тарифе доступны фиксированные пакеты: 10 000 JT, 50 000 JT или 100 000 JT.

💡 ПОДСКАЗКА

Бесплатный тариф подходит для знакомства с платформой и небольших проектов. Если вы регулярно запускаете много процессов, платный тариф будет выгоднее.

Платные

Платные тарифы включают ежемесячный пакет [запусков](#). Дополнительные запуски и запросы к базам знаний (RAG) оплачиваются Ж-токенами по ставке вашего тарифа.

Преимущества платных тарифов

- Ниже стоимость дополнительных запусков и RAG-запросов — чем выше тариф, тем выгоднее цена.
- Непрерывная работа — при превышении лимита запусков списание идет с баланса Ж-токенов.

⚠ ЧТО ВХОДИТ В ТАРИФ

В тариф включены **только запуски процессов** — в пределах ежемесячного лимита. Запросы к LLM, работа с базами знаний (RAG), хранение данных, векторизация и другие операции оплачиваются J-токенами и списываются с баланса **независимо от тарифа**.

J-токены не входят в тарифные планы и покупаются отдельно. Ими же оплачиваются запуски сверх месячного лимита.

Сравнение тарифов

Параметр	Бесплатный	Старт	Соло	Команда	Отдел	Бизнес	Enterprise
Цена в месяц, ₽	0	1 990	4 990	9 990	24 490	49 900	По запросу
Включено запусков/мес	Не включены	15 000	50 000	120 000	300 000	620 000	
Запуск сверх лимита, JT	2	1.3	1	0.8	0.6	0.5	
Запрос в базу знаний RAG, JT	2	1.3	1	0.8	0.6	0.5	
Запросы к LLM через Just AI	Стоимость зависит от модели						
Единый вход (SSO)	×	×	×	×	×	×	✓
Git-репозиторий	×	×	×	Скоро			

⚠ ПРЕДУПРЕЖДЕНИЕ

Цены могут меняться. Уточняйте актуальные условия и тарифы в личном кабинете или у менеджеров: contact@just-ai.com.

Калькулятор стоимости

Обращений в месяц

15000

Запусков на обращение

5

LLM

claude-sonnet-4-5 (Anthropic) ▾

Input-токенов на запуск

800

Цены указаны при работе с моделями напрямую, без OpenRouter

Output-токенов на запуск

400

☐ Используется база знаний (RAG)

• **Итого запусков в месяц: 75 000.** Запуск — одно выполнение процесса по триггеру. [Подробнее](#)

	Бесплатный	Старт	Соло	Команда	Отдел	Бизнес
Абонентская плата	0 ₽	1 990 ₽	4 990 ₽	9 990 ₽	24 490 ₽	49 900 ₽
Запуски сверх лимита	150 000 JT (37 500 ₽)	78 000 JT (19 500 ₽)	25 000 JT (6 250 ₽)	В рамках лимита	В рамках лимита	В рамках лимита
LLM для процессов: <i>claude-sonnet-4-5 (Anthropic)</i>	415 800 JT (103 950 ₽)	415 800 JT (103 950 ₽)	415 800 JT (103 950 ₽)	415 800 JT (103 950 ₽)	415 800 JT (103 950 ₽)	415 800 JT (103 950 ₽)
Итого в месяц	141 450 ₽	125 440 ₽	115 190 ₽	113 940 ₽ 🏆 Выгоднее всего	128 440 ₽	153 850 ₽

❗ К СВЕДЕНИЮ

Калькулятор предназначен для ознакомления. За финальными ценами и условиями обращайтесь к менеджерам: contact@just-ai.com.

Что считается запуском

Запуск — это однократное выполнение процесса, инициированное **триггером**. Каждое сообщение клиента агенту — это отдельный запуск.

Новый запуск списывается каждый раз, когда процесс запускается заново — например, клиент отправляет следующее сообщение после получения ответа или наступает время, заданное в настройках **триггера «Планировщик»**.

Обращение — это сессия общения клиента с агентом, которая может включать несколько сообщений и, соответственно, несколько запусков процесса.

❗ ПРИМЕР: АГЕНТ ПОДДЕРЖКИ

1. Клиент написал вопрос — процесс запустился. Это **1-й запуск**. Агент обратился к базе знаний (RAG) и выдал ответ — списался **1 RAG-запрос**.
2. Клиент прочитал ответ, не помогло — написал новое сообщение. Это **2-й запуск**. Агент снова обратился к RAG — **2-й RAG-запрос**.
3. Клиент уточнил еще пару деталей — еще **2 запуска**.

Итого за обращение: **4 сообщения = 4 запуска, 2 RAG-запроса**.

Статистика запусков и LLM

Вы можете посмотреть статистику внутри проекта на вкладке *Обзор*:

- *Расход токенов LLM* — количество потраченных токенов с разбивкой по моделям и датам.
- *Запуски процесса* — количество запусков с разбивкой по типам триггеров и датам.

Общий баланс с Jay Knowledge Hub

Agent Platform и [Jay Knowledge Hub](#) используют общий баланс J-токенов. Пополнив баланс один раз, вы можете использовать токены в обоих продуктах: оплачивать запуски процессов и запросы

к LLM в Agent Platform, а также индексацию, поиск, хранение данных и векторизацию в Jay Knowledge Hub.

⚠ ПРЕДУПРЕЖДЕНИЕ

Баланс J-токенов в Jay Copilot не связан с балансом Agent Platform и Jay Knowledge Hub. Токены, приобретенные в Jay Copilot, нельзя использовать в Agent Platform (Jay Knowledge Hub), и наоборот. Пополняйте баланс этих продуктов отдельно.

Импорт и экспорт проектов

Импорт и экспорт позволяют сохранять полную копию проекта в архив. Это удобно для переноса данных между аккаунтами, создания шаблонов и резервного копирования.

Экспорт проекта

Экспорт сохраняет структуру и логику проекта в ZIP-архив. Файл можно передать другому пользователю или использовать как бэкап.

Чтобы экспортировать проект:

1. Перейдите к списку проектов.
2. Нажмите **⋮** на карточке нужного проекта.
3. Выберите пункт *Скачать проект* и выберите место для сохранения файла.

Начнется загрузка архива.

Состав архива

Архив содержит JSON-файлы с описанием компонентов проекта:

- `metadata.json` — метаданные и информация о проекте.
- `nodes.json` — блоки процесса со всеми параметрами.
- `userFunctions.json` — пользовательские функции.
- `collections.json` — коллекции функций.
- `integrations.json` — настройки интеграций (без учетных данных).
- `credentials.json` — шаблоны для учетных данных, которые нужно будет заполнить после импорта.
- `README.txt` — служебный файл с описанием архива.

Ограничения экспорта

Из соображений безопасности экспорт не включает:

- Учетные данные: API-ключи, токены и пароли удаляются. Их нужно создать заново после импорта.

- Интеграции: телефонный канал, голос TTS, модель ASR не сохраняются. Они настраиваются отдельно в новом проекте.
- Настройки SIP-подключения. Однако, если SIP-подключение уже настроено в другом проекте этого же аккаунта, оно отобразится в проекте автоматически.

Импорт проекта

1. Нажмите кнопку  в верхней панели на странице со списком проектов.
2. В открывшемся окне перетащите ZIP-архив или выберите файл на компьютере.
3. Укажите название нового проекта. По умолчанию используется имя файла.
4. Нажмите *Импортировать*.

Интерфейс будет заблокирован на время операции. После завершения новый проект появится в общем списке.

К СВЕДЕНИЮ

Если в аккаунте уже есть проект с таким названием, создастся дубликат с тем же именем.

Настройка после импорта

Так как учетные данные не переносятся, после импорта требуется донастройка:

- *Учетные данные*: перейдите в соответствующий раздел и добавьте актуальные API-ключи, токены и пароли в качестве учетных данных.
- *Интеграции*: выберите новые учетные данные в настройках интеграций.
- *Процесс*: в блоках, требующих интеграций со сторонними сервисами, проверьте и заполните пустые поля внутри сценария.

После этого можно протестировать процесс и опубликовать проект.

Быстрый старт

В этой статье вы узнаете, как использовать **триггеры**, **AI-агента** и функции в Agent Platform.

В качестве примера мы создадим процесс для работы с новостями про космос. Процесс позволит:

- Запрашивать статьи на нужную тему.
- Регулярно получать последние новости.

Чтобы начать работу:

1. **Создайте проект.**
2. **Настройте поиск статей через AI-агента.**
3. **Настройте регулярную отправку последних новостей в чат.**

Шаг 1. Создание проекта

1. Зарегистрируйтесь в **Agent Platform**.

❗ К СВЕДЕНИЮ

Agent Platform и [Conversational Cloud](#) используют общую базу аккаунтов, поэтому если вы зарегистрированы в Conversational Cloud, дополнительно регистрироваться в Agent Platform не нужно.

2. На странице *Проекты* нажмите *Создать AI-приложение*.
3. Выберите шаблон *Пустой проект* и укажите название.

💡 ПОДСКАЗКА

Вы также можете создать проект на основе шаблона *Быстрый старт*. Он уже содержит полностью готовый процесс из этой статьи — останется только добавить учетные данные с токеном Telegram-бота и указать `chatID` в функции `Telegram.sendText`.

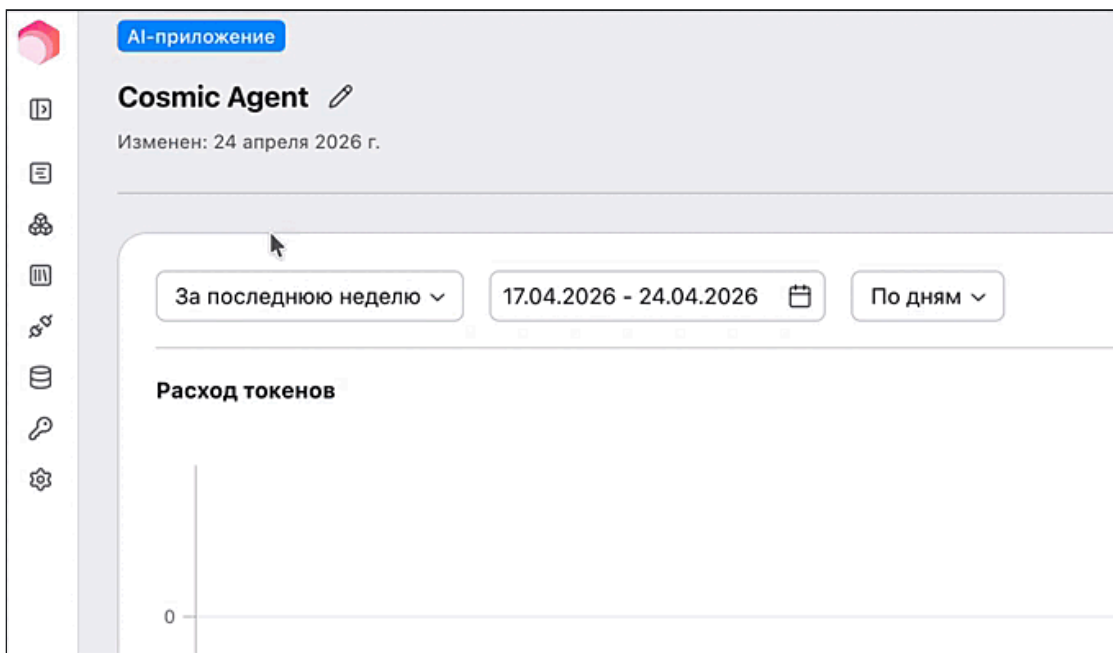
4. Нажмите *Создать*. Вы сразу перейдете внутрь проекта.

Шаг 2. Поиск статей через AI-агента

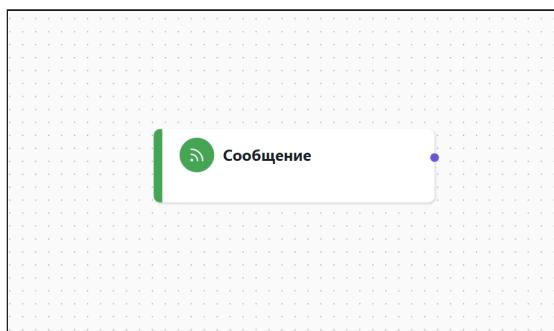
Триггер по новому сообщению

Триггер

1. На левой панели нажмите , чтобы открыть конструктор процессов.
2. Раскройте раздел *Триггеры*:



3. Перетащите **блок** *Сообщение* на холст. **Триггер** будет срабатывать каждый раз, когда пользователь отправляет сообщение в чат.



Функция для агента

Пользовательская функция

Инструмент

Добавьте новую функцию:

1. В списке блоков откройте раздел *Функции*.
2. Нажмите + справа от раздела *Пользовательские*.

3. В открывшемся окне выберите, куда сохранить новую функцию — *Новая коллекция* и нажмите *Далее*.
4. Укажите информацию о коллекции:
 - *Название*: `Новости про космос`.
 - *Идентификатор* `SpaceArticles`.
 - *Описание*: `Коллекция функций для работы с новостями про космос`.
5. Нажмите *Добавить*.
6. В открывшемся окне укажите информацию о новой функции:
 - *Название*: `Найти статьи`.
 - *Идентификатор*: `searchArticles`.
 - *Описание*: `Найти последние статьи по нужной теме`. По описанию агент будет понимать, что делает функция.
7. Нажмите *Сохранить*. После сохранения откроется окно настроек функции.
8. Нажмите *Добавить* в секции *Параметры* и выберите *string*. Далее:
 - В поле *Название* укажите название параметра, например `query`.
 - В поле *Описание* введите `Термин на английском языке, по которому нужно найти статьи`. По этому описанию агент будет понимать, что нужно передать в параметр.
9. В секции *Поля ответа* в поле *Описание* укажите `Данные статей`.
10. Перейдите на вкладку *Код* и введите тело функции на языке JavaScript. Реализация зависит от **версии движка** в вашем проекте.

Версия 2 **Версия 1**

```
try {
  // Отправляем запрос, чтобы получить статьи по термину
  var response = Http.get({
    url: "https://api.spaceflightnewsapi.net/v4/articles/",
    params: {
      // Передаем в API-запрос параметр query
      search: query,
      ordering: "-published_at"
    },
    headers: {}
  });

  var body = typeof response.body === "string" ? JSON.parse(response.body) :
response.body;

  // Сохраняем нужные данные о статьях
  var articles = (body.results || []).map(function (a) {
    return {
      title: a.title,
```

```
        summary: a.summary,
        url: a.url,
        publishedAt: a.published_at
    };
});

// Если статей нет
if (articles.length === 0) {
    return "No data";
}

return JSON.stringify(articles);
} catch (err) {
    return "Error fetching articles: " + err;
}
```

💡 ПОДСКАЗКА

Вы можете протестировать JavaScript-функцию прямо в редакторе, без запуска всего процесса — эта возможность доступна в проектах на движке версии 2. Подробнее читайте в разделе [Тестирование функции в редакторе](#).

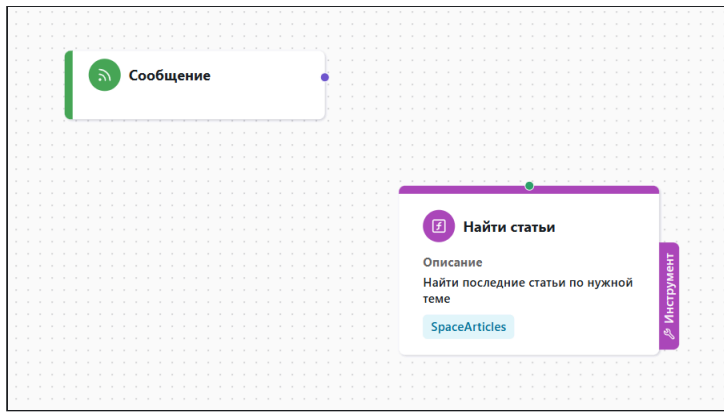
Функция получает релевантные статьи из [Spaceflight News API](#) и возвращает их данные.

11. Нажмите *Сохранить* вверху окна настроек.

Настройте функцию:

1. Перетащите новую функцию *Найти статьи* из раздела *Функции* → *Пользовательские* → *Новости про космос* на холст.
2. На холсте дважды нажмите на блок этой функции, чтобы открыть ее настройки.
3. Переведите функцию в режим инструмента, чтобы AI-агент мог ее использовать:
 - i. В окне настроек включите *Режим инструмента*.
 - ii. У параметра `query` включите опцию  (*Заполнить через LLM*).
4. Нажмите *Сохранить* вверху окна настроек.

Теперь на холсте есть два блока:



Создание AI-агента

Агент

Добавьте агента:

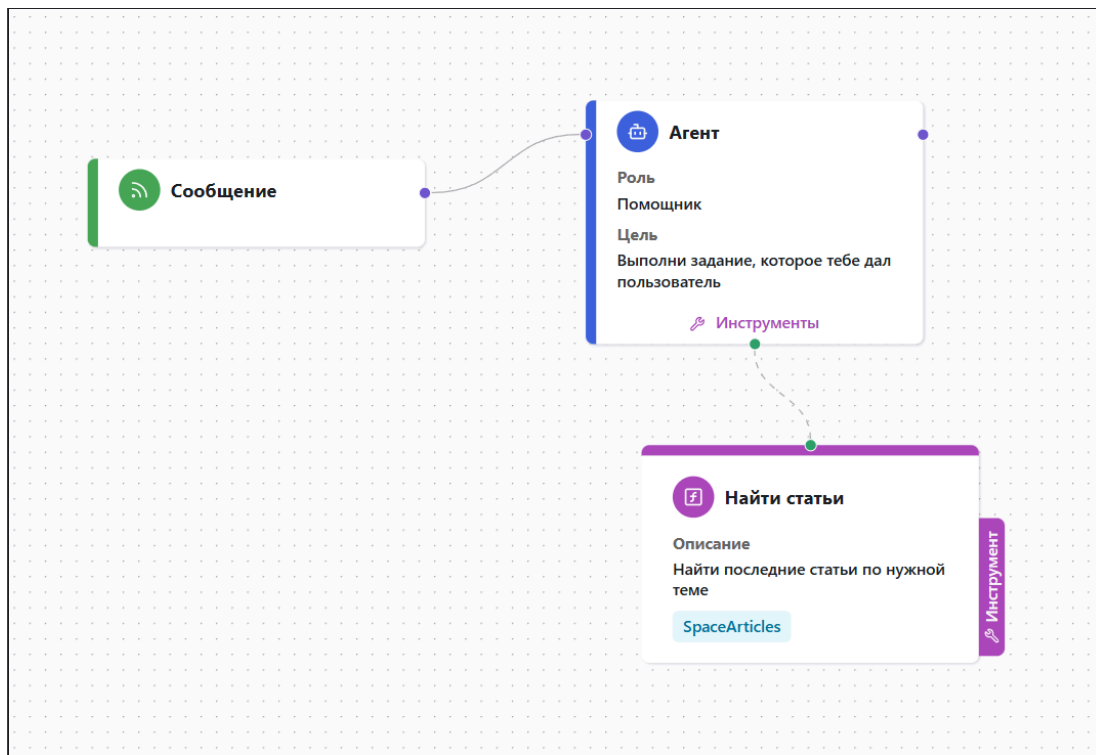
1. Перетащите блок *Агенты* → *Агент* на холст.
2. На холсте дважды нажмите на блок агента.
3. Раскройте панель *Основные*:
 - i. В поле *LLM* выберите *Создать интеграцию*.
 - ii. В поле *Доступ к LLM* выберите *Через Just AI*.
 - iii. В поле *Провайдер* выберите *OPEN_AI*, а в поле *Модель* — *gpt-5.2*.
 - iv. Нажмите *Добавить*.
4. Раскройте панель *Промты*:
 - o *Роль*: **Помощник**.
 - o *Цель*: **Выполни задание, которое тебе дал пользователь**.
 - o *Инструкции*:

Если пользователь просит найти новости:

1. Вызови функцию `searchArticles` и передай термин на английском языке в качестве параметра.
2. Когда получишь новости, составь один абзац текста со всей информацией.
3. Переведи на русский язык, подпиши, за какой период новости, и приложи список ссылок.
4. Отправь пользователю.

5. Нажмите *Сохранить* вверху окна настроек.

Далее соедините триггер *Сообщение* с агентом, а функцию *Найти статьи* присоедините к агенту как инструмент.



💡 ПОДСКАЗКА

Чтобы удалить связь между блоками, нажмите на связь и затем нажмите Delete или Backspace.

Тестирование

Нажмите *Тестировать* в правом верхнем углу и выберите триггер *Сообщение* — откроется тестовый чат-виджет. В нем вы можете переписываться с агентом и проверять его ответы.

Пример диалога:

1. 👤 **Пользователь:**

Есть что-то интересное про Марс?

2. 🤖 **Ассистент:**

Вот последние интересные новости о Марсе за август 2025 года:

В начале августа марсоход Curiosity продолжил исследование уникальных структур на Марсе, известных как «boxwork» — это сложные рельефы, которые могут дать новые данные о геологической истории планеты. Команда марсохода подробно изучила так называемые «rease sign» гребни, чтобы узнать больше о процессах формирования этих структур. Кроме того, NASA активно использует марсоход Perseverance для создания панорам и изучения атмосферы Марса, а Европейское космическое агентство разрабатывает новые технологии для будущих марсианских миссий. Важными остаются

также международные планы по отправке новых аппаратных комплексов для поиска признаков жизни и исследования климата на Марсе.

Ссылки на источники:

- <https://science.nasa.gov/blog/curiosity-blog-sols-4624-4626-a-busy-weekend-at-the-boxwork/>
- <https://www.nasa.gov/missions/mars-2020-perseverance/perseverance-rover/nasas-perseverance-rover-captures-mars-vista-as-clear-as-day/>
- <https://europeanspaceflight.com/esa-to-develop-hummingbird-inspired-flying-mars-explorer/>
- <https://europeanspaceflight.com/italy-signs-agreement-with-spacex-for-starship-mars-mission/>

Шаг 3. Регулярная отправка новостей в чат

Расписание и вебхук

Триггеры

Добавьте блок *Планировщик*:

1. Перейдите к блоку *Триггеры* → *Планировщик* и перетащите его на холст.
2. На холсте дважды нажмите на блок.
3. Перейдите на вкладку *Cron-выражение* и вставьте значение `0 0 9 * * ?`, чтобы запускать процесс каждый день в 9:00 UTC.
4. Нажмите *Сохранить*.

Далее добавьте блок *Вебхук* — перейдите в раздел *Триггеры* и перетащите этот блок на холст. Когда GET-запрос приходит на адрес вебхука, триггер срабатывает и запускает процесс. Это позволяет внешним системам запускать процесс, например, при наступлении какого-то события.

Планировщик

Расписание

Первый запуск
25 апр. 2026 г., 12:00

Cron-выражение
0 0 9 * * ?

Вебхук

URL для запуска триггера
https://agentplatform.just-ai.com/api/text-campaign-service/scenario-triggers/...

Получение новостей

Пользовательская функция

Добавьте новую функцию:

1. В списке блоков выберите *Функции*.
2. Нажмите + справа от раздела *Пользовательские*.
3. Выберите коллекцию *Новости про космос* и нажмите *Далее*.
4. В открывшемся окне укажите информацию о новой функции:
 - *Название*: *Сделать пост*.
 - *Идентификатор*: *sendNews*.
 - *Описание*: *Получить последние новости и составить пост*. По описанию агент будет понимать, что делает функция.
5. Нажмите *Сохранить*. После сохранения откроется окно настроек функции.
6. Функция не имеет параметров, поэтому оставьте их пустыми.
7. В секции *Поля ответа* в поле *Описание* укажите *Текст поста*.
8. Перейдите на вкладку *Код* и введите тело функции. Реализация зависит от **версии движка** в вашем проекте.

Версия 2

Версия 1

```
try {  
    // Получаем дату и время, которые были день назад, в формате ISO  
    var oneDayAgoISO = new Date(Date.now() - 24 * 60 * 60 * 1000).toISOString();  
  
    // Отправляем запрос, чтобы получить статьи, опубликованные за последний день  
    var response = Http.get({
```

```

url: "https://api.spaceflightnewsapi.net/v4/articles/",
params: {
  published_at_gte: oneDayAgoISO,
  ordering: "-published_at"
},
headers: {}
});

var body = typeof response.body === "string" ? JSON.parse(response.body) :
response.body;

var articles = (body.results || []).map(function (a) {
  return {
    title: a.title,
    url: a.url,
    publishedAt: a.published_at
  };
});

// Если статей нет
if (articles.length === 0) {
  return "☁ Сегодня ничего нового";
}

// Формируем текст поста
var postText = "🚀 Статьи про космос за день:\n\n" +
  articles.map(function (a) {
    return "**" + a.title + "**\n" +
      "🔗 " + a.url + "\n" +
      "📅 " + new Date(a.publishedAt).toUTCString();
  }).join("\n\n");

return postText;

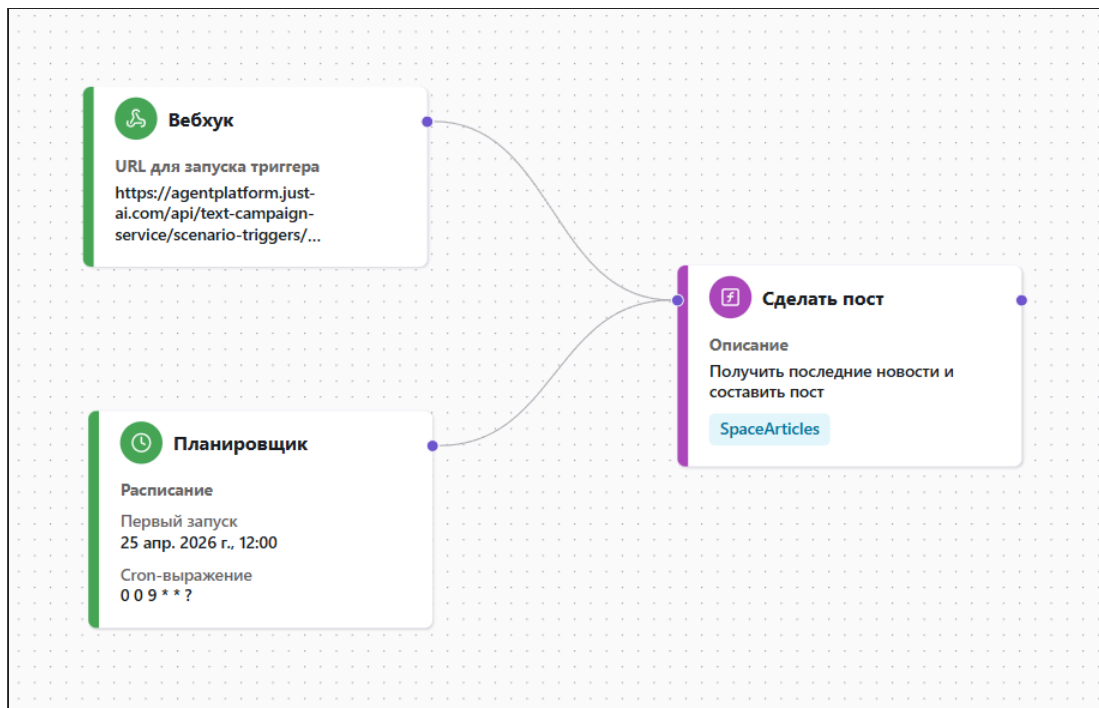
} catch (err) {
  return "Произошла ошибка: " + err;
}

```

Функция получает новости про космос за день из [Spaceflight News API](#) и возвращает текст поста.

9. Нажмите *Сохранить* вверху окна настроек.

Далее перетащите новую функцию из списка блоков на холст и соедините ее с триггерами.



Перевод новостей и отправка сообщения

Встроенные функции

Функция `sendNews` получает заголовки статей на английском языке. Добавьте запрос к LLM, чтобы перевести текст на русский язык:

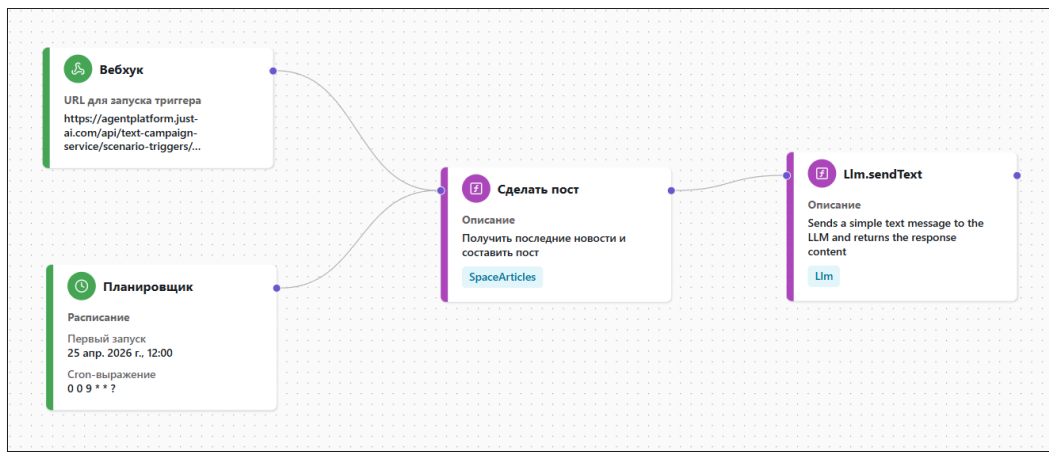
1. В списке блоков откройте раздел *Функции* → *Встроенные* → *Llm* и перетащите функцию `sendText` на холст.
2. На холсте дважды нажмите на блок этой функции.
3. В параметре `llmModelKey` выберите `OPEN_AI — gpt-5.2`.
4. В параметре `text` введите:
 - Для **движка версии 2**: `Переведи пост полностью на русский язык, верни только перевод в ответе: {{ previousResult }}`.
 - Для версии 1: `{{"Переведи пост полностью на русский язык, верни только перевод в ответе: " + Context.getLastFunctionResult()}}`.

💡 ПОДСКАЗКА

`{{}}` используется для подстановки выражений в параметры функций.

Переменная `previousResult` и встроенная функция `Context.getLastFunctionResult()` позволяют получить результат последней выполненной функции. В данном случае это текст сообщения с новостями, который формируется в `sendNews`.

5. Нажмите *Сохранить* и соедините *Сделать пост* с `Llm.sendText`, чтобы эти функции выполнялись последовательно:



Добавьте функцию для отправки сообщения в Telegram

Получите `chatID` Telegram-чата, куда будут приходить новости:

1. Откройте Telegram и найдите бота [@ShowJsonBot](#).
2. Запустите бота и отправьте ему любое сообщение.
3. Бот вернет JSON с данными вашего аккаунта — скопируйте значение поля `"id"`. Это и есть ваш `chatID`.

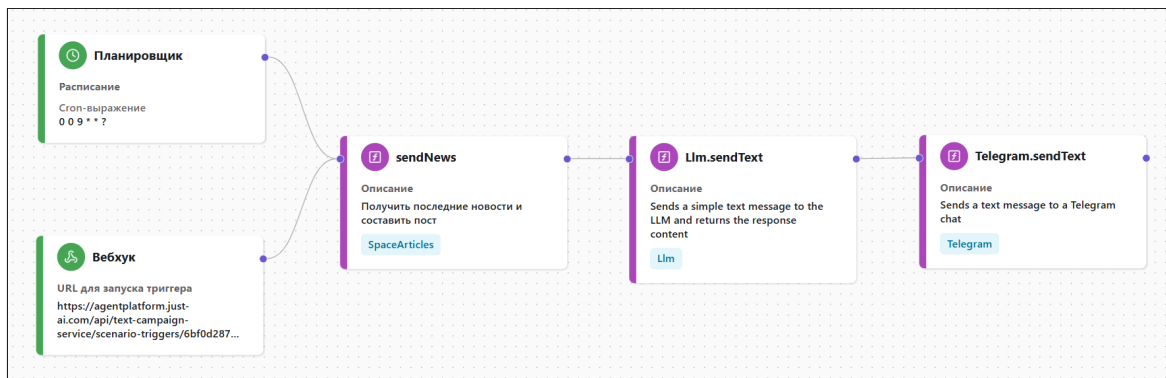
Сохраните токен Telegram-бота в учетных данных:

1. Получите токен бота через [@BotFather](#).
2. В проекте перейдите в раздел *Учетные данные* и **добавьте новые данные** с токеном бота.

Добавьте функцию отправки:

1. В списке блоков откройте раздел *Функции* → *Встроенные* → *Telegram* и перетащите функцию `sendText` на холст.
2. На холсте дважды нажмите на блок этой функции.
3. Укажите параметры:
 - `credentialKey` — выберите созданные учетные данные с токеном бота.
 - `chatId` — `chatID`, который вы получили выше.
 - `text` — для **движка версии 2** укажите `{{ previousResult }}`, для версии 1 — `{{Context.getLastFunctionResult()}}`.
4. Нажмите *Сохранить*.
5. Соедините эту функцию с `Llm.sendText`.

В итоге три функции будут выполняться последовательно:



Тестирование

1. Нажмите *Тестировать* в правом верхнем углу редактора.
2. Выберите триггер для тестирования, например *Вебхук*. Запустится выполнение процесса, вы можете за ним наблюдать в **нижней панели**.
3. В указанный Telegram-чат придет сообщение с последними новостями про космос:

 Статьи про космос за день:

Взгляд на рождение звезд с телескопа «Джеймс Уэбб» от NASA 

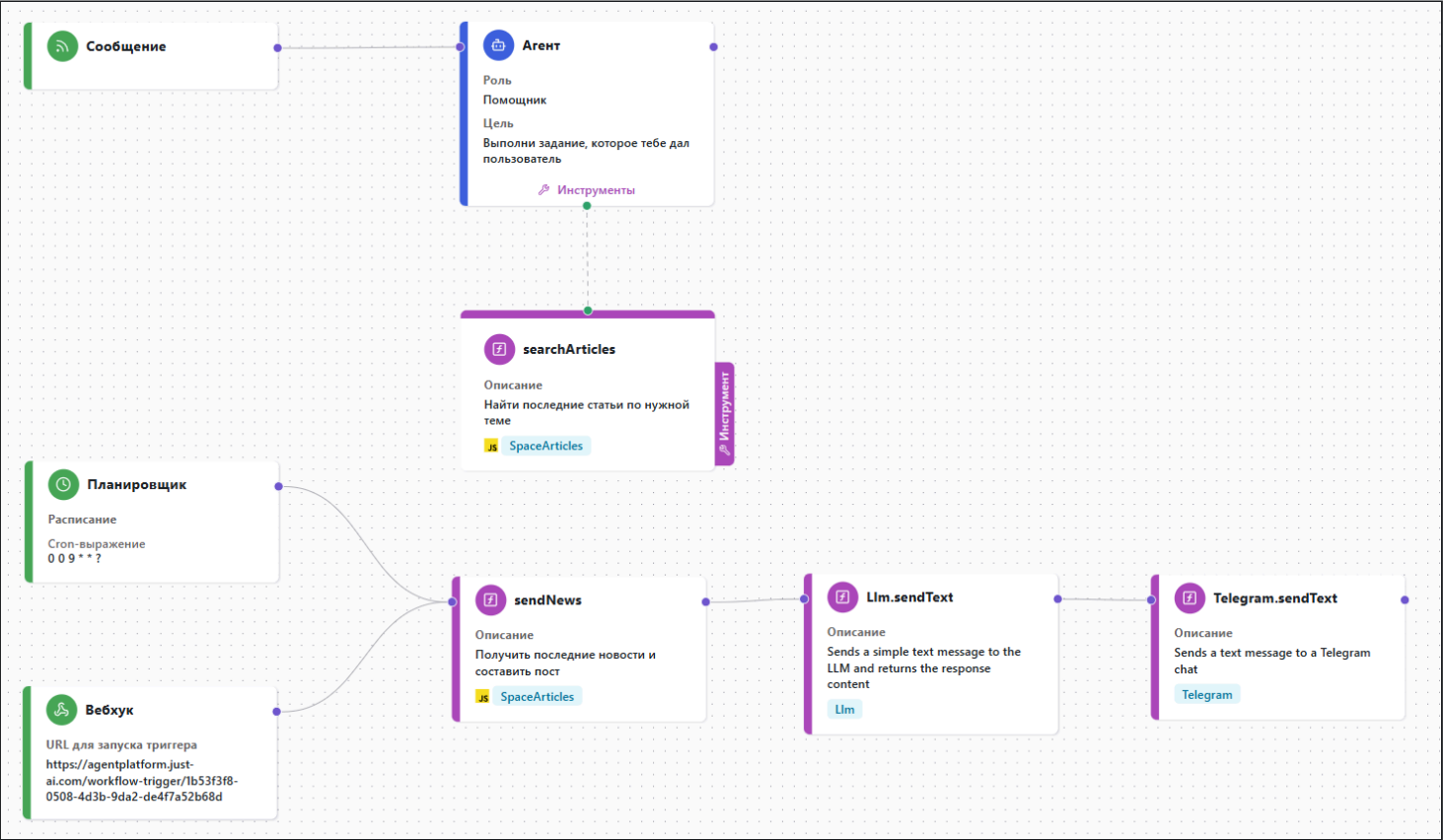
<https://science.nasa.gov/missions/webb/glittering-glimpse-of-star-birth-from-nasas-webb-telescope/>  Чт, 04 сен 2025 14:00:00 GMT

Офицер NASA помогает осуществить возвращение на Луну 

<https://www.nasa.gov/image-article/co-patricia-white-helps-moon-journey/>  Чт, 04 сен 2025 14:00:00 GMT

...

Полный процесс



Дальнейшие шаги

Чтобы пользователи могли общаться с агентом, **подключите канал** к процессу — например, виджет на сайте.

Подходы к построению ИИ-решений

При создании ИИ-систем — цифровых ассистентов, инструментов автоматизации или умных интерфейсов — применяют несколько архитектурных подходов. Каждый из них подходит для определенного типа задач:

- Workflow (рабочий процесс) — линейная, заранее определенная последовательность действий, как в классических чат-ботах.
- **Агентский подход** — логика на базе LLM, которая позволяет адаптироваться к контексту и вызывать внешние инструменты.
- **MAS (Multi-Agent System)** — система из нескольких агентов, объединенных в общую структуру для решения сложных задач.

Пример задачи: Пользователь хочет проанализировать договор и получить краткое резюме с ключевыми условиями. Система должна извлечь из документа важную информацию (стороны, сроки, суммы), проверить ее на корректность и сформировать структурированный отчет.

Рассмотрим, как эта задача решается разными подходами.

Workflow-подход

Решение через workflow:

1. Пользователь загружает файл → проверка формата (только PDF или DOCX).
2. Если формат правильный → извлечение текста.
3. Поиск ключевых слов: «стороны», «сумма», «срок».
4. Заполнение фиксированного шаблона отчета.
5. Отправка результата пользователю.

Workflow работает по жесткому алгоритму: каждый шаг выполняется последовательно, без возможности адаптации к содержимому документа.

Агентский подход

Что такое агент

Агент — это полноценный автономный исполнитель на базе языковой модели (LLM). У агента есть своя цель, логика и набор инструментов. Он не требует жестко заданной цепочки действий, а сам решает, какие шаги предпринять для достижения цели.

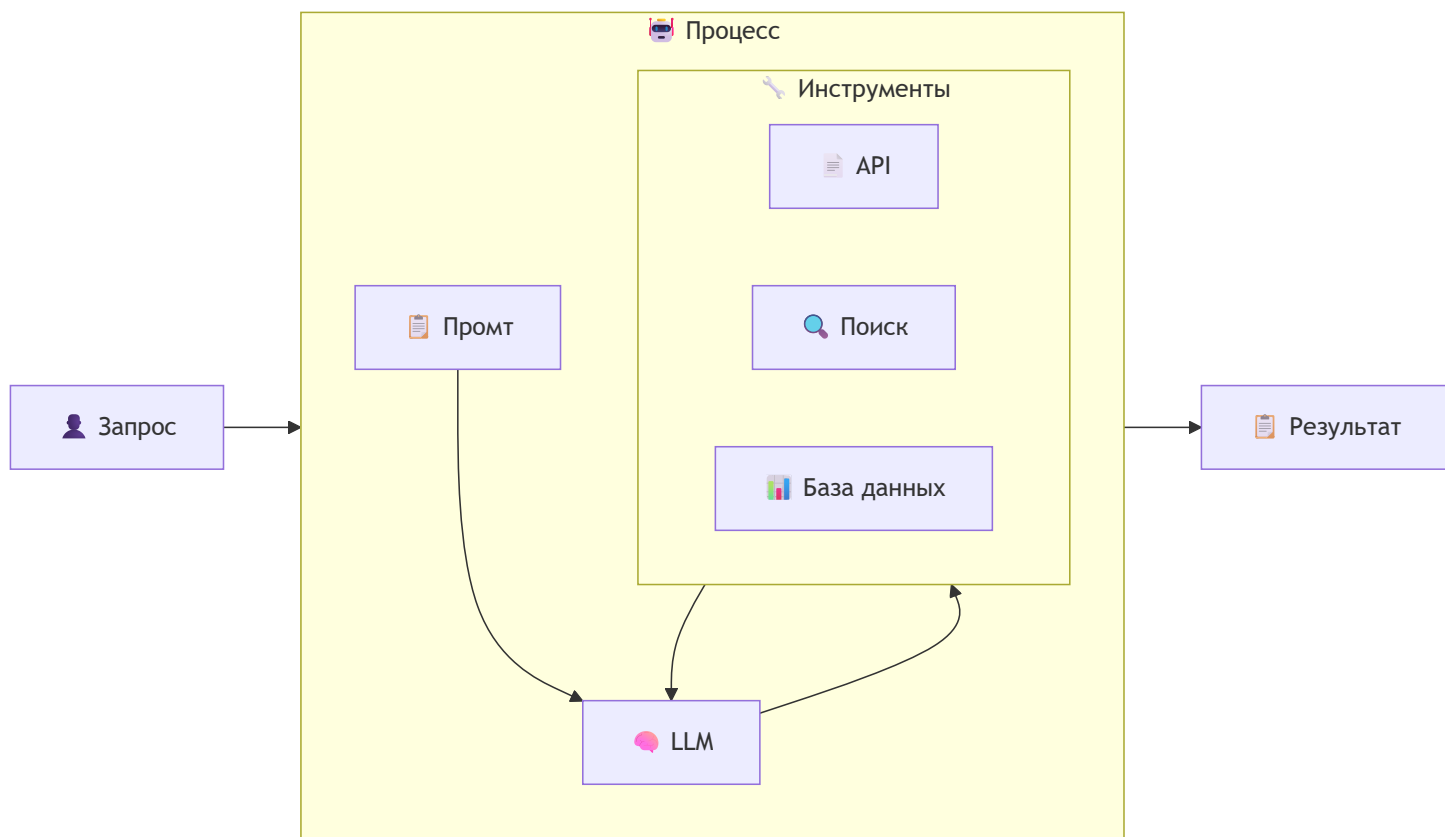
Агент может:

- самостоятельно понимать запрос пользователя;
- определять, какие действия необходимо выполнить;
- вызывать внешние инструменты (функции, API, парсеры);
- уточнять у пользователя детали;
- адаптироваться к различным ситуациям и контексту;
- возвращать осмысленный результат.

Пример реализации агента вы можете увидеть в [Быстром старте](#), а подробную настройку — в [документации по блоку «Агент»](#).

Компоненты агента

Компонент	Назначение
LLM (модель)	«Мозг» агента: интерпретирует запросы и принимает решения.
Промт	Определяет роль агента, стиль ответов и правила работы.
Инструменты (tools)	Функции, которые агент может использовать для взаимодействия с системами, которые делают то, на что LLM не способна самостоятельно: API-запросы, поиск в базах данных, обработка файлов.



Отличия от workflow

Workflow (рабочий процесс) — это заранее определенная последовательность действий и условий, где каждый шаг четко регламентирован. В workflow-системах логика работы описывается через схемы типа «если условие X, то выполнить действие Y», что обеспечивает предсказуемость, но ограничивает гибкость.

Характеристика	Workflow	Агент
Определение действий	Заранее заданная последовательность: «если А, то Б».	Самостоятельно определяет необходимые действия.
Предсказуемость	Высокая — при одинаковых условиях результат всегда одинаковый.	Адаптивная — результат зависит от контекста.
Отладка	Простая — четкие правила и логика.	Сложнее — требует анализа решений модели.
Гибкость	Низкая — плохо справляется с неопределенностью.	Высокая — адаптируется к различным ситуациям.

Характеристика	Workflow	Агент
Понимание запросов	Заранее определенные категории (интенты или другая логика).	Контекстуальная интерпретация.
Последовательность действий	Фиксированная.	Динамическая, в зависимости от задачи.

Агентский подход особенно полезен, когда:

- запросы пользователя разнообразны;
- логика зависит от входных данных;
- существуют разные пути решения задачи;
- невозможно заранее описать все сценарии.

 ПРИМЕЧАНИЕ

На практике агент часто встраивается в workflow. Workflow управляет общим процессом, а агент используется на интеллектуальных этапах: анализ, генерация, поиск.

Рекомендации по настройке агента

При создании агента определите три ключевых аспекта:

Поле	Назначение	Принцип	Пример
Роль	Профессия или специализация агента.	Конкретность, не общие термины.	Аналитик договоров.
Цель	Конкретная задача, которую выполняет агент.	Измеримость, четкие границы.	Извлечь из договора: стороны, сроки, суммы, риски. Вернуть структурированный JSON.

Поле	Назначение	Принцип	Пример
Инструкции	Правила работы, стиль общения, ограничения.	Структурированность, примеры.	Процесс работы, формат ответа, ограничения (пример хорошей инструкции смотрите ниже).

Структура эффективных инструкций:

- 1. Процесс работы — пошаговый алгоритм действий.
- 2. Стиль общения — тон и манера ответов.
- 3. Формат ответа — как структурировать результат.
- 4. Ограничения — что агент не должен делать.

Пример:

Роль: Аналитик договоров
Цель: Проверь, соответствует ли документ стандартам

Инструкции:
ПРОЦЕСС:

- Изучи документ полностью
- Извлеки ключевую информацию (стороны, сроки, суммы)
- Проверь соответствие стандартам компании

ОТВЕТ:

- Краткое резюме
- Список ключевых условий
- Выявленные риски

ОГРАНИЧЕНИЯ:

- Не давай юридических советов
- При сомнениях рекомендуй специалиста

Создание инструментов для агента

Принципы хороших инструментов:

Принцип	Описание	Пример
Атомарность	Один инструмент = одна цель	Поиск документов вместо Поиск документа и рисование картинок в нем
Понятное название	Название отражает действие	Извлечь данные из PDF
Четкое описание	Агент понимает, что делает инструмент	Ищет документы по ключевым словам в базе данных

Рекомендации по описанию инструментов:

- Начинайте с глагола: Извлекает, Анализирует, Отправляет.
- Указывайте источник данных: из базы, из файла, через API.
- Поясняйте формат результата: возвращает список документов, создает отчет в PDF.
- Отмечайте ограничения: работает только с текстовыми файлами.

Пример хорошего описания:

Название: Поиск договоров

Описание: Ищет договоры в корпоративной базе по названию компании, номеру договора или дате. Возвращает список найденных документов с основной информацией. Поддерживает только активные договоры.

Частые ошибки:

- Неясные описания инструментов: обрабатывает данные, работает с файлами.
- Отсутствие ограничений: не указано, с какими форматами работает инструмент.
- Противоречивые инструкции: будь кратким + объясняй подробно.

Пример агента

Решение через агента:

Агент анализирует тот же договор, но адаптируется к его содержанию:

- Получает документ любого формата (PDF, DOCX, изображение).
- Самостоятельно определяет тип договора (поставка, аренда, услуги).
- Извлекает релевантную информацию в зависимости от типа.

4. Проверяет логическую согласованность данных.
5. Формирует отчет в подходящем формате.

Агент самостоятельно:

- выбирает подходящий инструмент для извлечения текста.
- определяет, какая информация важна для данного типа договора.
- решает, нужна ли дополнительная проверка данных.
- адаптирует формат отчета под сложность договора.

Мультиагентная система (MAS)

Что такое MAS

Мультиагентная система (MAS) — это архитектура, в которой несколько агентов работают совместно, каждый выполняя свою задачу. Подробнее о создании мультиагентного процесса вы можете прочитать в [документации](#).

Вместо одного универсального агента создается несколько специализированных. Например:

- один анализирует документы;
- другой извлекает информацию;
- третий формирует документ по шаблону;
- четвертый отправляет результат.

Преимущества:

- Упрощение разработки — каждый агент отвечает за свою задачу.
- Гибкость — изменения можно вносить точечно в конкретного агента.
- Масштабируемость — систему легко расширять, добавляя новых агентов.
- Повторное использование — агентов можно применять в других сценариях.

Почему не один универсальный агент

Может показаться, что проще создать одного агента с объемной инструкцией. Этот подход работает для простых задач, но в сложных сценариях он неэффективен. Если логика разветвленная, инструкции противоречат друг другу или нужно подключать новых исполнителей, универсальный агент начинает работать нестабильно:

- появляются «галлюцинации»;

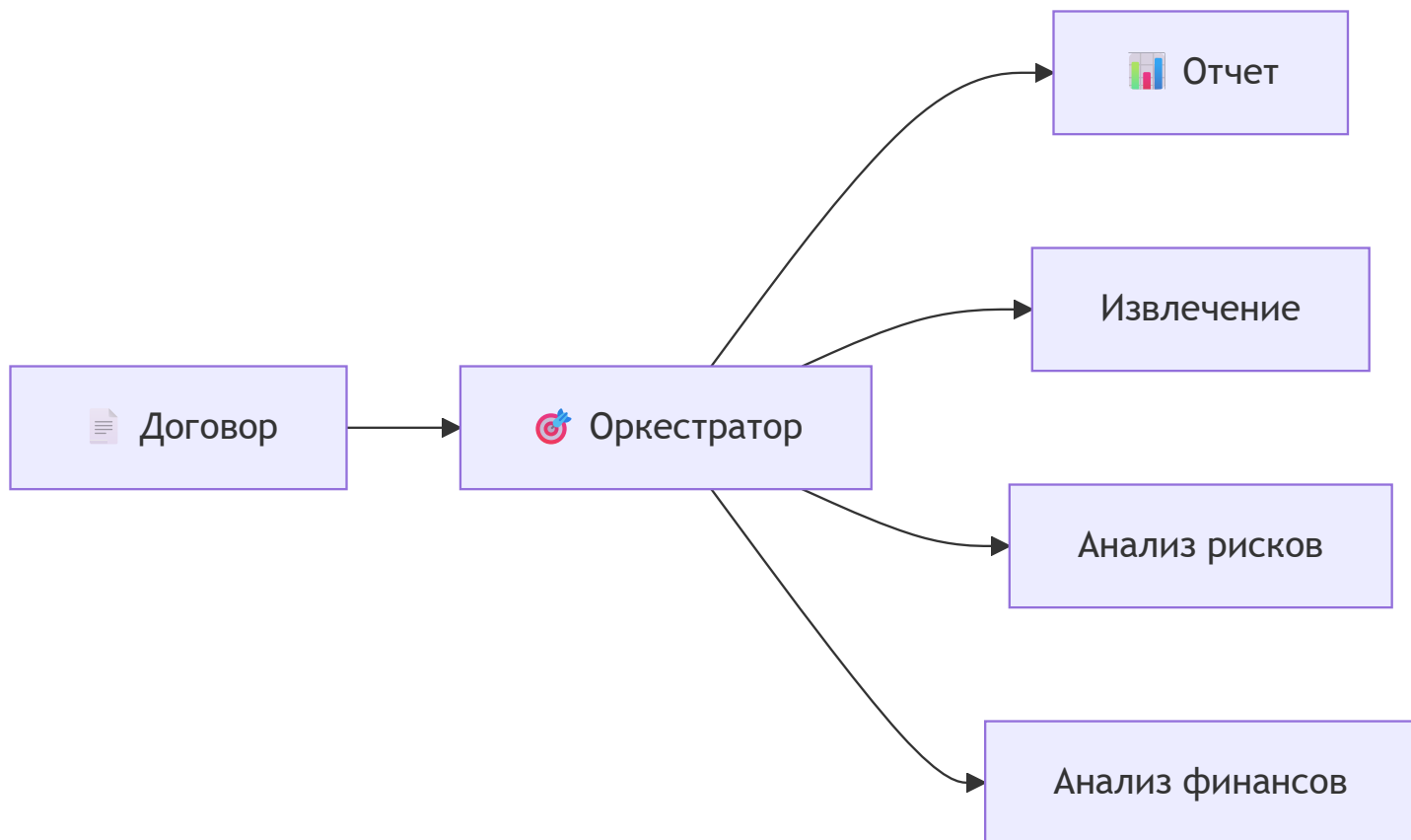
- ошибки сложно диагностировать;
- инструкция становится слишком объемной и противоречивой.

Сравнение: один агент vs. MAS

	Один агент	MAS
Логика	Общая, в рамках одного промта.	Разделена между отдельными агентами.
Поддержка	Усложняется при росте требований.	Легче изменять отдельных агентов.
Масштабируемость	Ограничена размером промта.	Легко добавить нового агента.
Переиспользуемость	Низкая.	Высокая.
Надежность	Зависит от одной модели.	Ошибка локализуется в рамках одного агента.

Как устроена MAS

Координация агентов обычно происходит через workflow или специального агента-оркестратора, который управляет остальными.



Когда использовать MAS

Использовать MAS, если:

- У задачи много этапов с разной логикой.
- Требуются разные навыки (поиск, генерация, разметка).
- Сценарий может меняться или расширяться.
- Разные специалисты могут разрабатывать отдельных агентов.

Не использовать MAS, если:

- Один агент хорошо справляется с задачей.
- Нет жестких требований к масштабированию.
- Процесс достаточно однообразный и не требует экспертизы в разных областях.

❗ ПРИМЕЧАНИЕ

Важно выбирать MAS только когда это действительно нужно. Если задача простая, один агент будет эффективнее и проще в поддержке.

Пример MAS

Решение через MAS:

Для анализа договора создается команда из трех специализированных агентов:

- 1. **Агент извлечения** — обрабатывает файл и извлекает текст.
- 2. **Агент анализа рисков** — ищет проблемные пункты и юридические риски.
- 3. **Агент анализа финансов** — проверяет суммы, валюты и условия оплаты.

Роль оркестратора:

- Координирует работу агентов (сначала извлечение, затем параллельный анализ).
- Решает, какие агенты нужны в зависимости от типа договора.
- Объединяет результаты всех агентов в единый отчет.
- При обнаружении противоречий запрашивает повторный анализ.

Например, если агент рисков находит проблему с суммой, оркестратор может попросить агента финансов дополнительно проверить именно этот пункт.

Сравнение подходов на примере

Сравним все подходы на примере задачи по обработке документов.

Аспект	Workflow	Агент	MAS
Гибкость к формату	Только PDF/DOCX	Любой формат, поддерживаемый платформой	Любой формат, поддерживаемый платформой
Адаптация к типу договора	Одинаково для всех	Универсальный подход	Специализированные агенты под тип
Параллельная обработка	Последовательная	Нет	Да (аналитики работают одновременно)
Обработка неясностей в тексте	Пропускает или выдает ошибку	Пытается интерпретировать самостоятельно	Привлекает нужного эксперта

Аспект	Workflow	Агент	MAS
Разрешение противоречий	Не предусмотрено	Ограниченное	Оркестратор координирует решение
Сложность внедрения	Простая	Средняя	Высокая
Качество анализа	Базовое	Хорошее	Экспертное (каждый агент — специалист)
Масштабируемость	Ограниченная	Хорошая	Отличная (легко добавить экспертов)

Вывод: Выбор подхода зависит от требований к качеству, разнообразия задач и ресурсов на разработку.

МСП-серверы в агентных системах

Современные модели искусственного интеллекта обладают огромным потенциалом, но часто остаются изолированными от данных и внешних систем. **МСП-серверы** (от англ. Model Context Protocol) решают эту проблему, выступая в роли «органов чувств» и «рук» для AI, открывая путь к созданию по-настоящему автономных систем.

Зачем нужны МСП-серверы

Главная задача МСП-сервера — подключать искусственный интеллект к внешним системам, базам данных и сервисам. Он работает по открытому протоколу Model Context Protocol (МСП), который позволяет AI-моделям получать актуальные данные и выполнять действия, снижая необходимость в индивидуальных интеграциях.

Это помогает преодолеть изоляцию AI. Вместо того чтобы опираться только на обучающие данные, модель получает доступ к информации в реальном времени, что повышает ее контекстную осведомленность и эффективность. МСП-серверы становятся ключевым инструментом для интеграции AI с корпоративными системами, базами данных и сервисами.



Без МСП-сервера: Изолированная модель



С МСП-серверами: Расширенная модель

Чем МСП-сервер отличается от обычного API

На первый взгляд МСП-сервер напоминает API, но между ними есть принципиальные различия. МСП — это не просто набор эндпоинтов, а интеллектуальный посредник, работающий по единому протоколу, специально разработанному для взаимодействия AI-моделей с внешним миром.

Ключевые отличия:

- **Стандартизация.** MCP описывает функции как именованные действия с единым JSON-форматом, тогда как обычные API используют разные HTTP-методы и структуры запросов.
- **Динамическое обнаружение.** Клиент (оркестратор) может программно получить список доступных инструментов у MCP-сервера, без жестко заданных эндпоинтов.
- **Безопасность.** MCP-сервер хранит чувствительные данные (например, ключи доступа) локально и не передает их модели, повышая уровень защиты.
- **Двустороннее взаимодействие.** AI может не только получать данные, но и инициировать действия через сервер под контролем пользователя.
- **Локальная работа.** MCP может функционировать без доступа к сети, что упрощает интеграцию и повышает безопасность.
- **Простота разработки.** Единый протокол облегчает подключение новых источников и инструментов, снижая объем кастомных интеграций.

Иными словами, MCP — это унифицированный слой абстракции над API, адаптированный под нужды моделей и автономных агентов с упором на безопасность и масштабируемость.

Преимущества MCP для автономных агентов

Для автономных агентов MCP-серверы дают ряд преимуществ, повышающих их функциональность и адаптивность:

- **Универсальное подключение.** Агентам не нужно знать детали каждого API — MCP предоставляет единый способ доступа к различным источникам данных и инструментам.
- **Доступ к данным в реальном времени.** Агенты получают актуальную информацию, необходимую для принятия решений и адаптации к меняющимся условиям.
- **Возможность действовать.** MCP позволяет агентам выполнять действия: обновлять документы, запускать транзакции, управлять смарт-контрактами и др.
- **Повышенная безопасность.** Доступы и ключи хранятся на стороне MCP-сервера, снижая риски. Уровень защиты зависит от конкретной реализации.
- **Координация агентов.** MCP может быть общим слоем доступа к инструментам и данным для нескольких агентов при наличии координации.
- **Приватность.** MCP поддерживает создание персональных AI-ассистентов, интегрированных с локальными приложениями и данными, без передачи информации внешним серверам в локальных конфигурациях.

Кейсы использования MCP

На практике MCP уже показывает свою ценность в разных сферах:

- **Корпоративные ассистенты.** AI-агенты подключаются к **Jira** и **Confluence** через MCP, чтобы отвечать на вопросы о задачах и спецификациях, ускоряя работу команд.
- **Анализ данных.** Имея доступ к БД, агенты переводят запросы на естественном языке в SQL, выполняют их и возвращают готовые результаты, упрощая аналитику.
- **Интеграция с CRM.** AI-ассистенты получают данные о клиентах и сделках, помогая персонализировать общение и автоматизировать рутину.
- **Персональные помощники.** **Claude Desktop** поддерживает MCP для безопасного доступа к локальным файлам и приложениям, минимизируя передачу данных при корректной настройке.
- **Изолированные среды.** MCP позволяет создавать автономные AI-системы без доступа к интернету, что особенно важно для организаций с высокими требованиями к безопасности.

Таким образом, MCP-серверы — это ключевая технология, позволяющая AI-системам выйти за рамки изоляции и эффективно взаимодействовать с реальным миром.

Обзор конструктора процессов

Конструктор процессов в Agent Platform — это визуальный low-code инструмент, разработанный с фокусом на простоту использования. Благодаря интуитивному drag-and-drop интерфейсу, он не требует глубоких технических знаний. Это делает конструктор доступным даже для менее подготовленных пользователей.

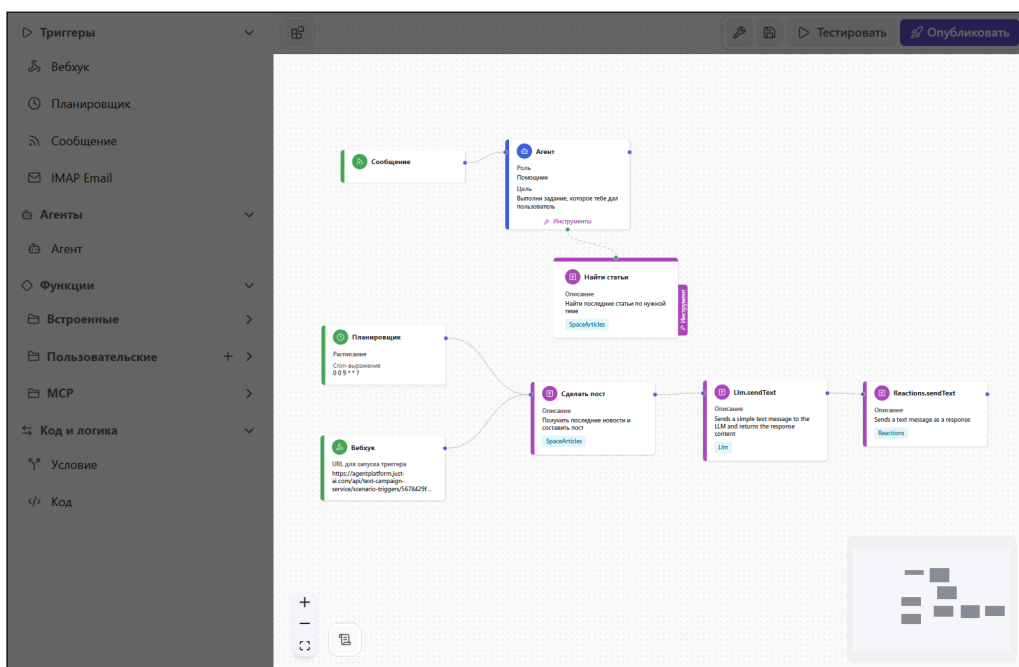
Технические специалисты могут расширить возможности конструктора с помощью встроенных и пользовательских функций, **интеграций с MCP-серверами** и блоков кода.

После создания проекта AI-приложения вы увидите интерфейс с основными компонентами: **холст**, **список блоков** и **панель управления**.

Компоненты интерфейса

Холст

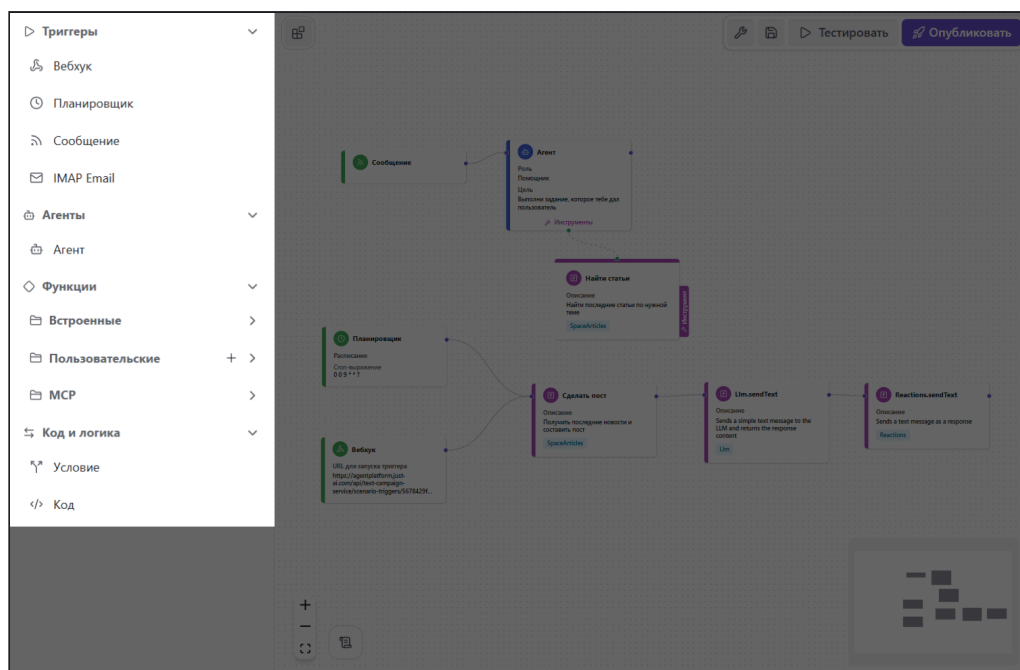
Холст — это основное рабочее пространство, на котором вы выстраиваете процесс, **размещая и соединяя функциональные блоки**.



Список блоков

На панели слева находится список функциональных блоков. Блоки — это компоненты вашего процесса, которые принимают, обрабатывают и передают данные между различными частями вашего AI-приложения.

Чтобы развернуть список блоков, нажмите  в левом верхнем углу холста.



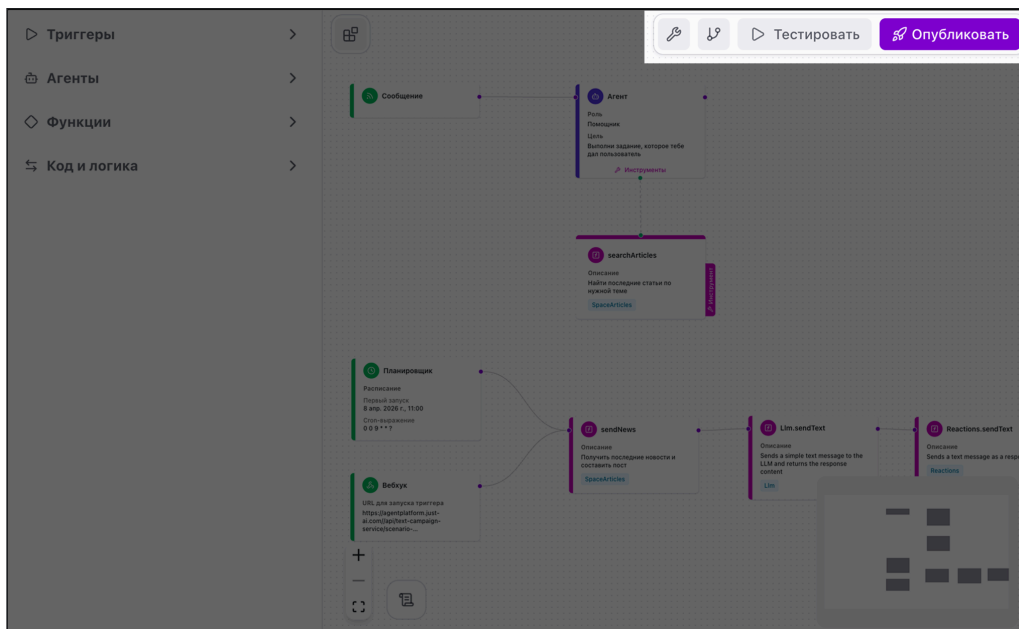
Основные категории блоков:

- *Триггеры*: запускают процесс в ответ на определенные события. Вы можете настроить запуск по вебхуку, при получении сообщения ассистентом или с определенной периодичностью (планировщик). Подробнее в разделе [Триггеры](#).
- *Агенты*: позволяют интегрировать и использовать возможности различных AI-моделей от множества провайдеров, таких как OpenAI, Anthropic, Google и других. Подробнее в разделе [Агент](#).
- *Функции*: предоставляют функции для обработки данных. Вы можете выбрать из доступных встроенных функций, создать свои собственные или подключить MCP-серверы, которые предоставляют функции нужных сервисов.
- *Код и условия*: позволяют добавлять пользовательский JavaScript-код и логические условия для сложной обработки данных.

Панель управления

Панель управления в правом верхнем углу позволяет:

- просматривать ошибки;
- [управлять версиями](#);
- [тестировать процесс](#);
- [публиковать процесс](#).



Основные сценарии

Работа с блоками

В конструкторе можно работать как с отдельными блоками, так и с группами блоков.

Отдельные блоки

С блоками можно выполнять следующие действия:

- Добавлять на холст:
 - Перетащите один блок из боковой панели.
 - Перетащите **коллекцию** функций, если хотите добавить сразу все ее функции на холст.
- Редактировать. Откройте настройки блока одним из способов:
 - Выберите блок и нажмите .
 - Дважды нажмите на блок.
 - Нажмите на блок правой кнопкой мыши и выберите *Редактировать*.
- Перемещать: перетащите блок в нужное место на холсте.
- Удалять. Используйте один из способов:
 - Выберите блок и нажмите .
 - Выберите блок и нажмите `Delete` или `Backspace`.
 - Нажмите на блок правой кнопкой мыши и выберите *Удалить*.

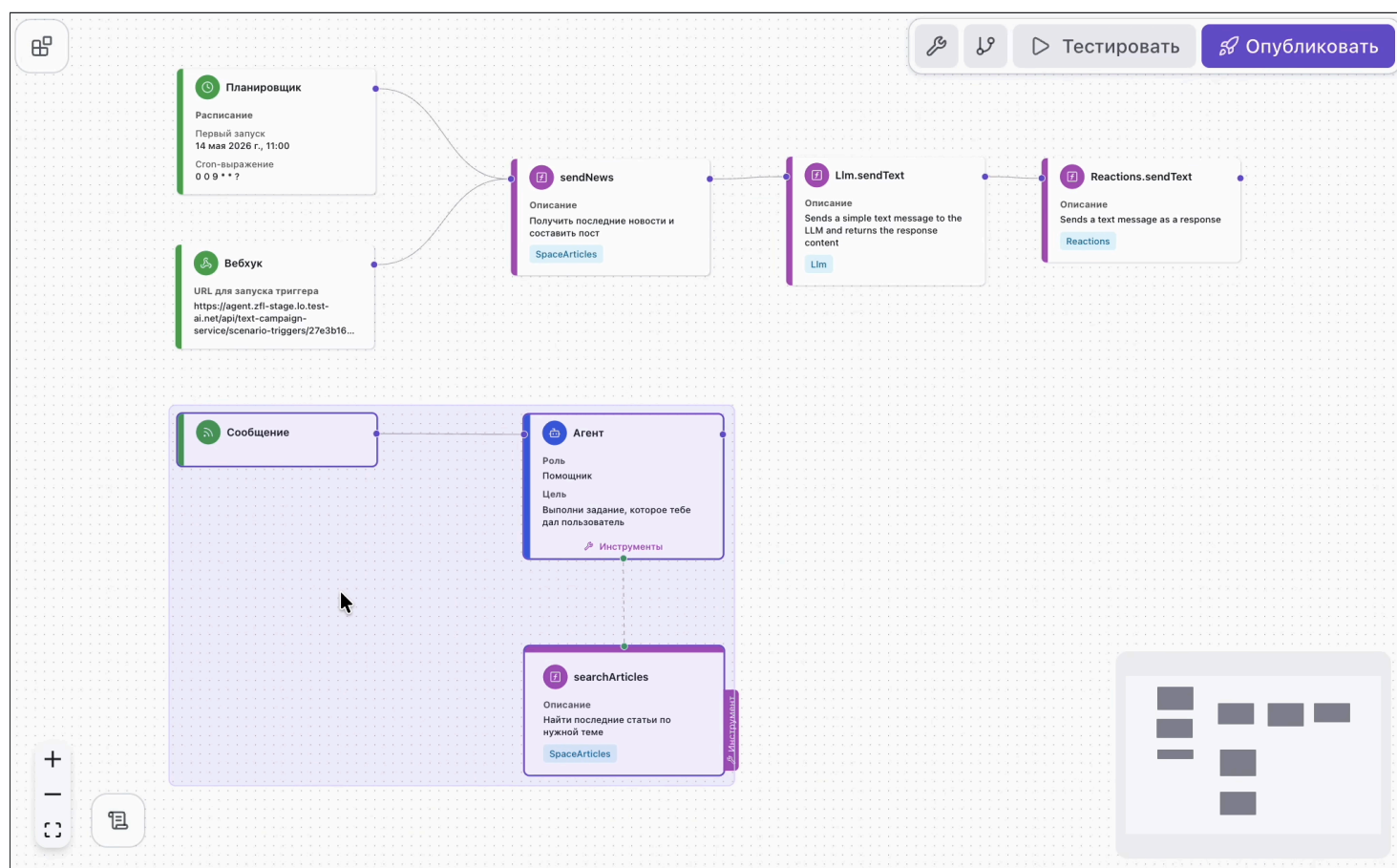
Группы блоков

Группы блоков помогают быстрее перестраивать процесс: вы можете выделить несколько блоков и выполнить действие сразу со всей группой.

Чтобы выбрать несколько блоков, зажмите **Shift** и выделите их на холсте.

С группой блоков можно выполнять следующие действия:

- Перемещать: перетащите выделенную группу в нужное место на холсте.
- Удалять. Используйте один из способов:
 - Нажмите **Delete** или **Backspace**.
 - Нажмите на выделенную группу правой кнопкой мыши и выберите *Удалить*.



Связи между блоками

Связи устанавливают порядок выполнения блоков. Чтобы связать блоки между собой, наведите курсор на маленький кружок блока-источника и перетащите линию к кружку блока-приемника.

Для блока *Агент* также доступны связи с другими блоками в качестве инструментов. Они позволяют агенту использовать дополнительные функции для обработки запросов.

Последовательность соединения инструментов с агентом не имеет значения.

Чтобы удалить связь, нажмите на нее, а затем `Delete` или `Backspace`. Перетаскивать связи между блоками нельзя.

Тестирование процесса

Перед публикацией процесс можно протестировать прямо в конструкторе: запускайте тест с любого триггера, общайтесь с ассистентом в чате и смотрите подробную информацию о выполнении блоков.

Подробнее смотрите в статье [Тестирование процесса](#).

Сохранение и версии

Работа с версиями зависит от того, как хранится ваш проект: внутри платформы или во внешнем Git-репозитории.

ПРЕДУПРЕЖДЕНИЕ

Вы можете выбрать способ хранения только при создании проекта. Изменить его позже нельзя.

Внутри платформы

По умолчанию проект хранится внутри платформы.

- Чтобы сохранить текущее состояние проекта, нажмите  → *Сохранить* в правом верхнем углу.
- Чтобы посмотреть список сохраненных версий, нажмите  → *История версий*. Вы можете:
 - сравнить текущее состояние с любой из предыдущих версий;
 - восстановить один или несколько файлов из старой версии.

Пользователи с [доступом к аккаунту](#) могут открывать и редактировать такой проект. Если над проектом работает одновременно несколько человек, обычного доступа к аккаунту недостаточно — есть риск случайно перезаписать чужие изменения.

В Git-репозитории

Если вам нужна совместная работа над проектом, прозрачная история изменений или интеграция с CI/CD, подключите его к [Git-репозиторию](#). Git дает прозрачный процесс работы с изменениями: pull и push, историю версий и разрешение конфликтов в специальном редакторе.

Подробнее о работе и настройках смотрите в статье [Подключение Git-репозитория](#).

Публикация процесса

Чтобы сделать процесс доступным для внешнего использования, например в каналах:

1. Сохраните текущую версию процесса:
 - i. Нажмите  → *Сохранить* или *Commit & push*.
 - ii. Введите описание изменений и подтвердите сохранение.
2. Нажмите *Опубликовать*.

Подключение Git-репозитория

Если вам нужна командная работа, прозрачная история изменений или интеграция с CI/CD, подключите к проекту внешний Git-репозиторий.

⚠ ПРЕДУПРЕЖДЕНИЕ

Вы можете подключить репозиторий только при создании проекта. У существующего проекта способ хранения изменить нельзя.

Подключение при создании проекта

При создании проекта:

1. Выберите *Где хранить проект? — В Git-репозитории*.
2. Создайте репозиторий в любом Git-провайдере (например, GitHub или GitLab) и укажите его адрес в *URL репозитория*.
3. Укажите *Логин* и *Персональный токен доступа* для авторизации. Токен должен иметь права на чтение и запись в репозиторий. Agent Platform создаст запись с вашим токеном в разделе *Учетные данные*.
4. Выберите существующую ветку или создайте новую.
5. По умолчанию Agent Platform будет использовать файлы из вашего репозитория как начальное состояние проекта. Если вы выберете шаблон, он перезапишет файлы в ветке.

Информация о подключении будет доступна на левой панели в разделе *Настройки проекта → Хранение*.

Действия с репозиторием

Нажмите  в правом верхнем углу:

- Чтобы сохранить изменения и отправить их в репозиторий, выберите *Commit & push*. Вы сможете посмотреть список своих изменений и добавить описание перед отправкой.
- Чтобы получить изменения из подключенной ветки, выберите *Pull*.
- Чтобы посмотреть список версий, нажмите *История версий*. Вы можете:
 - сравнить текущее состояние с любой из предыдущих версий;
 - восстановить один или несколько файлов из старой версии.

Если изменения появились и в платформе, и в репозитории одновременно, возникнет конфликт. Agent Platform предупредит об этом — вы сможете выбрать, какую версию оставить: текущую или из репозитория. Вы принимаете версию целиком: выбрать отдельные файлы или строки нельзя.

Какие данные попадают в репозиторий

В репозитории сохраняется все, кроме:

- Записей в **базах данных**. Сами интеграции с базами сохраняются, но содержимое хранится только внутри платформы.
- Токенов, паролей и другой конфиденциальной информации из **учетных данных**. Сами записи об учетных данных сохраняются, но их секретные значения в репозиторий не попадают. Поэтому, если позже создать новый проект из этого репозитория, секретные данные нужно будет ввести заново.

Управление сессией

Сессия — последовательность запросов пользователя и ответов процесса, объединенных общим контекстом. В рамках одной сессии процесс может учитывать предыдущие сообщения пользователя и ответы агента, а также хранить временные данные.

В сессии доступны:

- **История сообщений** — сообщения пользователя и ответы агента в текущей сессии. Эту историю можно получить с помощью функции `Context.getChatHistory()`.
- **Сессионные данные** — временные данные, которые хранятся только в рамках **текущего диалога**. Для работы с ними используйте функции из коллекции `SessionDb`.
- **Идентификатор сессии** — функция `Context.getSessionID()` позволяет получить идентификатор текущей сессии.

Сессия в телефонном канале

Один звонок — это одна сессия. Платформа управляет сессией автоматически:

- В начале звонка создается новая сессия.
- Сессия завершается при окончании звонка.

Сброс сессии в телефонном канале игнорируется.

Ручное управление сессией

Новая сессия нужна, когда процесс должен начать диалог заново и не учитывать данные предыдущей сессии. Например, это полезно при тестировании процесса или когда пользователь хочет начать новый сценарий взаимодействия.

При сбросе запускается новая сессия, очищаются данные `SessionDb` и история сообщений.

Вы можете сбросить сессию одним из способов:

- В процессе вызовите функцию `Context.resetSession()`. Функция доступна только для проектов с **движком версии 2**.
- В **тестовом виджете** нажмите .

Передача управления другому проекту

Продвинутая возможность

Из процесса можно передать управление другому процессу Agent Platform или сценарию **JAICP**, который далее ведет диалог с клиентом, а после завершения своей задачи может вернуть управление исходному процессу.

Например, вы можете использовать основного бота как диспетчера, который определяет намерение пользователя и передает управление специализированному боту для оформления заказа или технической поддержки.

Как настроить

[Переход в JAICP](#)

[Переход в Agent Platform](#)

Чтобы передать управление из процесса Agent Platform в сценарий JAICP:

- Опубликуйте сценарий JAICP в любом входящем канале, например чат-виджете, и скопируйте ID бота:
 - Нажмите на панели управления *Каналы* → *Входящие* → *Подключить канал*. Выберите канал *Чат-виджет* и нажмите *Создать*.
 - Скопируйте *botId* из свойств канала.
- В исходном проекте Agent Platform добавьте вызов функции `ProjectRouter.switchTo()`.
Функцию можно использовать как отдельный шаг в процессе, как инструмент для AI-агента или вызвать в коде других блоков.
Задайте параметры:
 - botId* — вставьте ID целевого проекта.
 - state* — стейт, в который нужно перейти в целевом проекте (необязательно, по умолчанию `/`).
 - customData* — JSON-объект с дополнительными данными (необязательно).

⚠ ПРЕДУПРЕЖДЕНИЕ

Диалог не возвращается обратно, пока в вызываемом проекте не будет выполнена функция `backToOriginalProject()`.

Какие данные получает вызываемый проект

[Переход в JAICP](#)

[Переход в Agent Platform](#)

Сценарию JAICP доступны следующие данные из исходного процесса:

Параметр	Как получить
Сообщение клиента	<code>\$request.query</code>
Полные данные запроса	<code>\$request.rawRequest</code>
<code>customData</code>	<code>\$request.data</code>

Возврат управления

Возврат управления происходит в начало («корень») исходного процесса Agent Platform. Выбрать конкретный блок невозможно. После возврата исходный процесс не выполняет никаких действий, а ожидает срабатывания триггера — например, сообщения от клиента.

[Возврат из JAICP](#)

[Возврат из Agent Platform](#)

Для возврата из сценария JAICP в исходный процесс используйте метод

`$agentPlatform.backToOriginalProject()`.

❗ ПРИМЕЧАНИЕ

У метода `$agentPlatform.backToOriginalProject()` есть необязательный параметр `customData`, но в исходном процессе Agent Platform эти данные недоступны.

Особенности и ограничения

- **Общий аккаунт:** передача управления возможна только между проектами одного аккаунта.
- **Защита от циклических вызовов:** чтобы предотвратить бесконечные переходы, установлено ограничение: **не более 6** последовательных переключений в рамках одного запроса пользователя. При превышении лимита диалог будет прерван.
- **Одно переключение на блок:** избегайте следующих конструкций, они могут привести к нестабильной работе:
 - два вызова `switchTo()` из одного блока;
 - два вызова `backToOriginalProject()` из одного блока;
 - вызов `switchTo()` и `backToOriginalProject()` из одного блока.

Пример

Процесс Agent Platform принимает заказ у клиента и передает его в сценарий JAICP для обработки. Для передачи заказа используется функции `ProjectRouter.switchTo()` с параметрами:

- *botId*: ID бота JAICP, например `12345-jaicp-bot-12345`
- *state*: `/HandleOrder`
- *customData*: `{ "productId": "product-123", "amount": 7 }`

Сценарий JAICP обрабатывает заказ и возвращает управление в Agent Platform:

```
theme: /

state: HandleOrder
script:
    $context.productId = $request.data.productId;    // "product-123"
    $context.amount = $request.data.amount;          // 7

    // Логика обработки
    ...

    // Возврат управления
    $agentPlatform.backToOriginalProject();
```

Вызов другого проекта

Продвинутая возможность

Из процесса можно вызвать другой процесс Agent Platform или сценарий **JAICP**. Такой вызов работает как запрос к внешнему сервису: ответ вызываемого процесса не отправляется пользователю, а возвращается в родительский процесс и используется в нем.

Эта функция позволяет делегировать вызываемым проектам выполнение специфических задач — например, извлечение именованных сущностей (NER).

Как настроить

Вызов сценария JAICP

Вызов процесса Agent Platform

Чтобы вызвать сценарий JAICP:

1. В вызываемом проекте JAICP настройте входящий канал Chat API и получите токен:
 - i. Нажмите на панели управления *Каналы* → *Входящие* → *Подключить канал*. Выберите канал *Chat API* и нажмите *Создать*.
 - ii. Перейдите в окно редактирования созданного канала *Chat API* и скопируйте значение поля *Токен*.

Подробная информация приведена в разделе **Chat API** документации JAICP.

2. В родительском проекте Agent Platform добавьте вызов функции `ProjectRouter.callProject()`. Функцию можно использовать как отдельный шаг в процессе, как инструмент для AI-агента или вызвать в коде других блоков.

Задайте параметры:

- *token* — вставьте токен Chat API вызываемого проекта.
- *query* — запрос к вызываемому проекту.
- *customData* — JSON-объект с дополнительными данными (необязательно).

Какие данные получает вызываемый проект

❗ К СВЕДЕНИЮ

Вызываемый процесс или сценарий получает **изолированный контекст** и не имеет доступа к сессии родительского процесса.

Вызов сценария JAICP

Вызов процесса Agent Platform

Сценарию JAICP доступны следующие данные:

Параметр	Как получить
Идентификатор клиента	<code>\$request.data.chatId</code>
<code>query</code>	<code>\$parseTree.text</code>
<code>customData</code>	<code>\$request.data</code>

Возвращаемые данные

Функция `ProjectRouter.callProject()` возвращает ответ в формате, аналогичном [ответу Chat API](#).

Особенности и ограничения

- **Отправка ответов:** чтобы родительский проект мог получить ответ, все реакции в вызываемом процессе или сценарии не уходят в канал пользователю, а возвращаются в теле ответа.
- **Защита от циклических вызовов:** если проект А вызывает проект Б, а проект Б снова вызывает проект А, процесс прервется с ошибкой.
- **Тарификация:** вызов JAICP из Agent Platform тарифицируется в JAICP (если пользователь новый).

Пример

Рассмотрим сценарий JAICP, который выделяет дату и время из запроса:

```
theme: /
```

```
state: Datetime
```

```
q!: * @duckling.time::extractedTime *
```

```
script:
```

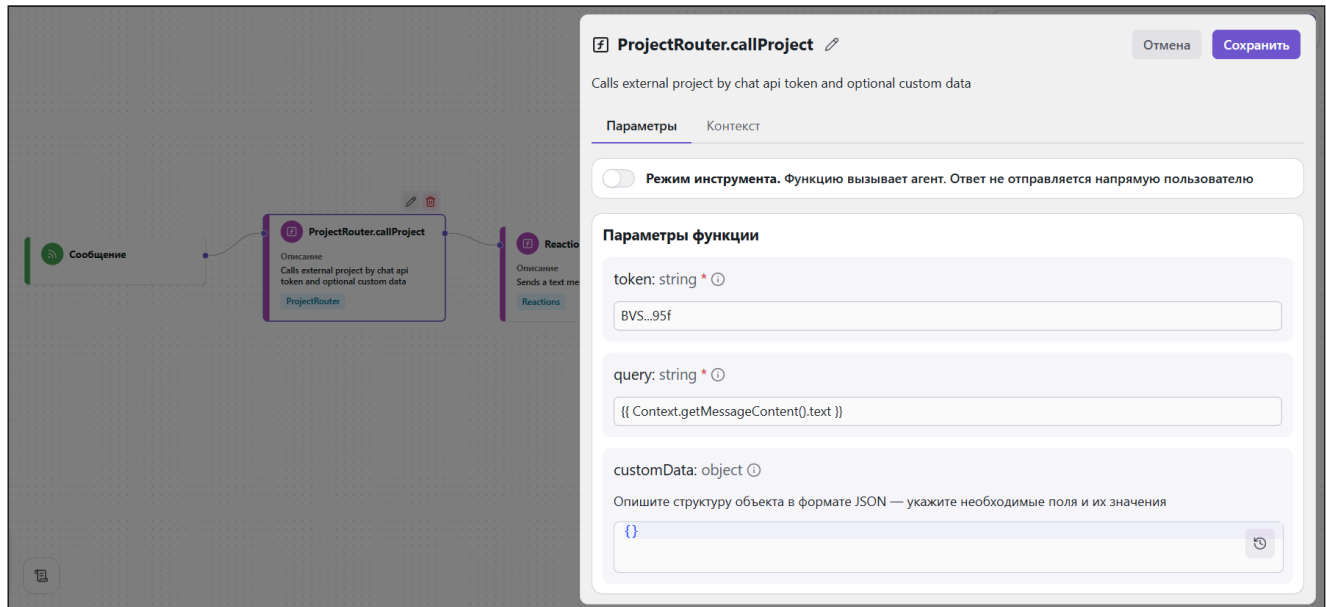
```
$temp.time = $parseTree._extractedTime.value;  
$reactions.answer($temp.time);
```

```
state: NoMatch
```

```
event!: noMatch
```

```
a: Я не нашел время и дату в запросе
```

Добавим в процесс функцию `ProjectRouter.callProject()` с параметрами:



Получив запрос клиента, например, «послезавтра в полдень», функция вызовет указанный сценарий JAICP и вернет следующие данные:

```
{  
  "token": "BVS...95f",  
  "clientId": "05be5c1d-0aba-1f54-edcc-d2c42cf061b0",  
  "questionId": "227471ad-b78f-4479-8dce-c21c3e1464ac",  
  "data": {  
    "replies": [{  
      "type": "text",  
      "text": "2026-03-03T12:00:00",  
      "state": "/Datetime",  
      "lang": "ru"  
    }  
  ],  
  "answer": "2026-03-03T12:00:00",  
  "newSessionStarted": false,  
  "transitionHistory": ["/Datetime"],  
  "isNewClient": false,  
  "isNeedToSaveToReporter": true,  
  "endSession": false,  
  "messageTimestamp": 1772346816951  
},  
  "timestamp": "2026-03-01T06:33:36.955",  
  "blockForm": false,
```

```
"customData": {}
```

```
}
```

Тестирование процесса

Как запустить тест

1. Нажмите *Тестировать* в правом верхнем углу.
2. В выпадающем меню выберите триггер, с которого хотите запустить процесс.

Сообщение

Если тест начинается с триггера *Сообщение*, то платформа покажет тестовый виджет. Переписывайтесь с ассистентом в чате и изучайте, как процесс реагирует на ваши сообщения.

Более подробная информация о шагах процесса отображается в [панели выполнения](#).

Отправлять /start автоматически

`/start` — системное сообщение, которое отправляется в начале диалога. Включите, чтобы агент первым начинал разговор при открытии виджета:

1. Нажмите  в заголовке виджета.
2. Включите *Отправлять /start при запуске виджета*.

Настройка срабатывает при каждом открытии виджета, по кнопке «Новая сессия»  и «Начать тест заново» .

Другие триггеры

Если тест начинается с другого триггера, то платформа запустит процесс сразу. Следите за его шагами в [панели выполнения](#).

Тестовые данные

Вы можете указать в триггерах *Вебхук* и *IMAP* тестовые данные. Это позволяет смотреть, как процесс будет работать с разными входными данными, не дожидаясь реальных событий.

Чтобы указать тестовые данные:

1. Дважды нажмите на триггер на холсте.
2. Перейдите на вкладку *Тестовые данные*.

Выберите тип запроса — GET или POST. Для POST-запроса укажите тело в формате JSON.

Вебхук будет имитировать входящий HTTP-запрос с этими данными.

Режим отладки

⚠ ПРЕДУПРЕЖДЕНИЕ

Режим отладки доступен только для процессов на движке [версии 2](#).

В режиме отладки вы можете пошагово контролировать выполнение процесса и ставить брейкпоинты.

Чтобы включить режим отладки:

1. Нажмите *Тестировать* в правом верхнем углу.
2. Включите *Режим отладки*.

Далее вы можете добавить брейкпоинты и запустить тестирование.

Брейкпоинты

Поставьте брейкпоинт на блок, чтобы процесс остановился прямо перед ним.

Чтобы добавить брейкпоинт:

1. Нажмите на блок на холсте.
2. Нажмите .

Чтобы убрать брейкпоинт:

1. Нажмите на блок на холсте.
2. Выберите .

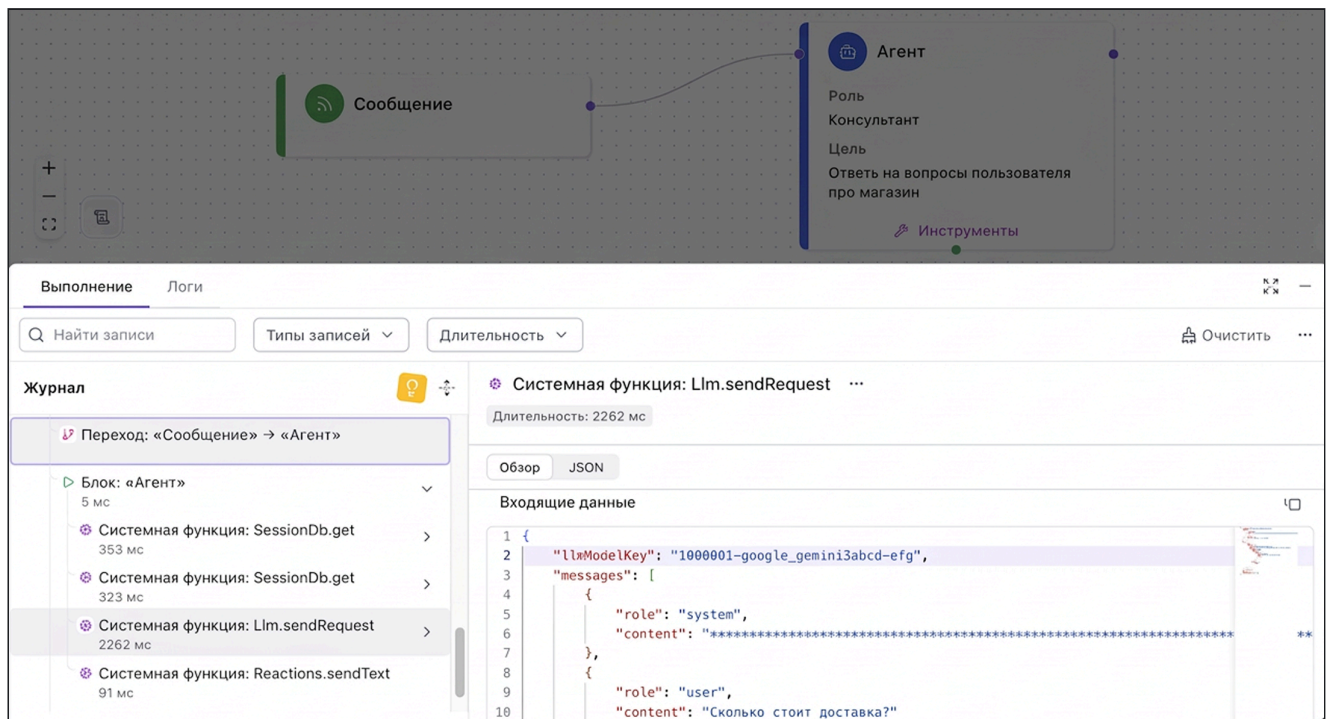
Управление выполнением

1. Нажмите *Тестировать* в правом верхнем углу.
2. В выпадающем меню выберите триггер, с которого хотите запустить процесс.

3. Процесс дойдет до первого брейкпоинта и встанет на паузу.
4. Нажмите *Следующий шаг*, чтобы выполнить блок и снова встать на паузу. Либо нажмите *Продолжить выполнение*, чтобы пройти сразу до следующего брейкпоинта или до конца процесса.

Панель выполнения

Нажмите  в левом нижнем углу холста — на вкладке *Выполнение* вы увидите записи о работе процесса.



❗ ПРИМЕЧАНИЕ

Здесь появляются записи из тестовых запусков. Запросы из реальных каналов сюда не попадают — полные системные логи по всем подключениям собираются на соседней вкладке *Логи*.

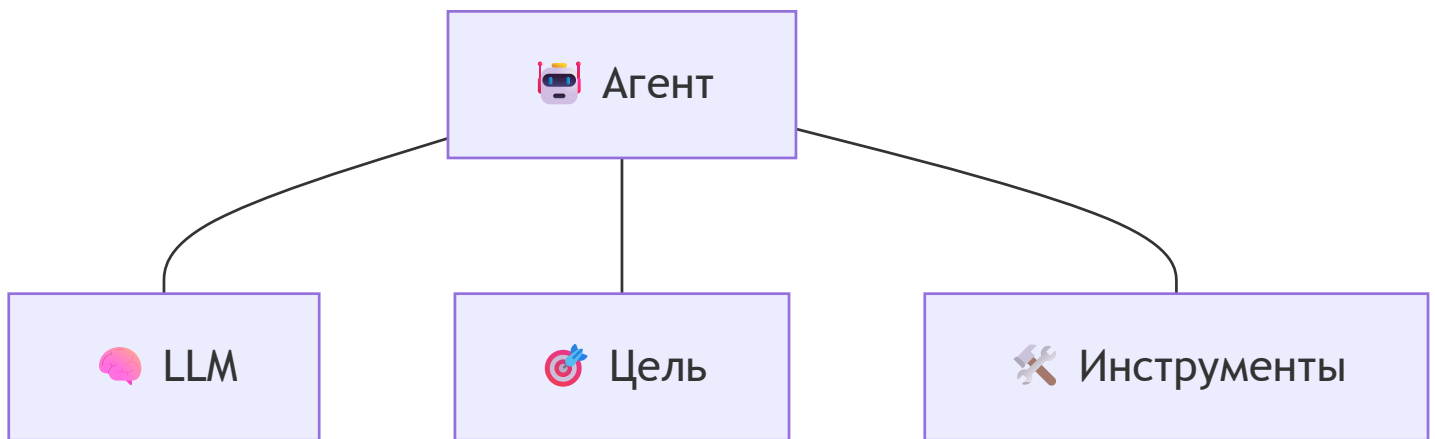
На вкладке *Выполнение* вы можете:

- **Просматривать ход выполнения:** в панели отображается последовательность всех блоков, а также их длительность. Вы также можете использовать функции **Log**, чтобы добавлять свои записи и отслеживать значения переменных.
- **Изучать детали каждого шага:** нажмите на блок в панели выполнения, чтобы открыть детали. Справа вы увидите общую информацию о блоке, входящие и исходящие данные, а также ошибки.
- **Быстро находить блоки на холсте:** нажмите **...** в правом верхнем углу панели и включите *Автофокус на блоке*. Теперь при выборе записи в панели выполнения на холсте будет показан

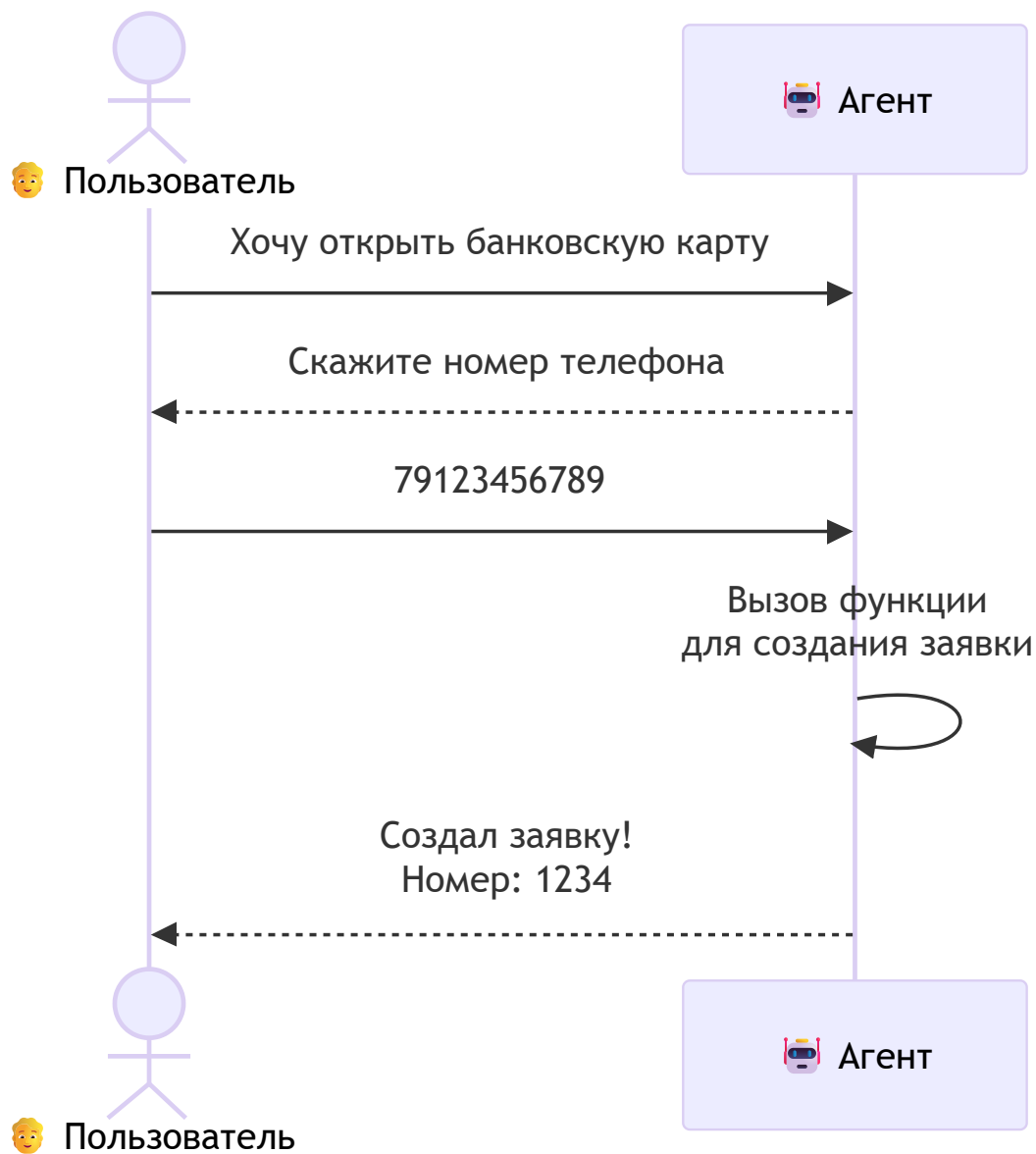
нужный блок.

Блок «Агент»

С помощью блока «Агент» вы можете добавить в процесс **AI-агента** — умного помощника на основе LLM, который сам принимает решения для достижения цели. Он способен гибко отвечать на вопросы, уточнять информацию и вызывать нужные инструменты, как это сделал бы живой оператор.



Например, если пользователь хочет оформить банковскую карту, агент может уточнить его данные, самостоятельно создать заявку на выпуск карты и сообщить номер заявки пользователю:



Как добавить агента на холст

1. Перейдите в проект и на левой панели нажмите , чтобы открыть конструктор процессов.
2. В верхнем левом углу холста нажмите  — откроется список блоков.
3. Перетащите блок *Агенты* → *Агент* на холст.

Настройки агента

Чтобы изменить настройки агента:

1. Дважды нажмите на блок *Агент* на холсте.
2. В открывшемся окне укажите настройки.
3. Нажмите *Сохранить*.

Основные настройки

В поле *Настройки* → *Основные* → *LLM* выберите существующую интеграцию с LLM или добавьте новую. Если вы создадите новую интеграцию, она станет доступна в разделе *Интеграции*.

Агент будет использовать выбранную модель для генерации ответов, принятия решений и вызова инструментов.



ПРЕДУПРЕЖДЕНИЕ
Для работы агента выбранная модель LLM должна поддерживать function calling. Вы можете найти информацию о поддержке function calling в официальной документации самой модели — например, на сайте OpenAI, Google Gemini и т. д.

Промты

В разделе *Настройки* → *Промты* заполните следующие поля:

Поле	Описание	Пример
Роль	Роль или персона агента в диалоге. Влияет на тон общения и на то, как агент отвечает на вопросы о себе.	Сотрудник банка
Цель	В диалоге агент будет пытаться выполнить указанную цель.	Создай заявку на выдачу банковской карты
Инструкции	Представьте, что вы даете задание новому сотруднику. Здесь укажите, как именно он должен выполнить свою работу: с чего начать, какие шаги предпринять и какие инструменты использовать.	Сначала уточни номер телефона. После этого создай заявку с помощью функции <code>Application.create</code> . Отвечай коротко и всегда на русском языке
Запрос для агента	Текст, который будет обрабатывать агент. По умолчанию агент берет текст из последнего сообщения	- Для движка версии 2: <code>Условия новой акции: {{ previousResult }}</code>. - Для версии 1: <code>Условия новой акции: {{ Context.getLastFunctionResult() }}</code>.

Поле	Описание	Пример
	пользователя. Заполните это поле вручную: • Если запускаете агента без сообщения — по триггерам <i>Планировщик</i>, <i>Вебхук</i>, <i>IMAP Email</i>. Смотрите пример в статье Работа агента в процессе. • Если хотите изменить или дополнить запрос пользователя перед отправкой в агента. Настройка игнорируется, если у агента включен <i>Режим диалога</i>.	

 ПОДСКАЗКА

Мы рекомендуем подробно указывать роль, цель и инструкции для агента. Так вы сможете получать более предсказуемые и стабильные результаты.

Ответы и диалог

В разделе *Настройки* → *Ответы и диалог* вы можете настроить генерацию ответов и режим общения с пользователем.

Поле	Описание
<i>Температура</i>	Регулирует креативность ответов. При высоких значениях результаты будут более творческими и менее предсказуемыми. Не рекомендуем менять параметры <i>Температура</i> и <i>Тор Р</i> одновременно.
<i>Тор Р</i>	Регулирует разнообразность ответов. При низких значениях нейросеть выбирает из меньшего количества вероятных слов, а при высоких — ответ может получиться разнообразнее. Не рекомендуем менять параметры <i>Температура</i> и <i>Тор Р</i> одновременно.

Поле	Описание
<i>Presence penalty</i>	Снижает вероятность повторного использования слов в тексте. Помогает сделать ответ разнообразнее по лексике.
<i>Frequency penalty</i>	Снижает вероятность слишком частого повторения одних и тех же слов. Уменьшает риск того, что агент будет заикливаться на одинаковых фразах.
<i>Максимальный размер ответа, в токенах</i>	Максимальное количество токенов, которое модель может сгенерировать за одну итерацию.
<i>Отправлять ответ пользователю</i>	По умолчанию агент сразу отвечает пользователю в чат. Вы можете отключить эту опцию, чтобы использовать агента как функцию. Подробнее смотрите в статье Работа агента в процессе . Не действует при запуске процесса с помощью workflow-триггеров .
<i>Режим диалога</i>	Включите, чтобы агент мог продолжать диалог с пользователем, не переходя к другим блокам процесса. Подробнее смотрите в статье Работа агента в процессе . Не действует при запуске процесса с помощью workflow-триггеров .

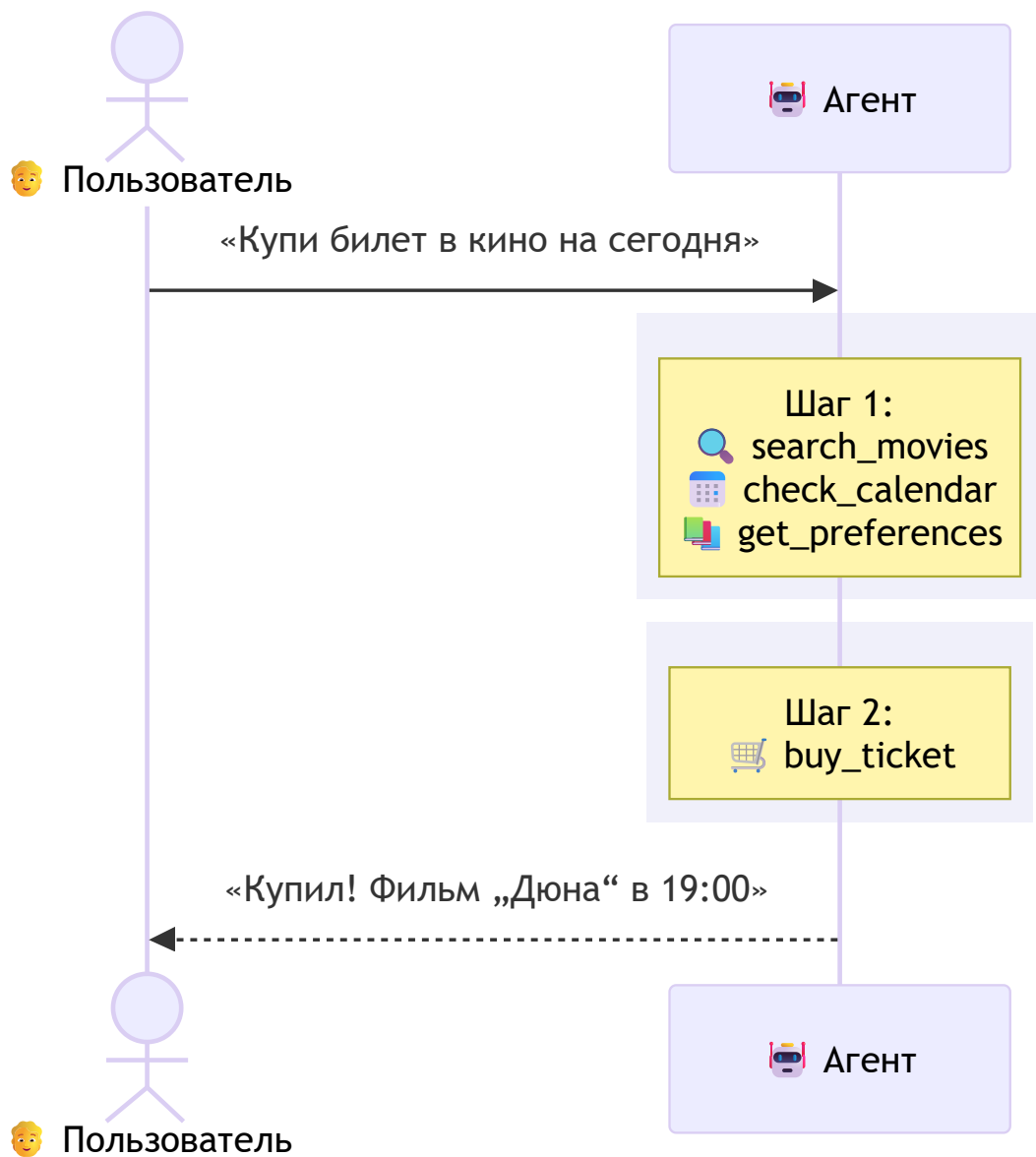
⚠ ПРЕДУПРЕЖДЕНИЕ

Убедитесь, что выбранная LLM поддерживает указанные значения параметров — например, температуру. Несовместимые настройки могут привести к ошибкам и невозможности получить ответ модели.

Контроль

На вкладке *Контроль* вы можете указать лимиты, чтобы агент не заикливался на вызове инструментов и не обрабатывал запрос слишком долго.

Перед ответом пользователю агент может сделать несколько последовательных шагов с вызовом инструментов.



Поле	Описание	По умолчанию	Ошибка при превышении
Шагов подряд	Максимальное количество шагов подряд без ответа пользователю. Защищает от заикливания. Повторные попытки при ошибках не влияют на счетчик — они происходят внутри одного шага.	5 шагов	Sequential tool calls limit exceeded
Инструментов за один шаг	Максимальное количество разных инструментов, которые агент может вызвать за один шаг. Несколько одинаковых инструментов в шаге не увеличивают счетчик.	10 инструментов	Unique tool calls limit exceeded

Поле	Описание	По умолчанию	Ошибка при превышении
Общее время работы	Максимальное время работы агента от получения сообщения до ответа пользователю.	60 секунд	Total response timeout exceeded

Лимиты действуют до ответа пользователю. После ответа отсчет начинается заново.

При превышении лимита агент прерывает работу с ошибкой. Текст ошибки отправляется пользователю и записывается в логи.

Контекст

На вкладке *Контекст* вы можете настроить, как агент использует контекст.

Поле	Описание
Учитывать общий контекст	- Если опция включена, агенту доступен как свой, так и общий контекст. - Если опция отключена, агент работает в изоляции: ему доступен только его контекст. История диалога доступна всем агентам независимо от этой опции. Подробнее смотрите в статье Управление контекстом агента.
Сколько последних сообщений помнит агент	Максимальное количество сообщений в диалоге, отправляемых в LLM. Ограничение позволяет ускорить работу агента и уменьшить расход токенов. Если оставить поле пустым или указать <code>0</code>, в LLM отправляется история сообщений полностью. В этом случае история может превысить контекстное окно LLM, и провайдер вернет ошибку.

Поведение в процессе

На вкладке *Процесс* вы можете настроить логику переходов между блоками в процессе.

Поле	Описание
<i>Следующий шаг процесса</i>	Указывает, какой блок будет выполнен после агента. Если при этом включена опция <i>Режим диалога</i> , то переход будет выполнен, только когда агент достигнет цели или пользователь не захочет продолжать диалог с агентом. Подробнее смотрите в статье Работа агента в процессе .
<i>Передача управления другим агентам</i>	Выберите других агентов, которым текущий агент сможет передать диалог с пользователем. Текущий агент анализирует цели других агентов и сам решает, кто лучше сможет обработать запрос. Подробнее смотрите в статье Работа агента в процессе . Вы также можете использовать опцию <i>Включая новых созданных агентов</i> , чтобы любые новые агенты также становились доступны для передачи диалога.

Телефонный агент

Вы можете настроить поведение агента при звонках: как агент должен реагировать на перебивание и заполнять паузы.

На вкладке *Настройки* включите опцию *Телефонный агент*. После этого появятся настройки поведения в разговоре.

❗ ПРИМЕЧАНИЕ

Эти настройки не влияют на текстовых агентов. Они применяются только при звонках.

Параметры LLM

Лимит ожидания текста от LLM (мс) — максимальное время, которое агент будет ждать ответа от LLM. Если LLM не начала отвечать за это время, дальнейшее поведение зависит от опции *Режим диалога*:

- Если режим включен, агент будет ждать следующей реплики пользователя.
- Если режим выключен, работа агента завершится, и процесс перейдет к следующему блоку.

💡 ПОДСКАЗКА

Если вы настроили [Заполнение пауз](#), то таймер ожидания ответа запустится только после такой паузы. Например, при включенном заполнении пауз (2000 мс) и лимите (5000 мс):

1. Агент подождет 2 секунды.

2. Начнет воспроизводить фразу-филлер и параллельно будет ждать ответа LLM в течение 5 секунд.

Суммарно бот будет ждать ответа 7 секунд.

Обработка перебиваний

Поле	Описание
Действие агента	Настройте, что должен сделать агент, если клиент его перебивает: • <i>Не реагирует</i> — агент полностью озвучит ответ и только после этого будет слушать клиента. • <i>Прерывает ответ сразу</i> — агент сразу остановится и начнет слушать клиента.
Что считает перебиванием	• <i>Законченную фразу клиента</i> — агент будет говорить одновременно с клиентом и среагирует, только когда клиент полностью закончит свою мысль. • <i>Начало речи</i> — перебивание сработает сразу, когда клиент начнет говорить.
Отложить перебивание (мс)	Агент обязательно проговорит этот отрезок времени, даже если клиент начнет его перебивать. Это полезно, чтобы клиент услышал самую важную часть информации. Отсчитывается от начала ответа агента. Если клиент перебьет агента в это время, агент договорит защищенный фрагмент и только потом отреагирует на реплику клиента.

Заполнение пауз

Пока LLM генерирует текст, возникает пауза в речи агента. Вы можете настроить заполнение этих пауз, чтобы разговор с агентом звучал более естественно:

1. Добавьте в *Список фраз* несколько фраз, которые агент будет говорить во время пауз.
Примеры:
 - «Понял, сейчас поищу информацию»
 - «Хорошо, уточняю»
 - «Подождите пару секунд, я проверю»
2. Укажите длину паузы в поле *Сказать фразу, если молчание длится (мс)*. Если в начале ответа возникла пауза такой длины, то агент произнесет одну случайную фразу.

⚠ ПРЕДУПРЕЖДЕНИЕ

Мы не рекомендуем указывать значение меньше 500 мс, так как это может привести к нестабильной работе агента.

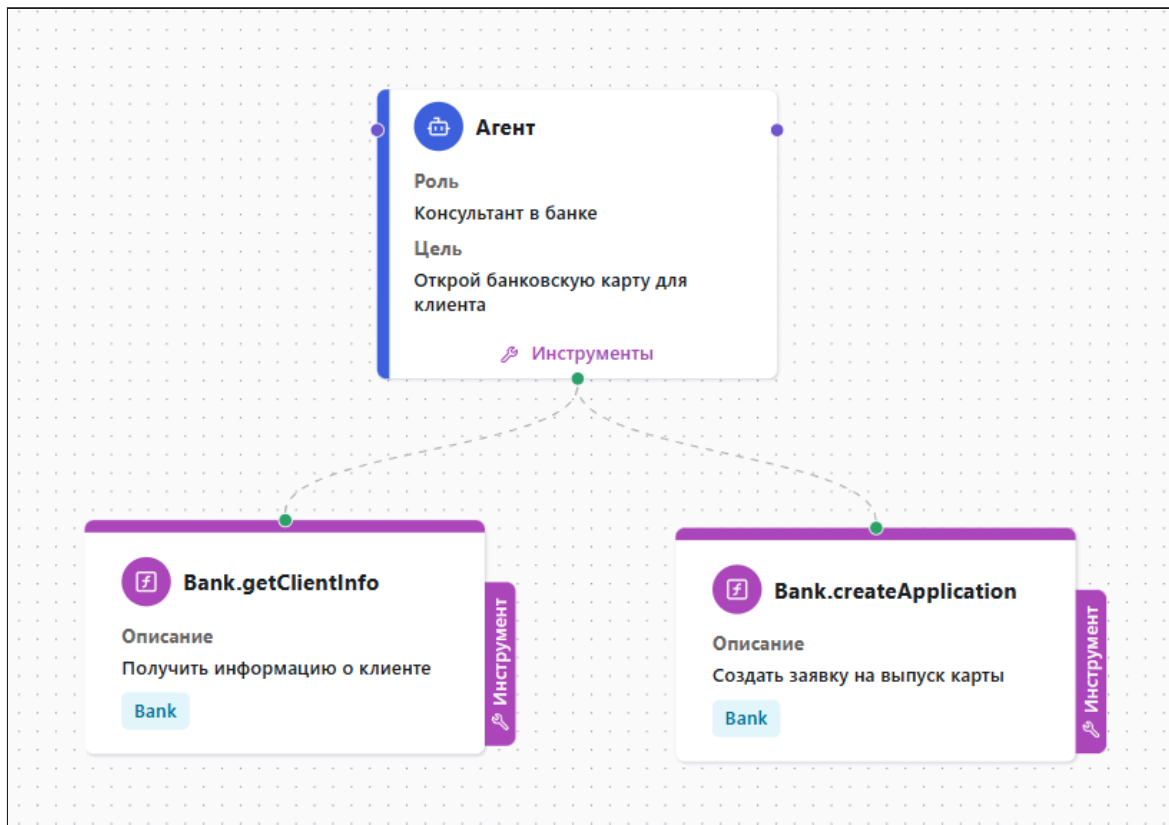
Подключение инструментов к агенту

Инструмент — функция, которую может вызвать агент для выполнения цели.

Чтобы подключить инструмент:

1. Перетащите любую функцию на холст.
2. Дважды нажмите на блок функции:
 - i. В открывшемся окне включите *Режим инструмента*.
 - ii. У параметров, которые должен заполнить агент, включите опцию .
 - iii. Нажмите *Сохранить*.
3. Соедините блок функции с блоком *Агент*.

К одному агенту может быть подключено несколько функций. Агент сам решает, когда и какую функцию вызывать.



Подтверждение перед вызовом

⚠ ПРЕДУПРЕЖДЕНИЕ

Доступно только для движка версии 2.

Если функция проводит платеж, удаляет важные данные или выполняет другое критическое действие, запросите у клиента подтверждение перед ее выполнением:

1. Дважды нажмите на блок функции.
2. На вкладке *Поведение* включите опцию *Запрашивать подтверждение перед вызовом*.
Настройка отображается, только если для функции включен *Режим инструмента*.
3. Укажите текст сообщения. Пример:

⚠ Удалить аккаунт?
Вы не сможете его восстановить, все данные будут также удалены

4. Нажмите *Сохранить*.

Ответ пользователя

Агент интерпретирует ответ пользователя с помощью LLM, поэтому точность распознавания намерений зависит от возможностей выбранной модели. Если ответ положительный, агент вызовет функцию. Если отрицательный — не вызовет и продолжит диалог.

Пример диалога

Пользователь просит удалить аккаунт, агент запрашивает подтверждение:

1.  **Пользователь:**

Хочу удалить свой аккаунт

2.  **Ассистент:**

⚠ Удалить аккаунт? Вы не сможете его восстановить, все данные будут также удалены

```
{  
  accountId: "12341111"  
}
```

📌 К СВЕДЕНИЮ

К сообщению агент прикрепляет аргументы, с которыми будет вызвана функция. В данном случае это ID аккаунта, который будет удален.

3. Далее пользователь подтверждает или отменяет действие:

Подтверждение

Отказ

 **Пользователь:**

Давай

 **Ассистент:**

Аккаунт удален

Работа агента в процессе

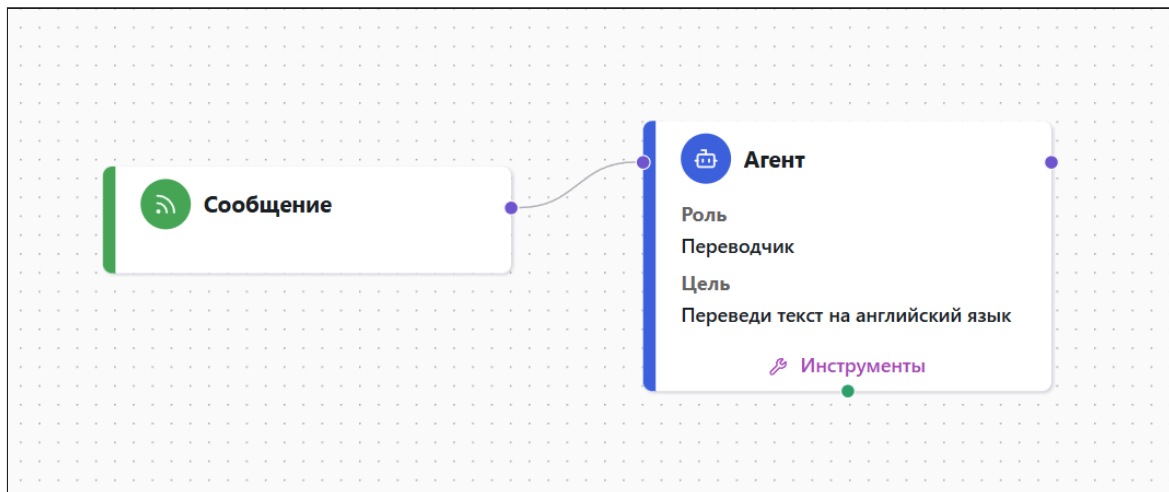
В зависимости от вашей задачи, агент может работать в процессе по-разному: быть частью сложной автоматизации, напрямую отвечать пользователю в чате или вести полноценный диалог.

Из этой статьи вы узнаете, как настроить агента для разных задач:

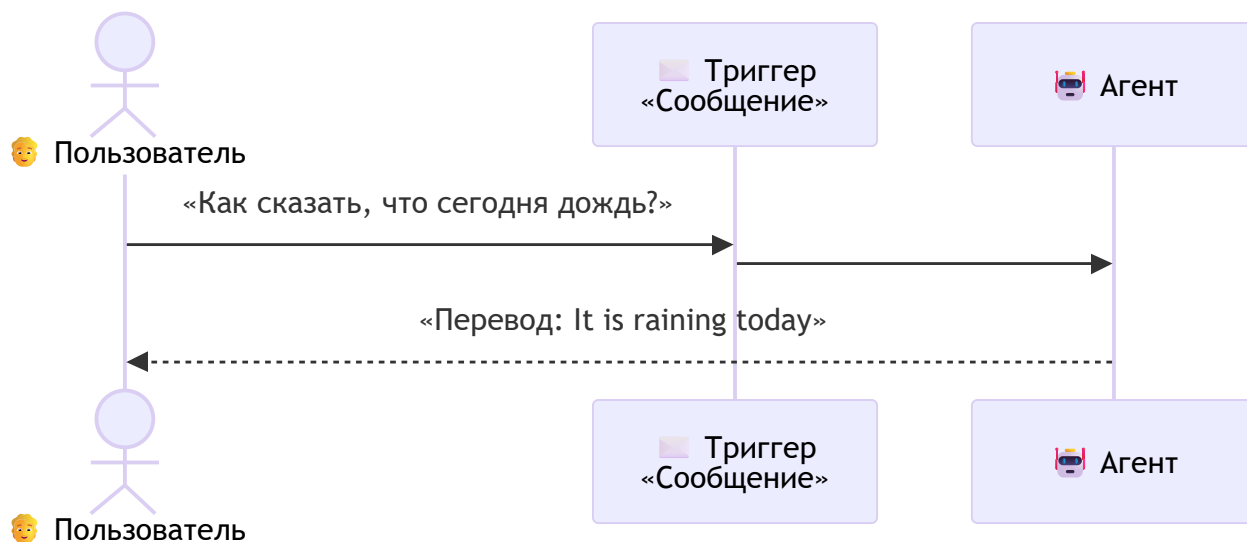
- **Напрямую отвечать пользователю:** режим по умолчанию, в котором агент сразу отправляет свой ответ в чат.
- **Вести полноценный диалог:** как научить агента поддерживать беседу для решения многошаговых задач.
- **Использовать как функцию для генерации контента:** когда нужно, чтобы агент создал текст и передал его в другой блок.
- **Создавать мультиагентные процессы:** как объединить несколько агентов для построения сложных и надежных систем.

Ответ пользователю

По умолчанию у агента включена опция *Отправлять ответ пользователю*, поэтому агент автоматически отправляет сообщение в чат сразу после генерации ответа.



Процесс будет работать так:



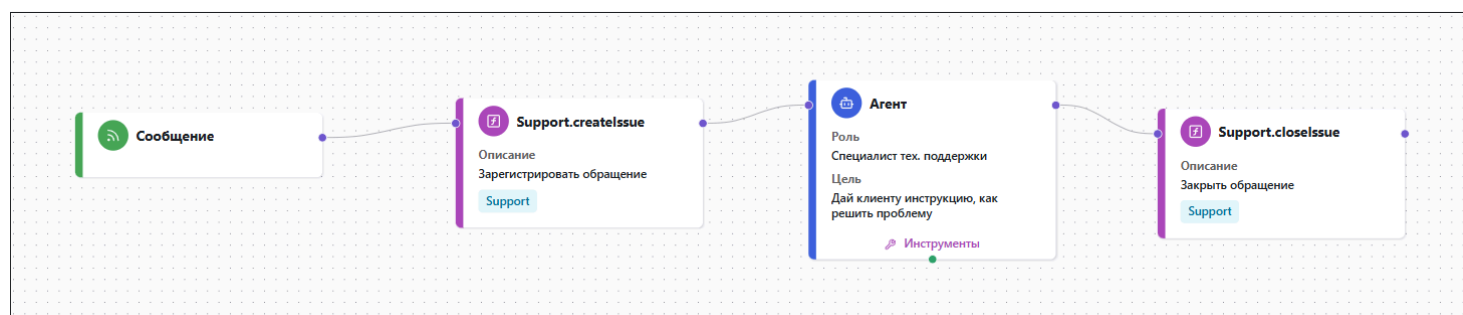
💡 ПОДСКАЗКА

При этом вы можете получить ответ агента и далее использовать его в процессе с помощью функции `Context.getLastFunctionResult()`.

Диалог с агентом

Часто для решения задачи агенту нужно обменяться с пользователем несколькими сообщениями, например, чтобы уточнить детали или задать вопросы. Однако по умолчанию агент обрабатывает только одно сообщение и процесс сразу переходит к следующему блоку.

Предположим, у нас есть процесс агента поддержки:

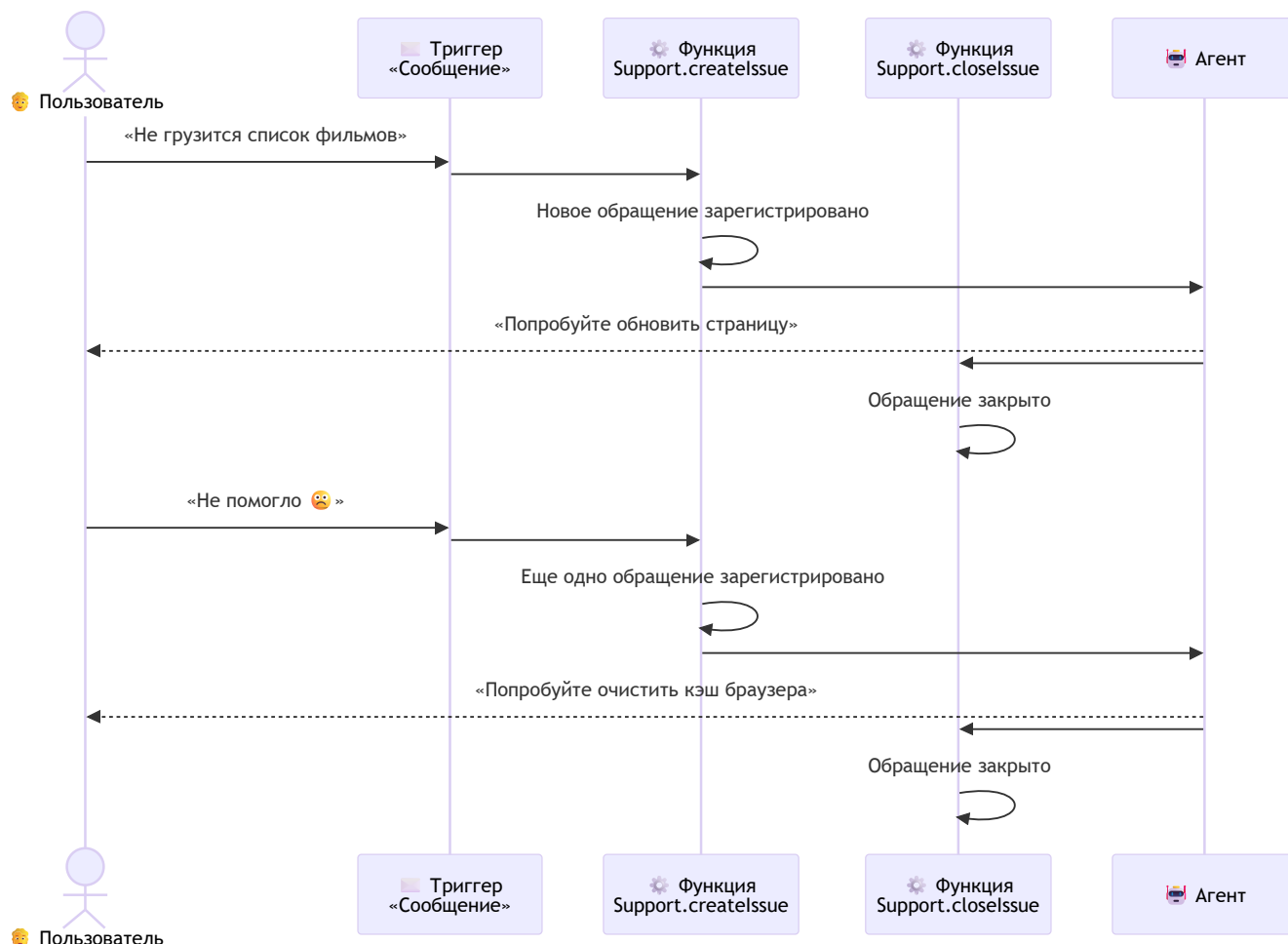


- У агента включена опция *Отправлять ответ пользователю*.
- Перед блоком агента выполняется функция `Support.createIssue`, которая регистрирует новое обращение в поддержку. После блока агента выполняется функция `Support.closeIssue`, которая закрывает обращение.

💡 ПОДСКАЗКА

`Support.createIssue` и `Support.closeIssue` — это пользовательские функции, которые мы создали заранее для этого примера.

В таком процессе на **каждое** сообщение пользователя будет создаваться новое обращение. Обращение также будет закрываться после **каждого** ответа агента.



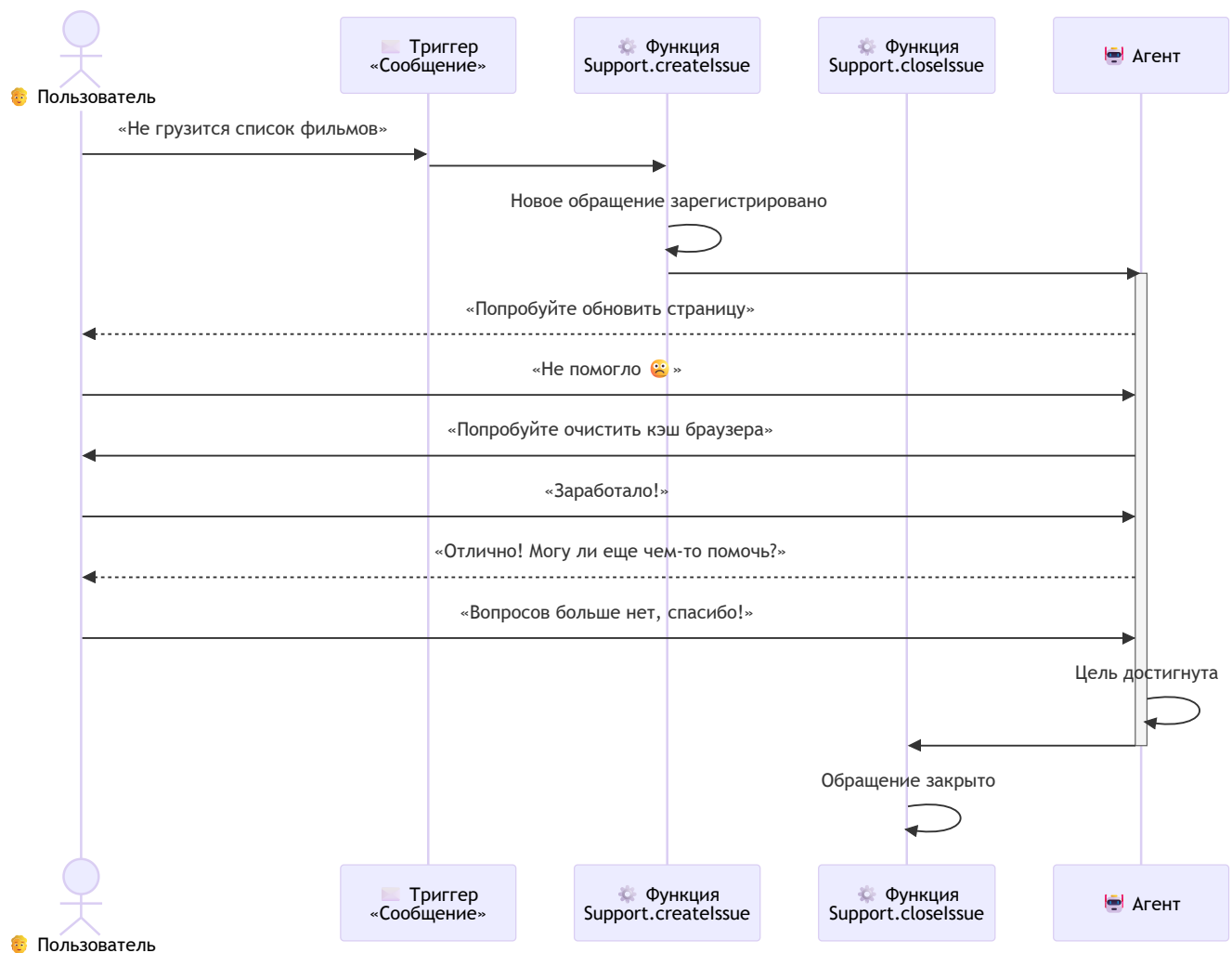
Чтобы агент мог вести полноценный диалог, нужно «остановить» процесс на его блоке до тех пор, пока задача пользователя не будет решена. Для этого:

1. Перейдите в настройки блока *Агент*.
2. На вкладке *Настройки* → *Ответы и диалог* включите опцию *Режим диалога*. При этом опция *Отправлять ответ пользователю* включится автоматически.
3. Выберите блок, в который должен происходить переход после достижения цели. Это можно сделать двумя способами:
 - В настройках агента на вкладке *Процесс* выберите блок в поле *Следующий шаг процесса*.
 - Просто присоедините блок к выходу агента на холсте.

💡 ПОДСКАЗКА

В нашем примере это уже блок с функцией `Support.closeIssue`.

Теперь процесс будет работать правильно: обращение откроется один раз и закроется только тогда, когда агент выполнит свою цель.



Агент как функция

Если вы хотите использовать ответ агента в других блоках или как-то дополнительно обрабатывать его, то используйте агента в качестве функции. Для этого:

1. Отключите настройку *Отправлять ответ пользователю*. Агент будет генерировать ответ, но не будет отправлять его в чат.
2. В следующем блоке получите ответ агента с помощью системной функции `Context.getLastFunctionResult()`.

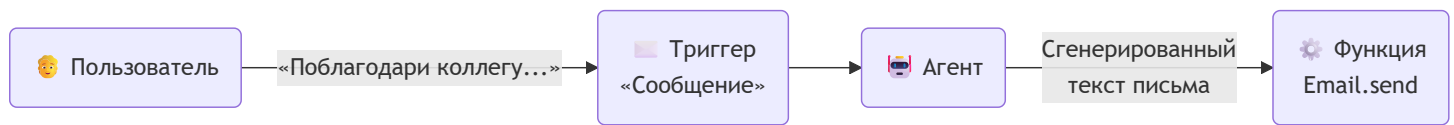
Например, вы можете генерировать тело письма через агента и использовать этот текст в следующем блоке. Процесс отличается в зависимости от **версии движка** в проекте.

Версия 2 Версия 1

1. Добавьте на холст блок функции `Context.getLastFunctionResult()` и соедините с агентом.
2. Добавьте блок функции `Email.send()` и соедините с предыдущим блоком.

3. У функции `Email.send()` в параметре `content` укажите `{{ previousResult }}`. Тогда в параметр будет подставляться результат последней выполненной функции, в данном случае это будет ответ агента.

В итоге процесс будет работать так:



Мультиагентные процессы

Преимущества

Преимущества использования нескольких агентов в одном процессе:

- **Разделение ответственности**

Каждый агент выполняет узкую роль — LLM лучше справляется с одной задачей.

- **Контроль данных**

Вы можете контролировать, какие данные получает каждый агент, и не передавать лишнюю информацию.

- **Гибкость и масштабируемость**

Легко добавлять новых агентов для дополнительных задач.

Такой подход облегчает разработку сложных процессов и повышает надежность.

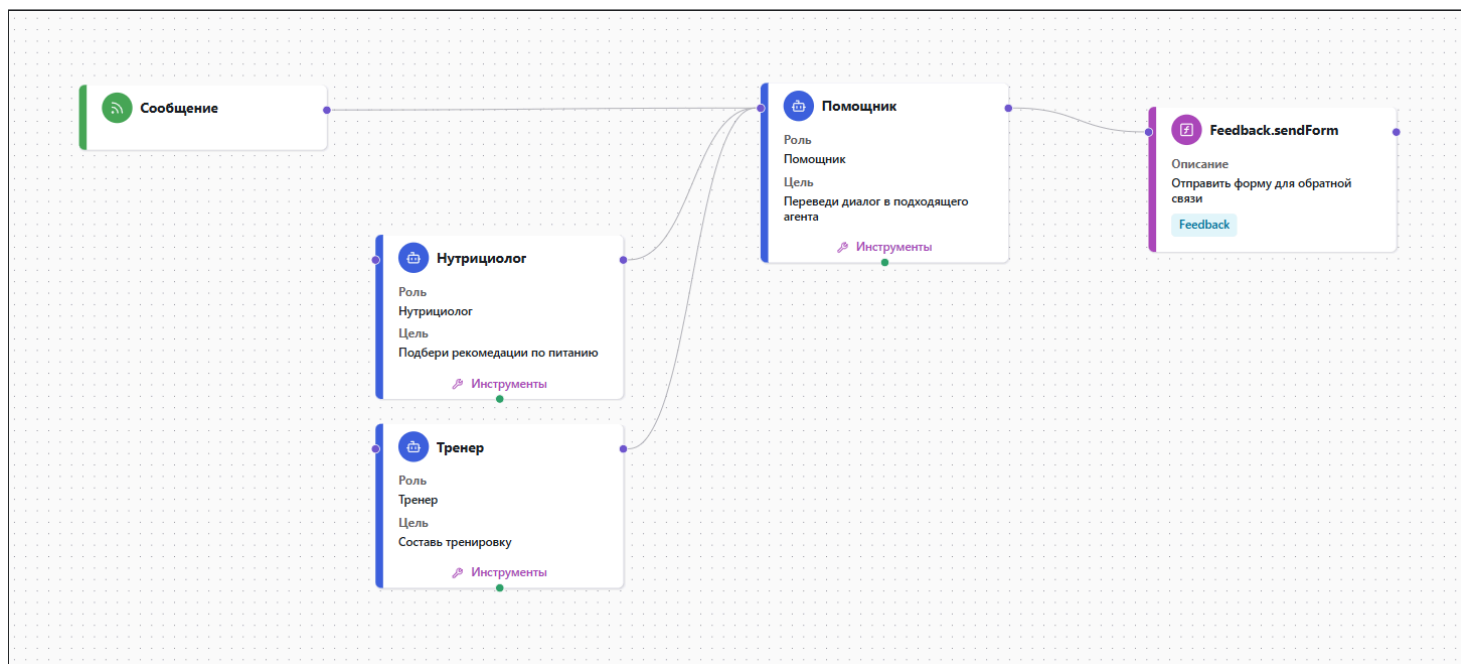
Как настроить процесс

В примере ниже показан процесс с тремя агентами. В этом процессе пользователь может получить консультацию по питанию или попросить составить план тренировок.

Первое сообщение пользователя попадет к агенту-оркестратору, который определит, к какому из двух специализированных агентов (по питанию или по тренировкам) нужно перенаправить запрос. Пользователь сможет продолжить диалог с выбранным агентом или переключиться на другого. Если цель достигнута или пользователь откажется от помощи, процесс завершится — будет выполнена функция `Feedback.sendForm`.

💡 **ПОДСКАЗКА**

`Feedback.sendForm` — это пользовательская функция, которую мы создали заранее для этого примера.



▲ ОТОБРАЖЕНИЕ СВЯЗЕЙ МЕЖДУ АГЕНТАМИ

Передача диалога другим агентам настраивается только в окне агента-оркестратора, при этом на холсте **не отображаются связи между агентами**.

Стрелки на холсте показывают, куда перейдет процесс после того, как агент-специалист (например, «Нутрициолог») полностью завершит свою работу. В нашем примере он вернет управление обратно «Помощнику».

Рассмотрим настройки каждого из агентов.

- **Агент «Помощник»** — это оркестратор, который перенаправит запрос к нужному агенту.
 - У агента включена опция *Режим диалога*, чтобы он мог поддерживать диалог с пользователем и процесс не завершился после первого ответа.
 - На вкладке *Процесс* в секции *Передача управления другим агентам* выбраны *Нутрициолог* и *Тренер*. Это позволяет агенту-оркестратору классифицировать запросы и передавать их нужному агенту.

Оркестратор учитывает цели агентов, чтобы правильно классифицировать запросы.

💡 ПОДСКАЗКА

Вы также можете включить опцию *Включая новых созданных агентов*. Так любые новые агенты в процессе будут автоматически доступны для оркестратора.

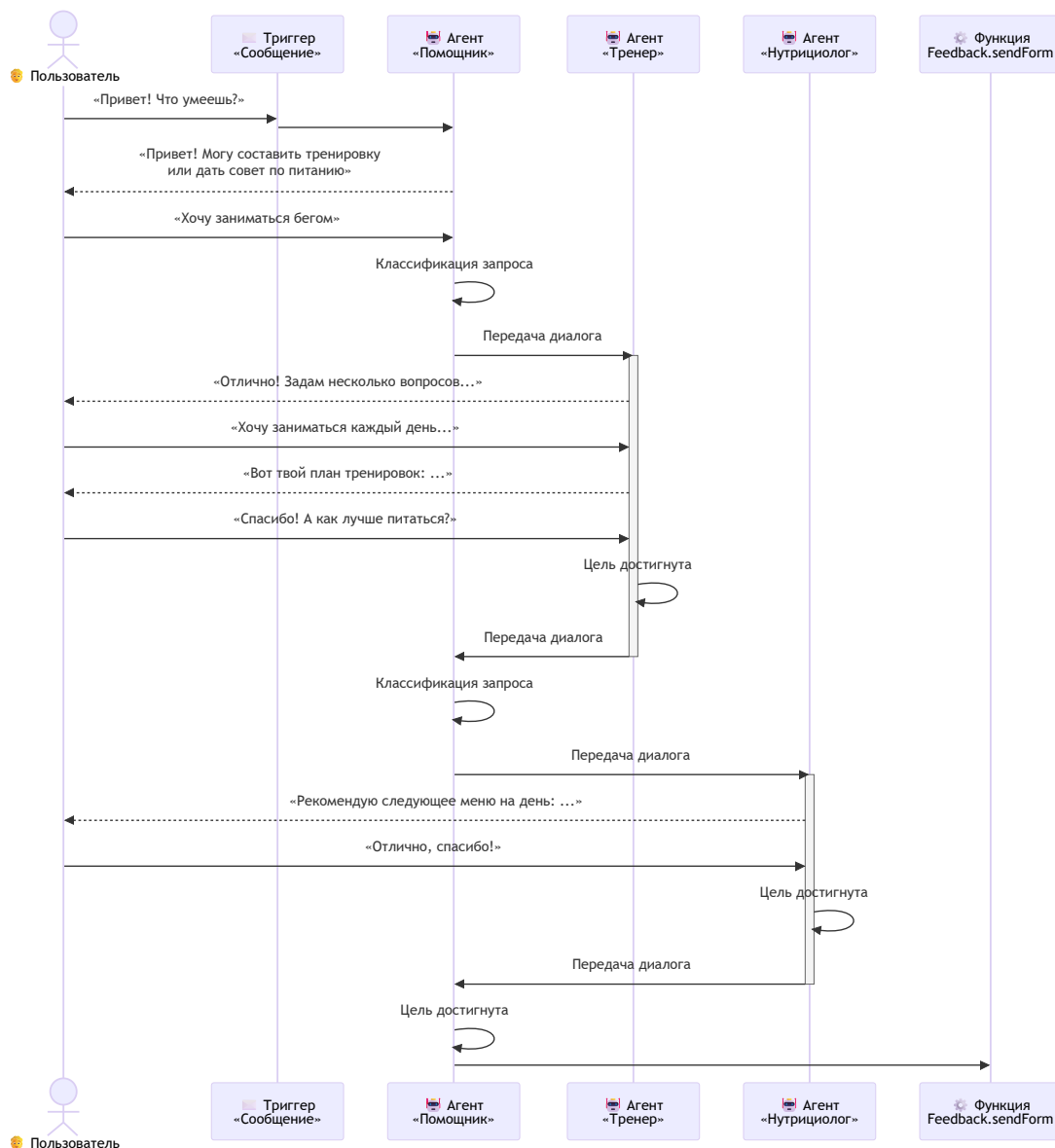
- **Агенты «Тренер» и «Нутрициолог»** — это специализированные агенты, которые обрабатывают запросы по своим направлениям.

- У каждого агента включена опция *Режим диалога*, чтобы они могли поддерживать диалог с пользователем и процесс не завершался после первого ответа.
- Следующий шаг процесса у каждого агента — это агент оркестратор.

Если у пользователя нет больше вопросов или вопрос не по теме агента:

- Диалог перейдет к агенту-оркестратору.
- Оркестратор может передать запросу другому агенту или посчитать, что можно завершить диалог.

Процесс работает так:



Управление контекстом агента

Продвинутая возможность

`AgentContext` — это служебный объект для сохранения структурированных данных (ключ-значение) и текстовых заметок в рамках текущей сессии. Доступные агенту данные из `AgentContext` автоматически добавляются к системному промту, передаваемому в LLM.

В `AgentContext` можно **добавлять и удалять** данные с помощью встроенной коллекции `AgentContext`, а также **записывать результаты выполнения функций**.

Каждая заметка или пара ключ-значение относится либо к общему контексту, либо к отдельному контексту конкретного агента. Отдельный контекст агента недоступен другим агентам. В некоторых случаях полезно **отключить** агенту и доступ к общему контексту — например, чтобы уменьшить потребление токенов LLM.

Редактирование контекста

Управлять контекстом прямо в процессе можно с помощью функций встроенной коллекции `AgentContext`. Функции можно использовать в качестве инструментов для AI-агента, отдельных шагов в процессе или вызвать в коде других блоков.

❗ К СВЕДЕНИЮ

Если в вашем проекте [движок версии 2](#), функции являются синхронными, если версия 1 — асинхронными.

Пример

Рассмотрим функцию, которая получает информацию о заказе и записывает ее в общий контекст. Реализация зависит от **версии движка** в вашем проекте.

Версия 2 **Версия 1**

```
async function getOrderInfo() {  
  
    // Логика получения информации о заказе...  
  
    // order = {
```

```
// id: "12345",  
// status: "В пути",  
// items: 2  
// }  
  
// Сохранение информации в общий контекст  
AgentContext.putValue({key: "lastOrder", value: order});  
}
```

Данные станут доступны агенту автоматически (если не отключен доступ к общему контексту), дополнительные настройки не требуются.

Получение agentId

Чтобы работать с отдельным контекстом конкретного агента, в функциях `AgentContext` нужно указать параметр `agentId`. Чтобы получить ID агента:

1. На холсте дважды нажмите на блок *Агент*.
2. Нажмите  ID.

ID агента скопируется в буфер обмена.

Автоматическое сохранение результатов в контекст

Результат выполнения функций можно записывать в `AgentContext` автоматически, без явного вызова `AgentContext.putValue()` или `AgentContext.addNote()`. Эта возможность доступна во всех блоках функций — как в режиме инструмента, так и в режиме отдельного шага в процессе.

Чтобы включить автоматическое сохранение для функции:

1. На холсте дважды нажмите на блок функции и перейдите на вкладку *Контекст*.
2. Включите опцию *Сохранять результат в контекст*.
3. Укажите *Имя поля в AgentContext* (необязательно).
4. Добавьте *Префикс* (необязательно), чтобы помочь агенту понять смысл данных. Например, `Информация о заказе:`.

ПРИМЕЧАНИЕ

Если указан префикс, то сохраненное значение по ключу будет строкой, даже если функция возвращает JSON-объект. Например, `Информация о заказе: { "id": "12345", "status": "В пути", "items": 2 }`.

5. Включите опцию *Записывать ошибки*, чтобы в случае сбоя в работе функции в контекст попало описание ошибки.
6. Выберите, кому будут доступны данные:
 - *Всем агентам*: результат сохранится в общий контекст.
 - *Только выбранным*: результат сохранится в отдельные контексты указанных агентов.

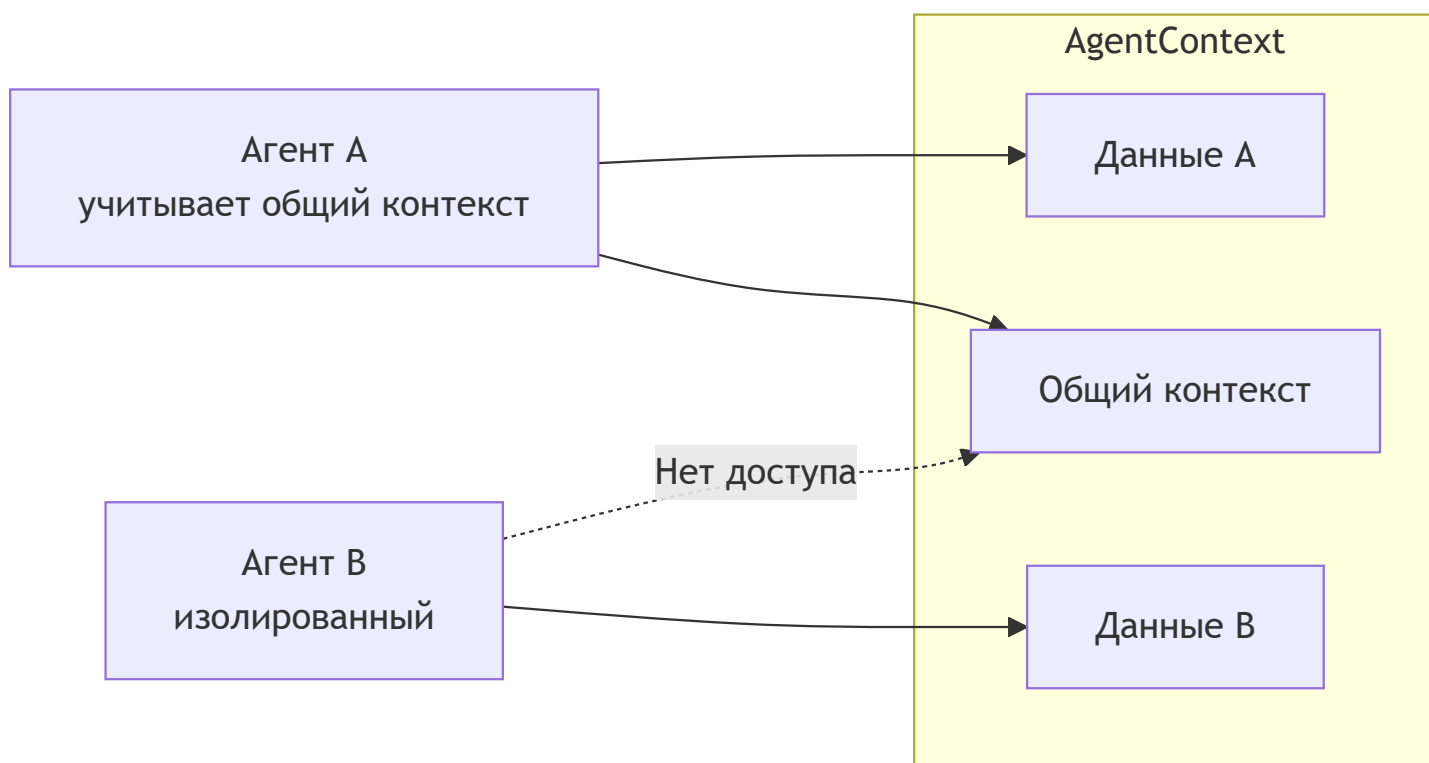
Изоляция контекста

Доступом агента к общему контексту управляет настройка *Учитывать общий контекст* (находится на вкладке *Контекст* блока *Агент*):

- Если включена (по умолчанию), агенту доступен как свой, так и общий контекст.
- Если отключена, агент работает в изоляции: ему доступен только его контекст.

❗ ПРИМЕЧАНИЕ

История сообщений не входит в `AgentContext` и доступна всем агентам.



Если в общем контексте и в контексте агента есть данные с одинаковым ключом, агент получит значение из своего контекста.

Встроенные функции

Agent Platform предоставляет встроенные функции, которые позволяют управлять диалогом, взаимодействовать с базами данных и интеграциями, отправлять HTTP-запросы и многое другое.

Подробнее о работе с функциями читайте в разделе [Использование функций в процессе](#).

Встроенные коллекции

Коллекция	Назначение
 <code>AgentContext</code>	Управлять общим контекстом и отдельными контекстами агентов
 <code>Asr</code> и <code>Tts</code>	Распознавать и синтезировать речь
  <code>Context</code>	Получать метаданные, подробную информацию о запросе пользователя, отслеживать состояние процесса
 <code>Credentials</code>	Получать учетные данные для интеграций
 <code>Db</code> и <code>SessionDb</code>	Работать с базами данных
 <code>Dialer</code>	Управлять входящими звонками
 <code>Email</code>	Отправлять email-сообщения
 <code>Http</code>	Выполнять HTTP-запросы
 <code>Llm</code>	Взаимодействовать с языковыми моделями
 <code>Log</code>	Записывать сообщения в журнал
 <code>ProjectRouter</code>	Взаимодействовать с другими проектами Agent Platform и JAICP
 <code>Rag</code>	Работать с базами знаний RAG
  <code>Reactions</code>	Отправлять ответы клиенту в подключенном канале

Коллекция	Назначение
 Telegram	Отправлять сообщения в Telegram без подключения канала

AgentContext

Функции для работы с контекстом. Подробнее смотрите в статье [Управление контекстом агента](#).

- `addNote(text, [agentId])` — добавляет текстовую заметку в контекст указанного агента, а если агент не указан — в общий контекст. Заметка должна быть непустой: если она состоит только из пробельных символов, то добавлена не будет.
- `getNotes([agentId])` — возвращает все заметки из общего контекста и контекста агента (если указан), объединенные в одну строку, в порядке добавления.
- `deleteNotes([agentId])` — удаляет все заметки из контекста указанного агента, если указано `all` — из всех контекстов, а если параметр не указан — из общего контекста.
- `putValue(key, value, [agentIds])` — сохраняет значение по ключу в контекст каждого из указанных агентов, а если агенты не указаны — в общий контекст.
- `getValue(key, [agentId])` — получает значение по ключу. Если указан агент, сначала ищет в его контексте, затем в общем.
- `deleteValue(key, [agentId])` — удаляет значение по ключу из контекста указанного агента, если указано `all` — из всех контекстов, а если параметр не указан — из общего контекста.
- `clear([agentId])` — удаляет контекст (все ключи и заметки) указанного агента, если указано `all` — все контексты, а если параметр не указан — общий контекст.

Asr и Tts

Для работы этих функций нужно подключить интеграции: [Модель ASR и Голос TTS](#).

- `Asr.recognize` — получает файл по URL и распознает речь в нем.
- `Tts.synthesize` — синтезирует речь из текста и возвращает URL, по которому можно скачать аудиофайл.

Подробнее о работе с этими функциями читайте в статье [ASR и TTS в текстовых каналах](#).

Context

- **Получайте различные метаданные процесса:**

JavaScript	Python	Описание
<code>getAccountId</code>	<code>get_account_id</code>	ID вашего аккаунта Agent Platform.
<code>getProjectShortName</code>	<code>get_project_short_name</code>	Короткое имя проекта.
<code>getBotId</code> <code>getChannelType</code> <code>isAsyncChannel</code> <code>isTestChannel</code>	<code>get_bot_id</code> <code>get_channel_type</code> <code>is_async_channel</code> <code>is_test_channel</code>	ID и тип канала.
<code>getChatId</code>	<code>get_chat_id</code>	Идентификатор диалога.
<code>getSessionId</code>	<code>get_session_id</code>	Идентификатор текущей сессии.
<code>getClientInfo</code>	<code>get_client_info</code>	Информация о пользователе, который взаимодействует с процессом.

- Получайте подробную информацию о запросе пользователя:

JavaScript	Python	Описание
<code>getRequestId</code>	<code>get_request_id</code>	ID запроса.
<code>getMessageContent</code>	<code>get_message_content</code>	Контент сообщения.
<code>getRawRequest</code>	<code>get_raw_request</code>	Полные данные запроса.
<code>getEmailContent</code>	<code>get_email_content</code>	Данные письма при использовании триггера IMAP Email .

- Управляйте диалогом:

JavaScript	Python	Описание
<code>getChatHistory</code>	<code>get_chat_history</code>	История сообщений в диалоге.
<code>getLastFunctionResult</code>	<code>get_last_function_result</code>	Результат последнего вызова функции в процессе.

JavaScript	Python	Описание
<code>resetSession</code>	<code>reset_session</code>	Запустить новую сессию, очистить данные <code>SessionDb</code> и историю сообщений (не поддерживается для движка версии 1).

- **Сохраняйте и получайте значения по ключу** (не поддерживаются для [движка версии 1](#)):

JavaScript	Python	Описание
<code>set</code>	<code>set</code>	Сохраняет значение по ключу. Ключ можно затем использовать в подстановках .
<code>get</code>	<code>get</code>	Возвращает сохраненное значение по ключу.

❗ ПРИМЕЧАНИЕ

В пользовательских Python-функциях и блоке *Код* доступны встроенные функции `sdk.context.*`. Названия функций соответствуют используемым в JavaScript с переводом в snake_case.

Пример: `sdk.context.get_channel_type()`.

Credentials

Получайте учетные данные для интеграций из хранилища с помощью функции `get` вместо непосредственной вставки в код.

Db и SessionDb

С помощью `db` вы можете работать с базами данных, которые вы добавили в [разделе Интеграции](#) или [Базы данных](#).

Функции из коллекции `SessionDb` работают аналогично, но не требуют настройки интеграции с БД и обрабатывают данные только в рамках текущей сессии. При вызове `Context.resetSession()` данные `SessionDb` очищаются.

- **Получайте данные:**
 - `get` — получить запись по ключу.

- `findByFilters` — найти одну или несколько записей по заданным фильтрам.
- `countByFilters` — получить количество записей, которые соответствуют фильтрам.
- **Добавляйте и изменяйте данные:**
 - `put` — добавить новую запись в базу.
 - `updateByFilters` — обновить поля в записях, которые соответствуют фильтрам.
- **Удаляйте данные:**
 - `delete` — удалить запись по ключу.
 - `deleteByFilters` — удалить все записи, которые соответствуют фильтрам.

Dialer

Функции для входящих звонков:

- Получение данных:
 - `getCaller` — получить номер телефона клиента.
 - `getSipHeaders` — получить **SIP-заголовки** звонка.
 - `getTransferStatus` — получить **статус перевода на оператора**.
- Управление звонком:
 - `hangUp` — завершить вызов. Если ассистент общается в текстовом канале, вы можете отправить клиенту сообщение через необязательный параметр `text`.
 - `setNoInputTimeout` — задать время ожидания ответа от клиента.
 - `transferCall` — **перевести вызов на оператора**.

❗ ПРИМЕЧАНИЕ

Остальные функции `Dialer` предназначены для исходящих звонков и сейчас не поддерживаются.

Email

Отправляйте email-сообщения с помощью функции `send`.

Http

- Функции для вызова HTTP-методов:
 - `get`
 - `post`
 - `put`

- `delete`
- `patch`
- `execute` — функция, в которой вы можете выбрать HTTP-метод сами.

Llm

- `getChatHistory` — получайте историю сообщений в специальном формате для LLM.
- `sendRequest` — настраивайте все параметры запроса вручную.
- `sendText` — отправляйте простые текстовые запросы к LLM.
- `summarizeText` — сокращайте длинные тексты до заданного размера с сохранением ключевой информации. Поддерживаются тексты до 200 000 символов.

Log

Используйте функции `Log`, чтобы выводить промежуточные значения переменных или оставлять заметки. Это поможет быстрее найти ошибку в сложном процессе.

Вы можете добавлять в журнал сообщения разных уровней. Смотрите результаты на вкладке *Выполнение*.

ProjectRouter

`ProjectRouter` предоставляет инструменты для создания сложных систем, в которых отдельные проекты реализуют разные части бизнес-логики.

- `callProject` — вызов другого проекта и JAICP. Ответ другого проекта не отправляется клиенту, а обрабатывается в исходном проекте. Подробнее смотрите в разделе *Вызов другого проекта*.
- `switchTo` — передача управления в другой проект и JAICP. Позволяет другому боту вести диалог с клиентом. Новый бот может вернуть диалог исходному боту с помощью `backToOriginalProject`. Подробнее смотрите в разделе *Передача управления в другой проект*.
- `backToOriginalProject` — вернуть управление в исходный проект.
- `getCustomData` — извлечь данные, полученные при передаче или возврате управления.

❗ К СВЕДЕНИЮ

JAICP (Just AI Conversational Platform) — платформа для разработки диалоговых решений любой сложности. Подробнее о JAICP можно узнать на [нашем сайте](#) или в [документации](#).

Rag

Используйте интеграцию с базой знаний RAG:

- `retrieveChunks` — получить фрагменты документов, релевантные запросу пользователя.
- `generateAnswer` — сгенерировать ответ на вопрос пользователя с помощью RAG.

Reactions

Отправляйте сообщения клиентам в подключенных каналах:

JavaScript	Python	Описание
<code>sendText</code>	<code>send_text</code>	Текстовое сообщение.
<code>sendHtml</code>	<code>send_html</code>	Сообщение в разметке HTML.
<code>sendAudio</code>	<code>send_audio</code>	Аудио.
<code>sendImage</code>	<code>send_image</code>	Изображение.
<code>sendVideo</code>	<code>send_video</code>	Видео.
<code>sendFile</code>	<code>send_file</code>	Файл.
<code>sendLocation</code>	<code>send_location</code>	Геолокация.
<code>sendRawRequest</code>	<code>send_raw_request</code>	Данные для ответа в формате, который поддерживает канал.

❗ ПРИМЕЧАНИЕ

В пользовательских Python-функциях и блоке *Код* доступны встроенные функции `sdk.reactions.*`. Названия функций соответствуют используемым в JavaScript с переводом в `snake_case`.

Параметры передаются в виде Python-словаря, ключи указываются в кавычках.

Пример: `await sdk.reactions.send_text({"text": "Привет!"})`.

❗ ПРИМЕЧАНИЕ

Функции `Reactions` предназначены для коммуникационных процессов с триггером *Сообщение*. В workflow-триггерах (*Вебхук*, *Планировщик*, *IMAP Email*) сессия не привязана к каналу или клиенту, поэтому `Reactions` не доставят сообщение.

Подробнее о работе триггеров читайте в [соответствующей статье](#) документации.

Telegram

Отправляйте сообщения в Telegram без подключения канала. В каждой функции укажите **учетные данные** с токеном Telegram-бота. Эти функции можно использовать вместо `Reactions`, например, в следующих случаях:

- Вы в процессе динамически определяете, какой Telegram-бот должен написать пользователю.
- Вы получили запрос от пользователя в одном Telegram-боте, но хотите отправить ответ из другого бота.
- Вы используете workflow-триггер (*Вебхук*, *Планировщик*, *IMAP Email*) и хотите отправить уведомление в конкретный Telegram-чат.

Функции:

- `sendText` — отправить текстовое сообщение.
- `sendImage`, `sendAudio`, `sendVoice` — отправить изображение, аудио или голосовое сообщение.
- `sendButtons` — отправить сообщение с кнопками.

Синхронность функций

JavaScript

Python

Синхронность или асинхронность встроенных функций зависит от **версии движка** в проекте.

Версия 2

Встроенные функции являются:

- синхронными, если вызываются из блока *Код* или из пользовательских функций в коллекциях без `prmt`-зависимостей;
- асинхронными, если вызываются из пользовательских функций в тех коллекциях, где подключены `prmt`-зависимости.

Версия 1

Большинство встроенных функций являются асинхронными. Вы можете вызывать их с помощью `await`.

Синхронными являются только следующие функции:

- `Context`:
 - `getAccountId`
 - `getProjectShortName`
 - `getBotId`
 - `getChannelType`
 - `isAsyncChannel`
 - `isTestChannel`
 - `getChatId`
 - `getSessionId`
 - `getClientInfo`
 - `getRequestId`
 - `getMessageContent`
 - `getRawRequest`
 - `getEmailContent`
 - `getLastFunctionResult`
- `Dialer`:
 - `getCaller`
 - `getSipHeaders`
 - `hangUp`
 - `setNoInputTimeout`
 - `transferCall`
 - `getTransferStatus`

Пользовательские функции

Пользовательские функции — это мощный инструмент для расширения стандартных возможностей платформы. Они позволяют вам писать собственный код на JavaScript или Python для решения уникальных задач: от сложных вычислений до интеграции со сторонними API.

Каждая созданная вами функция становится многоразовым компонентом, который можно использовать в качестве отдельного шага в процессе, в виде инструмента для AI-агента, а также вызывать из кода. Подробнее об этом в разделе [Использование функций в процессе](#).

JavaScript и Python

Вы можете использовать оба языка в одном проекте, с учетом ограничений:

- JavaScript-функции и Python-функции не могут вызывать друг друга.
- Внутри одной пользовательской коллекции все функции должны быть написаны на одном языке.

💡 ПОДСКАЗКА

- Если вам нужен доступ ко всем встроенным коллекциям, используйте JavaScript. В Python пока доступны только `reactions` и `context`.
- Если вам необходимы библиотеки из экосистемы Python, выбирайте Python.

Коллекции

Функции группируются в коллекции. При вызове функции в коде необходимо указывать коллекцию:

```
// Вызов функции myFunction из коллекции MyCollection  
MyCollection.myFunction();
```

Пользовательские функции можно добавить только в пользовательские коллекции. Если в проекте еще нет таких коллекций, потребуется создать коллекцию при [добавлении функции](#). Для JavaScript- и Python-функций требуется создать отдельные коллекции.

💡 ВСЯ КОЛЛЕКЦИЯ НА ХОЛСТЕ

Чтобы добавить сразу все функции коллекции на [холст](#), просто перетащите туда коллекцию.

Создание и настройка функции

Создание пользовательской функции включает несколько шагов:

1. **Добавление функции.**
2. **Настройка параметров.**
3. **Написание кода.**
4. **Добавление зависимостей.**
5. **Тестирование в редакторе.**

Добавление функции

Добавить функцию можно в левом меню или непосредственно на холсте — эти способы эквивалентны.

В меню На холсте

1. В левом меню выберите , затем нажмите *Добавить* → *функцию*.
2. Выберите или создайте **коллекцию**.

Для новой коллекции заполните поля:

- *Название* коллекции для отображения в интерфейсе. Должно быть уникальным среди пользовательских коллекций.
- *Идентификатор* коллекции для использования в коде. Должен быть уникальным в рамках проекта и не может совпадать со встроенными коллекциями.
- *Язык* — *JavaScript* (по умолчанию) или *Python*. Язык фиксируется для всей коллекции.
- *Описание* для отображения на странице *Функции*.

3. Для новой функции заполните поля:

- *Название* функции для отображения в интерфейсе.
- *Идентификатор* функции для использования в коде.
- *Описание* — особенно важно, если функция будет использоваться в качестве инструмента для AI-агента. Помогает агенту понять, для чего следует вызывать эту функцию.

4. Нажмите *Сохранить*. После сохранения откроется окно настроек функции.

Настройка параметров

Если вы только что создали функцию, редактор параметров откроется автоматически. Чтобы открыть настройки созданной ранее функции, используйте один из способов:

В меню На холсте

1. В левом меню выберите .
2. На странице *Функции* нажмите .

На вкладке *Параметры* укажите:

- **Параметры функции:**

- *Тип* параметра. Поддерживаются как простые типы (`string`, `number`, `boolean`), так и структуры (`object`, `array`) до 5 уровней вложенности.
- *Название* для использования в коде.
- *Описание* — помогает AI-агенту понять, что именно следует передавать в параметре.

💡 ПОДСКАЗКА

Вы можете загрузить параметры в виде JSON-схемы: нажмите *Импортировать* и скачайте образец.

- **Описание ответа:** помогает AI-агенту правильно интерпретировать результат работы функции.

Заказы

Проверить статус заказа

Отмена

Сохранить

Параметры

Код

Зависимости

Параметры и поля ответа

Параметры

Импортировать

Название	Тип	Описание	Обязательно	
orderId	string	Номер заказа	☒	 

Добавить

Поля ответа

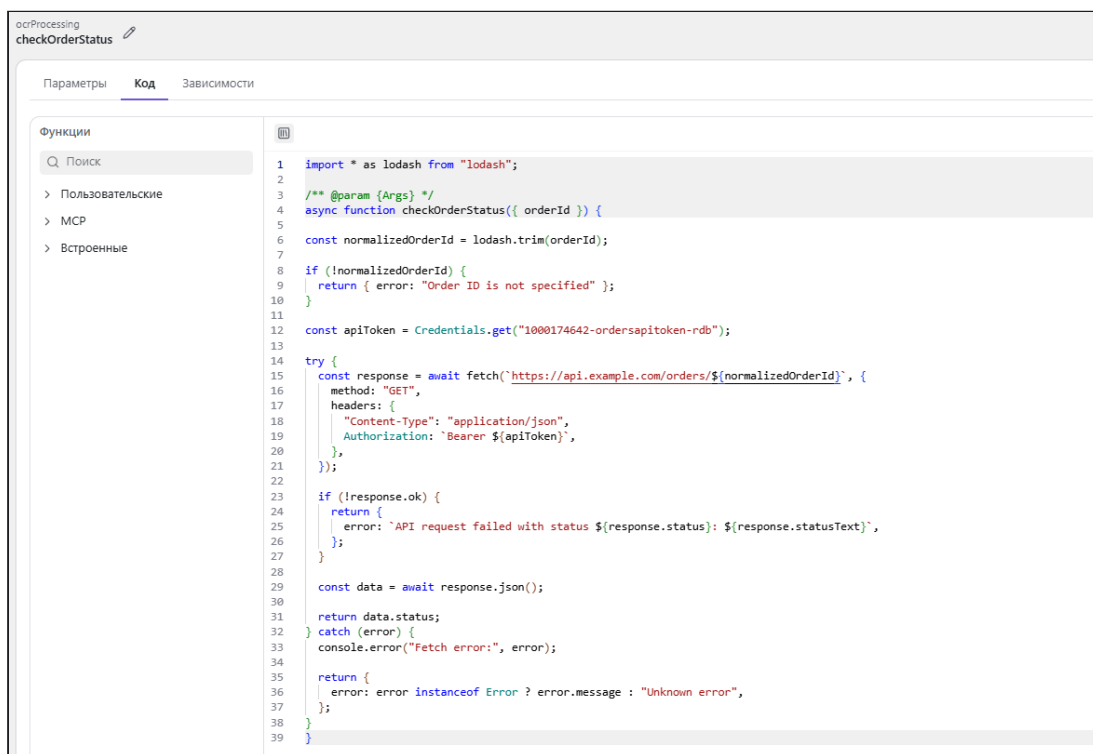
Описание

Статус заказа: UNPAID — не оплачен, PROCESSING — в обработке, DELIVERY — в доставке, DELIVERED — получен покупателем,

Написание кода

На вкладке *Код* напишите тело функции. Встроенный редактор поддерживает подсветку синтаксиса, автодополнение, подсказки при наведении на функцию и отображение ошибок.

В редакторе вы можете менять только тело функции. Чтобы изменить параметры функции, используйте вкладку *Параметры*, а чтобы подключить прт-пакеты — вкладку *Зависимости*.



Чтобы быстро добавлять в свой код вызовы функций — как встроенных, так и пользовательских, и MCP, используйте панель функций:

- Чтобы открыть панель и посмотреть доступные функции, нажмите .
- Чтобы вставить вызов функции в код, дважды нажмите на ее название в панели.
- Чтобы посмотреть описание и параметры функции, выберите ее в панели и нажмите .

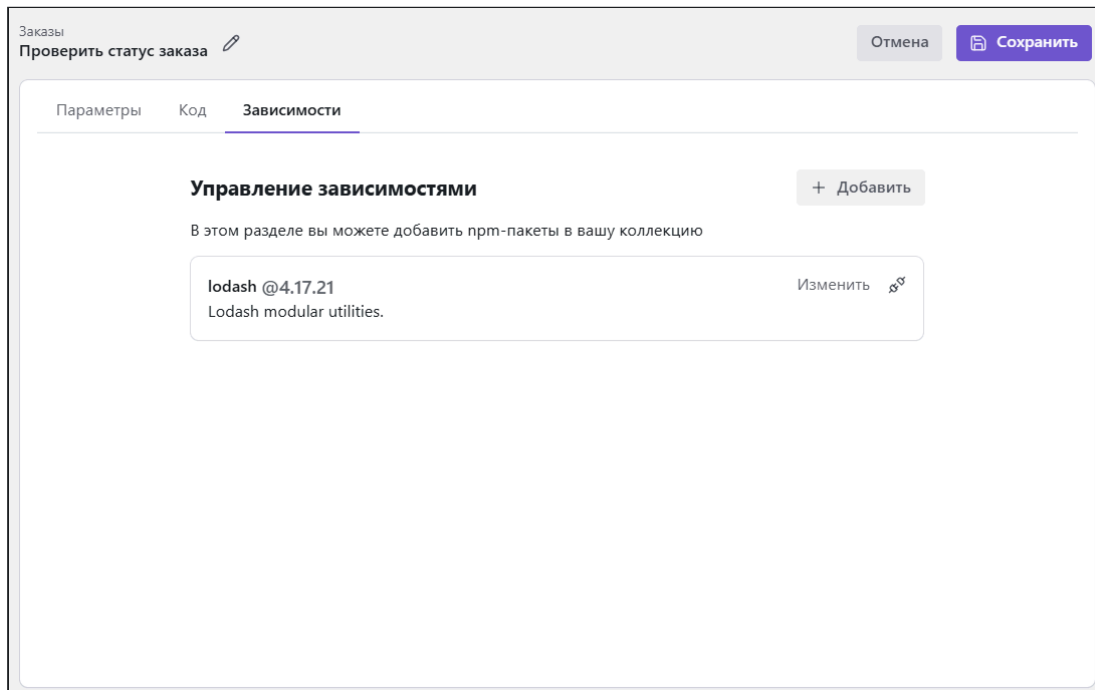
JavaScript **Python**

Особенности и рекомендации:

- Все пользовательские функции являются асинхронными (`async`). Это позволяет использовать в их коде `await` для асинхронных операций, например для выполнения HTTP-запросов с помощью `fetch()`.
- **Не вставляйте учетные данные и другие секреты в код функции.** Добавьте их в хранилище и используйте в коде с помощью встроенной функции `Credentials.get()`. Подробнее смотрите в разделе *Учетные данные для интеграций*.

Зависимости

Если для работы вашего кода требуются внешние пакеты npm (например, `axios` или `lodash`) или Python (например, `requests` или `pandas`), добавьте их на вкладке *Зависимости*.



Добавленные пакеты доступны для использования во всех функциях из той же коллекции. Вместе с тем, управление зависимостями выполняется на уровне проекта в целом. В частности, изменение версии пакета автоматически применяется ко всем коллекциям.

❗ ПРИМЕЧАНИЕ

Agent Platform использует версию Python 3.12. Библиотеки, которые используют конструкции, удаленные в Python 3.12, могут не работать.

Тестирование функции в редакторе

Редактор позволяет запустить функцию с мок-данными прямо в интерфейсе и сразу увидеть результат, ошибки и логи — без запуска всего процесса на холсте.

⚠ ПРЕДУПРЕЖДЕНИЕ

Тестирование функции в редакторе доступно только для JavaScript-функций и только в проектах на движке версии 2.

Чтобы запустить тестирование:

1. На вкладке *Код* нажмите .
2. На вкладке *Аргументы* укажите входные значения.

3. На вкладке *Данные* введите JavaScript-код, который создает мок-данные. Этот код будет выполнен перед тестовым запуском функции.

Например, если внутри функции вызывается `Context.getMessageContent().text`, добавьте следующий объект:

```
const Context = {  
  getMessageContent: function() {  
    return { text: "Тестовое сообщение от пользователя" };  
  },  
};
```

Когда функция вызовет `Context.getMessageContent().text`, она получит ответ из мок-данных.

4. Включите опцию *Запомнить значения для следующих запусков*, чтобы аргументы и данные были доступны при следующем открытии редактора.

5. Нажмите *Запустить*.

После тестирования:

- В панели тестирования появится результат выполнения функции: возвращаемое значение или ошибка.
- В нижней панели на вкладке *Логи* будут отображаться записи, которые функция вывела через `console.log()`.

Использование функций в процессе

Как встроенные, так и **пользовательские** функции можно использовать тремя способами:

- в качестве отдельного шага в процессе;
- в виде инструмента для AI-агента;
- в коде других блоков.

Режим «Функция»

Базовый режим, чтобы использовать функцию как отдельный шаг в процессе.

- **Как работает:** система вызывает выбранную функцию с заданными параметрами.
Для получения результата в следующем блоке используйте метод `Context.getLastFunctionResult()`.
- **Когда использовать:** для задач, где вызов внешней системы — это четкий шаг в процессе.
Например: «получить курс валют», «рассчитать доставку», «проверить статус заказа».

Настройка

1. В левом меню перейдите в *Конструктор* → .
2. Выберите *Функции* → *Встроенные* или *Пользовательские* → *<Нужная коллекция>*.
3. Перетащите функцию на холст и укажите параметры. Вы можете задать их вручную или использовать динамическую подстановку данных, например:
 - Для **движка версии 2**: ID этого канала: `{{ request.botId }}`.
 - Для версии 1: `{{ 'ID этого канала: ' + Context.getBotId() }}`.
4. Если в вашем проекте движок версии 2, вы можете задать параметры на вкладке *Поведение*:
 - *Время ожидания ответа*: до 60 минут. Если выполнение функции не завершится за это время, будет зафиксирована ошибка.
 - *Повторные попытки при ошибке*: до 10 повторных вызовов.
Например, если указать таймаут 20 секунд и количество повторных попыток 3, то суммарное время ожидания может составить до 80 секунд: $20 \times (1+3)$.
 - *Возвращать указанное значение вместо ошибки*: если функция не выполнится успешно после всех попыток, процесс не прервется, а на выход блока будет передано заданное значение (число, строка, JSON-объект, true/false или null).

Режим «Инструмент»

Используется совместно с блоком *Агент*, позволяя AI самостоятельно решать, когда вызывать функцию.

- **Как работает:** вы не вызываете функцию напрямую — она становится «инструментом» для агента. Агент сам решает, когда и как ее использовать, исходя из диалога с пользователем.
- **Когда использовать:** для гибких систем, где агент взаимодействует с внешним миром: например, «забронируй переговорную» или «какой у меня баланс?».

Подтверждение перед вызовом

Если функция проводит платеж, удаляет важные данные или выполняет другое критическое действие, запросите у клиента подтверждение перед ее выполнением.

💡 ПОДСКАЗКА

Подробнее о настройке функций в режиме инструмента смотрите в [разделе про блок «Агент»](#).

Вызов в коде

Функцию можно вызвать:

- Из выражений в блоках *Код* и *Условие*. Пример смотрите в разделах [Код](#) и [Условие](#).
- Из кода других (пользовательских) функций.
- Из параметров других JavaScript-функций (только если в вашем проекте [движок версии 1](#)).

▼ Пример

 **Reactions.sendText** 

Отмена Сохранить

Sends a text message as a response

☐ **Режим инструмента.** Функцию вызывает агент. Ответ не отправляется напрямую пользователю

Параметры функции

text: string
The text content to be sent

При вызове функций указывайте коллекцию, например:

```
// Вызов функции myFunction из коллекции MyCollection  
MyCollection.myFunction();
```

❗ ПРИМЕЧАНИЕ

Из JavaScript-кода нельзя вызывать Python-функции, и наоборот.

Сохранение результата в контекст

Результат выполнения блока функции можно записывать в служебную память, используемую агентами. Подробнее смотрите в разделе [Управление контекстом агентов](#).

Триггеры

Триггеры — это стартовые блоки процесса. Триггер определяет, какое событие и при каких условиях запустит ваш процесс. Когда происходит это событие, триггер активируется и передает управление следующему блоку.

На холст можно добавить несколько триггеров разных типов. Таким образом, один процесс может обрабатывать различные события.

На платформе доступны следующие типы триггеров:

- **Вебхук** — запускается по HTTP-запросу из внешней системы.
- **Планировщик** — активируется по заданному расписанию.
- **Сообщение** — реагирует на сообщения и события в текстовых каналах.
- **IMAP Email** — запускается при получении писем.

Триггеры делятся на два типа:

- **Коммуникационные** — триггер *Сообщение*. Он работает в контексте конкретного канала и клиента: ответ доставляется туда, откуда пришел запрос, а агент видит историю переписки. Функции `Reactions` работают корректно.
- **Workflow** — триггеры *Вебхук*, *Планировщик* и *IMAP Email*. Они запускают процесс сами, без диалога с пользователем, в отдельной служебной сессии `workflow-{triggerId}` — она не привязана к каналам и клиентам. В таком процессе нет истории переписки: агент не видит предыдущих сообщений и работает только с данными текущего запуска — например, телом вебхука или содержимым письма.

⚠ ПРЕДУПРЕЖДЕНИЕ

В workflow-триггерах сессия не привязана к каналу и клиенту, поэтому не работают механизмы, которые отвечают пользователю в канал:

- функции `Reactions` (`sendText` и другие) не доставляют сообщения;
- у блока *Агент* не действуют настройки *Отправлять ответ пользователю* и *Режим диалога*.

Чтобы отправить результат, используйте явные функции — например, `Email.send` или `Telegram.sendText` с указанием `chatID`.

Как добавить триггер на холст

1. Перейдите в проект и на левой панели нажмите , чтобы открыть конструктор.
2. В верхнем левом углу холста нажмите  — откроется список блоков.
3. Раскройте список *Триггеры* и перетащите нужный блок на холст.
4. Соедините триггер со следующим блоком в процессе.

Триггер «Вебхук»

Этот триггер запускает процесс по HTTP-запросу из внешней системы, например из CRM или ERP.

Особенности:

- Для каждого блока *Вебхук* на холсте автоматически генерируется уникальный URL.
- Вебхук не требует дополнительной аутентификации. Любой, у кого есть ссылка, может запустить связанный с ней процесс.
- Работает в изолированной служебной сессии `workflow-{triggerId}`, не привязанной к каналам и клиентам. У процесса нет истории чата — агент работает с данными текущего запроса.

Настройка

Дважды нажмите на блок *Вебхук* и скопируйте уникальный URL.

Запуск без параметров (GET)

Чтобы запустить процесс, отправьте GET-запрос на URL вашего вебхука:

```
curl "https://agentplatform.just-ai.com/api/text-campaign-service/scenario-triggers/.../invoke"
```

ПОДСКАЗКА

Чтобы быстро проверить вебхук, просто откройте ссылку в браузере.

При успешном вызове сервер вернет `200 OK`, и процесс запустится.

Запуск с передачей данных (POST)

Чтобы запустить процесс и передать данные, отправьте POST-запрос на URL вашего вебхука и укажите данные в теле:

```
curl -v -X POST "https://agentplatform.just-ai.com/api/text-campaign-service/scenario-triggers/.../invoke" \  
-H "Content-Type: application/json" \  
-d '{  
  "customer": {  
    "name": "Alex",  
    "email": "alex@example.com"  
  },  
  "order": {  
    "status": "created"  
  }  
}'
```

Функция `Context.getMessageContent()` возвращает эти данные в поле `payload`:

```
// Данные из тела запроса  
const data = Context.getMessageContent().payload;
```

Триггер «Планировщик»

Этот триггер запускает процесс по заданному расписанию. Он идеально подходит для автоматизации рутинных задач, таких как формирование отчетов или отправка уведомлений во внешние системы.

Особенности:

- Работает в изолированной служебной сессии `workflow-{triggerId}`, не привязанной к каналам и клиентам. У процесса нет истории чата — агент работает с данными текущего запуска.
- Планировщик не передает в процесс никакую дополнительную информацию.

Настройка

1. Дважды нажмите на блок *Планировщик*, чтобы открыть настройки.
2. Выберите вариант расписания:
 - *Простое* — для быстрой настройки с помощью готовых интервалов. Укажите дату и время первого запуска, выберите периодичность и условие окончания.

⚠ ПРЕДУПРЕЖДЕНИЕ

Если время первого запуска уже прошло и выбран вариант *Не повторять*, то триггер запустится один раз сразу после сохранения.

- *Cron-выражение* — для более гибкой настройки. Укажите Cron-выражение в формате **Quartz** и условие окончания. Например, `0 30 9 ? * MON` — каждый понедельник в 9:30.

После настройки расписания платформа отобразит информацию о ближайших запланированных запусках (не более 5).

3. Нажмите *Запустить*.

❗ К СВЕДЕНИЮ

Новое расписание начинает работать сразу, как только вы нажали *Запустить*, независимо от публикации процесса. Публикация нужна, только если вы меняли логику самого процесса: например, добавляли блоки или меняли код функций.

Триггер «Сообщение»

Этот триггер срабатывает, когда пользователь отправляет сообщение или происходит **событие**. Он также срабатывает при запуске тестового виджета с включенной настройкой «**Отправлять /start**».

Каналы работают независимо друг от друга. Ответ будет доставлен в тот канал, откуда пришло исходное событие.

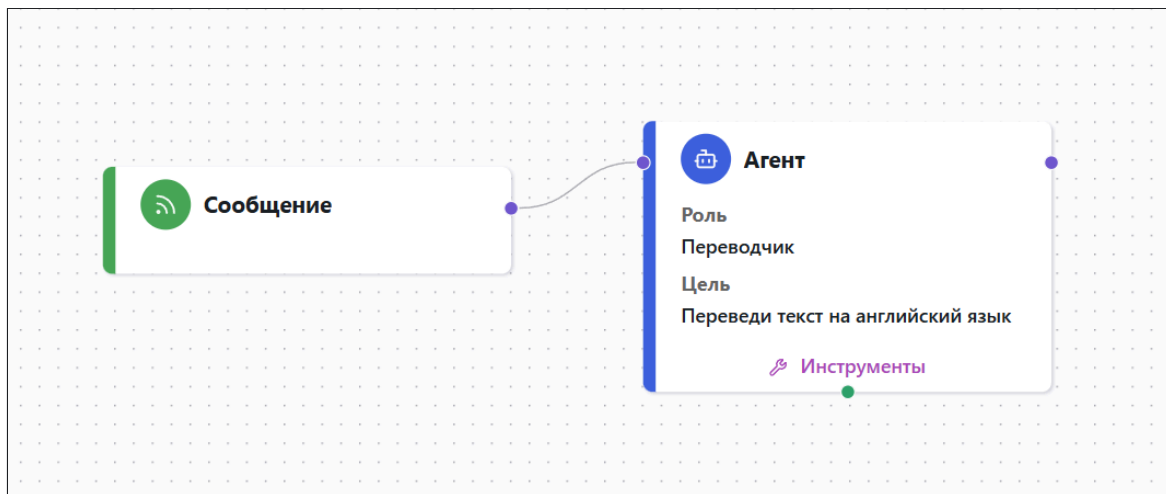
Настройка

В триггере *Сообщение* автоматически подключены все настроенные в проекте каналы. Добавить или редактировать каналы можно как в разделе *Интеграции*, так и в самом триггере — эти способы эквиваленты. Если в проекте не подключен ни один канал, триггер срабатывает только в тестовом виджете.

Использование

С агентом

Соедините триггер *Сообщение* с блоком *Агент*. Агент обработает сообщение пользователя в соответствии со своим промтом и другими настройками. Примеры смотрите в разделах **Быстрый старт: Шаг 2** и **Работа агента в процессе**.



С функцией

Триггер *Сообщение* можно соединить с функцией. Для извлечения различных данных о сообщении используйте встроенные функции из раздела *Context*. Например, функция `Context.getMessageContent()` возвращает содержание сообщения, `Context.getChannelType()` — тип канала.

Рассмотрим тривиальный пример эхо-бота:

1. Добавьте на холст триггер *Сообщение* и функцию `Reactions.sendText()`, соедините их последовательно.
2. В настройках функции `Reactions.sendText()` укажите значение:
 - Для **движка версии 2**: `Вы сказали: {{ + request.query }}`.
 - Для версии 1: `{{ "Вы сказали: " + Context.getMessageContent().text }}`.

Такой бот на сообщение «Привет» ответит «Вы сказали: Привет».



Обработка событий

Триггер *Сообщение* срабатывает не только на текстовые сообщения, но и на события — например, если пользователь отправил файл или положил трубку.

Чтобы получить данные о событии, используйте функцию `Context.getMessageContent()`.

Пример обработки изображений:

1. Добавьте блок **Условие** после триггера *Сообщение*.
2. Напишите JS-выражение для проверки полей `event` и `eventData`. Их содержимое зависит от действий пользователя:

Отправил текст **Отправил картинку**

```
{
  "text": "Хочу сделать заказ",
  "files": [],
  "eventData": {}
}
```

💡 ПОДСКАЗКА

Набор полей в `eventData` зависит от типа события и канала.

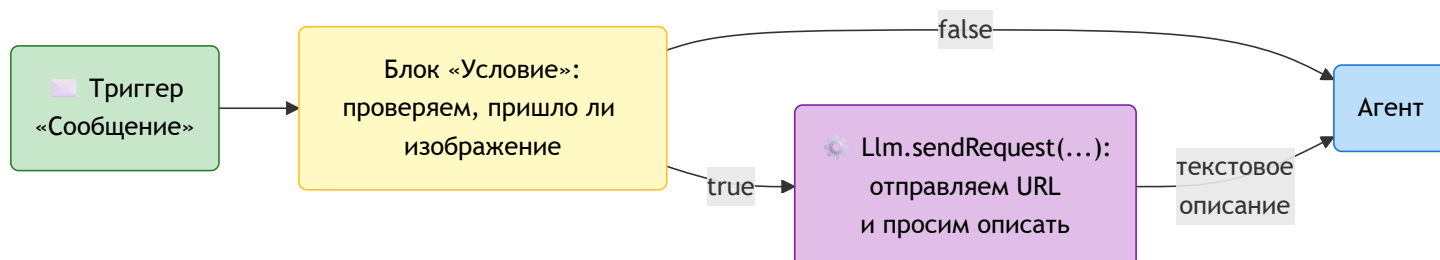
Так вы сможете разветвить процесс — обрабатывать текстовые сообщения и события по-разному. Чтобы обрабатывать события с изображениями, можно использовать условие:

```
Context.getMessageContent().event === 'fileEvent' &&
Context.getMessageContent().eventData['0'].type === 'image'
```

3. Поскольку агент принимает на вход текст, получите описание изображения — например, с помощью функции `Llm.sendRequest()`.
4. Передайте описание агенту. Для этого в его настройках, в поле *Запрос для агента*, укажите:

```
{{ Context.getMessageContent().text || Context.getLastFunctionResult().content }}
```

Так агент будет получать текст из сообщения или, если его нет, результат функции с описанием изображения.



Список событий

Событие	Описание	Канал
<code>fileEvent</code>	Пользователь отправил файл	Все текстовые каналы
<code>locationEvent</code>	Пользователь отправил геолокацию	Telegram
<code>contactEvent</code>	Пользователь отправил контакт	Telegram
<code>stickerEvent</code>	Пользователь отправил стикер	Telegram
<code>hangup</code>	Пользователь положил трубку	Телефонный канал
<code>botHangup</code>	Ассистент завершил звонок с помощью <code>Dialer.hangUp()</code>	Телефонный канал
<code>transfer</code>	Выполнен перевод звонка на оператора или произошла ошибка при переводе	Телефонный канал

Триггер «IMAP Email»

Этот триггер подключается к почтовому ящику по протоколу IMAP. Он отслеживает появление новых писем в указанной папке и запускает процесс при их получении.

Особенности:

- Передает в контекст полную информацию о письме: отправитель, получатели (to/cc/bcc), тема, содержимое (HTML и plain текст), вложения, заголовки, флаги, дата отправки. Эти данные доступны через функцию `Context.getEmailContent()`.
- Обрабатывает письма последовательно (по одному за раз). Если писем много, обработка очереди может занять некоторое время.
- Срабатывает не мгновенно, а с периодичностью опроса почтового сервера (polling).
- Требуется предварительной настройки подключения к IMAP-серверу в разделе *Интеграции*.
- Работает в изолированной служебной сессии `workflow-{triggerId}`, не привязанной к каналам и клиентам. У процесса нет истории чата — агент работает с данными текущего письма.

Настройка

Для работы триггера настройте IMAP-подключение к почтовому ящику.

Шаг 1. Настройте IMAP-подключение

Добавьте учетные данные:

1. В проекте перейдите в раздел *Учетные данные*. Подробнее о нем читайте в [статье](#).
2. Добавьте новые учетные данные с типом *IMAP*. У некоторых почтовых сервисов могут быть **особые требования к паролю**, обратитесь за точной информацией к документации вашего сервиса.

Далее создайте интеграцию:

1. Перейдите в раздел *Интеграции*.
2. Создайте и настройте новую интеграцию с типом **IMAP-сервер**.

Шаг 2. Настройте блок «IMAP Email»

1. Дважды нажмите на блок *IMAP Email*.
2. Выберите созданную интеграцию и задайте *Имя почтового ящика* (папка на сервере, которую нужно отслеживать, например, `INBOX` или `ReadyForAI`).
3. Настройте логику обработки писем:
 - *Загружать только новые письма*: если включено, триггер будет игнорировать письма, у которых уже стоит флаг «Прочитано». Это предотвращает повторную обработку старых писем при перезапуске процесса.
 - *Помечать прочитанными*: если включено, после успешного взятия письма в работу триггер установит на сервере для письма статус «Прочитано».

РЕКОМЕНДОВАННАЯ СВЯЗКА

Включите обе опции. Это гарантирует, что каждое письмо будет обработано только один раз.

4. Настройте скачивание вложений, если они необходимы для процесса.
5. При необходимости настройте фильтр по дате (обрабатывать письма начиная с определенного числа).
6. Нажмите *Запустить*.

После публикации процесса триггер начнет периодически проверять указанную папку на наличие писем, удовлетворяющих условиям.

Рекомендации по использованию

Для стабильной работы и экономии ресурсов, в том числе токенов LLM при использовании AI-агентов, мы рекомендуем придерживаться следующего подхода:

Отбор писем в папку

В текущей версии триггера нет сложных фильтров по теме или отправителю. Хотя вы можете отфильтровать ненужные письма логикой внутри процесса (например, блоком **Условие** на основе `Context.getEmailContent()`), загрузка лишних писем на платформу неэффективна: это замедляет обработку и забивает очередь.

Кроме того, если для обработки писем используется AI-агент, он будет читать лишние письма и тратить токены. Предварительная фильтрация на уровне почтового сервера обеспечивает максимальную прозрачность: вы точно знаете, какие письма попадут в обработку.

Лучшая практика:

1. Создайте в почтовом ящике (Outlook, Gmail, Яндекс или другой) отдельную папку для интеграции (например, `ReadyForAI`).
2. Настройте на стороне почтового сервера автоматическое правило (Filter/Rule), которое перемещает нужные письма в эту папку.
3. В настройках триггера укажите именно эту папку.

Работа с группами рассылки

Если для интеграции используется адрес вроде `support@company.com`, уточните у системного администратора его тип:

- Полноценный почтовый ящик: можно подключаться по IMAP напрямую, используя логин и пароль.
- Группа рассылки (Alias/Distribution Group): у такого адреса нет собственного хранилища, письма пересылаются участникам группы. Подключить такую группу по IMAP может быть невозможно. В таком случае попросите администратора создать реальный служебный ящик, включить его в группу рассылки и подключите триггер к нему.

Использование данных письма

Для извлечения различных данных о письме используйте функцию `Context.getEmailContent()` из коллекции `Context`. Например, `Context.getEmailContent().subject` возвращает тему письма, `Context.getEmailContent().from` — адрес отправителя.

Рассмотрим пример процесса, который отправляет уведомление о полученном письме в Telegram.

⚠ ПРЕДУПРЕЖДЕНИЕ

IMAP Email — это workflow-триггер, поэтому функции `Reactions` не доставят сообщение: сессия не привязана к каналу или клиенту. Для отправки уведомления используйте явные функции, например `Telegram.sendText` с указанием `chatID`.

Процесс отличается в зависимости от **версии движка** в проекте.

Версия 2 Версия 1

1. Добавьте на холст триггер *IMAP Email*.
2. Добавьте блок функции `Context.getEmailContent()` и соедините с триггером.
3. Добавьте блок функции `Telegram.sendText()` и соедините с предыдущим блоком.
4. В настройках функции `Telegram.sendText()` укажите:
 - `credentialKey` — **учетные данные** с токеном вашего Telegram-бота.
 - `chatId` — `chatID` получателя.
 - `text` — значение: `Получено письмо от {{ previousResult.from }} с темой: {{ previousResult.subject }}`

Теперь при получении письма от `client@example.com` с темой «Заказ № 123» в указанный Telegram-чат придет уведомление: «Получено письмо от **client@example.com** с темой: Заказ № 123».

Блок «Код»

С помощью блока «Код» вы можете добавить в процесс фрагмент кода на JavaScript или Python. Это универсальный блок для вычислений, логики и работы с данными, контекстом и переменными.

❗ ОТЛИЧИЕ ОТ ПОЛЬЗОВАТЕЛЬСКИХ ФУНКЦИЙ

Используйте блок «Код» для кода, который будете выполнять один раз, а пользовательские функции — для кода, который будете переиспользовать.

Как добавить блок «Код» на холст

1. Перейдите в проект и на левой панели нажмите , чтобы открыть конструктор процессов.
2. В верхнем левом углу холста нажмите  — откроется список блоков.
3. Из раздела *Код и логика* перетащите блок *Код* на холст.

Интерфейс редактора кода

Чтобы настроить блок, дважды нажмите на него на холсте. В этом блоке доступны следующие элементы:

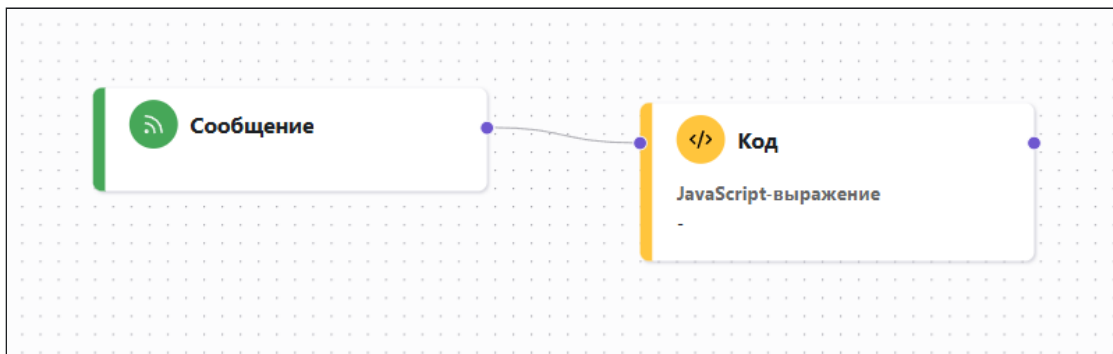
- Выбор языка: JavaScript или Python.
- Редактор кода с подсветкой синтаксиса и автокомплитом.
- Библиотека функций . Панель быстрого поиска встроенных и пользовательских функций проекта. Двойной клик — вставка вызова функции в код.
- Панель документации. Отображает подсказки и описание выбранной функции: имя, тип, описание. Автоматически появляется в правой части окна при нажатии на любую функцию из библиотеки.

Пример использования

Для примера создадим блок, который считает количество слов в сообщении.

1. Перейдите в проект и на левой панели нажмите , чтобы открыть конструктор процессов.
2. В верхнем левом углу холста нажмите  — откроется список блоков.
3. Перетащите блок *Триггеры* → *Сообщение* на холст.

4. Перетащите блок *Код и логика* → *Код* на холст и соедините его с блоком *Сообщение*.



5. Дважды нажмите на блок *Код*.

6. Вставьте в него следующий код, в котором используются встроенные функции **Context** и **Reactions**.

▼ JavaScript

Код зависит от используемой в проекте версии движка:

▼ Версия 2

```
// Получаем объект с сообщением
const content = Context.getMessageContent();

// Проверяем, есть ли вообще текст
if (!content || typeof content.text !== 'string') {
  Reactions.sendText({text: "Я не вижу сообщения"});
  return;
}

// Извлекаем текст и убираем лишние пробелы
const message = content.text.trim();

// Разбиваем сообщение по пробелам и фильтруем пустые элементы
const words = message.split(/\s+/).filter(Boolean);

// Считаем количество слов
const count = words.length;

// Отправляем ответ
Reactions.sendText({
  text: `В вашем сообщении ${count} ${count === 1 ? "слово" : "слов(а)"}`
});
```

▼ Версия 1

```
// Получаем объект с сообщением
const content = await Context.getMessageContent();

// Проверяем, есть ли вообще текст
if (!content || typeof content.text !== 'string') {
  Reactions.sendText("Я не вижу сообщения");
  return;
}

// Извлекаем текст и убираем лишние пробелы
const message = content.text.trim();

// Разбиваем сообщение по пробелам и фильтруем пустые элементы
const words = message.split(/\s+/).filter(Boolean);

// Считаем количество слов
const count = words.length;

// Отправляем ответ
Reactions.sendText(`В вашем сообщении ${count} ${count === 1 ? "слово" : "слов(а)"}`);
```

▼ Python

```
# Получаем объект с сообщением
content = sdk.context.get_message_content()

# Проверяем, есть ли вообще текст
if not content or not isinstance(content.get("text"), str):
    await sdk.reactions.send_text({"text": "Я не вижу сообщения"})
    return

# Извлекаем текст и убираем лишние пробелы
message = content["text"].strip()

# Разбиваем сообщение по пробелам и фильтруем пустые элементы
words = [w for w in message.split() if w]

# Считаем количество слов
count = len(words)
```

```
# Отправляем ответ
```

```
word_form = "слово" if count == 1 else "слов(а)"
```

```
await sdk.reactions.send_text({"text": f"В вашем сообщении {count} {word_form}"})
```

❗ ПРИМЕЧАНИЕ

В блоке «Код» на Python доступна глобальная переменная `sdk` и функции `sdk.reactions.*` и `sdk.context.*`. Подробнее смотрите в разделе [Встроенные функции](#).

7. Нажмите *Сохранить*.

8. В конструкторе нажмите *Тестировать*. Откроется окно чата, и вы сможете проверить работу вашего кода.

Рекомендации

- Используйте явные имена переменных, чтобы вам сразу было понятно, что она делает. Например, если вы сохраняете в нее текст сообщения, то назовите ее `message`.
- Старайтесь не делать слишком большой и сложный код — лучше разбить на несколько функций и блоков.

Блок «Условие»

Блок «Условие» позволяет создавать ветвления внутри процесса. От результата проверки условия зависит, по какому пути продолжится выполнение процесса.

Этот блок подходит для:

- Направления сценария в зависимости от полученных данных.
- Обработки результатов внешних API.
- Реализации гибкой пошаговой логики.

Для условий поддерживается только язык JavaScript.

Как добавить блок «Условие» на холст

1. Перейдите в проект и на левой панели нажмите , чтобы открыть конструктор процессов.
2. В верхнем левом углу холста нажмите  — откроется список блоков.
3. Перетащите блок *Код и логика* → *Условие* на холст.

Интерфейс настройки условия

После добавления блока на холст вы можете задать логику и указать, какая ветка будет выполнена при выполнении или невыполнении условия.

1. *JavaScript-выражение*: логическое выражение в JavaScript.
2. *Если True* — *перейти в* и *Если False* — *перейти в*: из выпадающего списка выберите следующий блок.

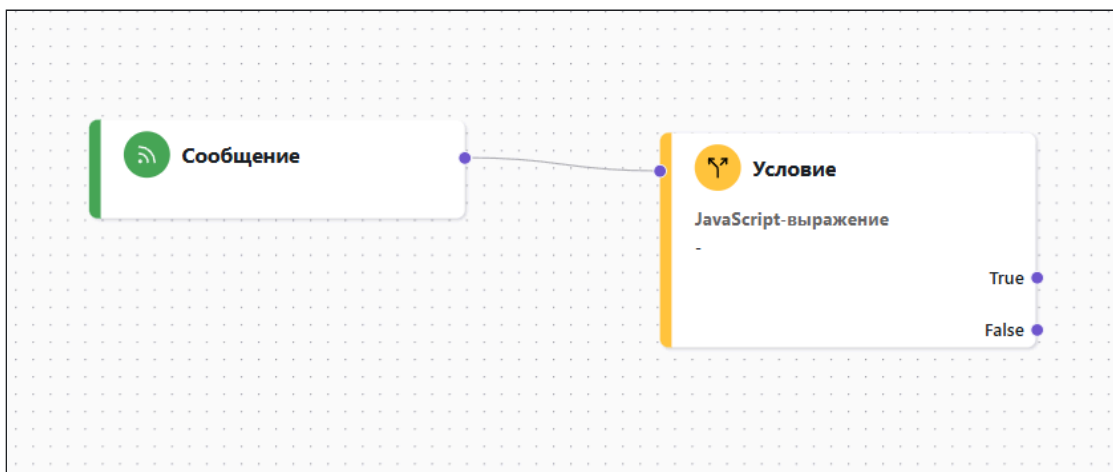
Вы также можете перетащить соединение от *True* или *False* к нужному блоку в конструкторе.

Примеры использования

Для примера проверим, будет ли значение числа больше 100 или нет.

1. Перейдите в проект и на левой панели нажмите , чтобы открыть конструктор процессов.
2. В верхнем левом углу холста нажмите  — откроется список блоков.
3. Перетащите блок *Триггеры* → *Сообщение* на холст.

4. Перетащите блок *Код и логика* → *Условие* на холст и соедините его с блоком *Сообщение*.

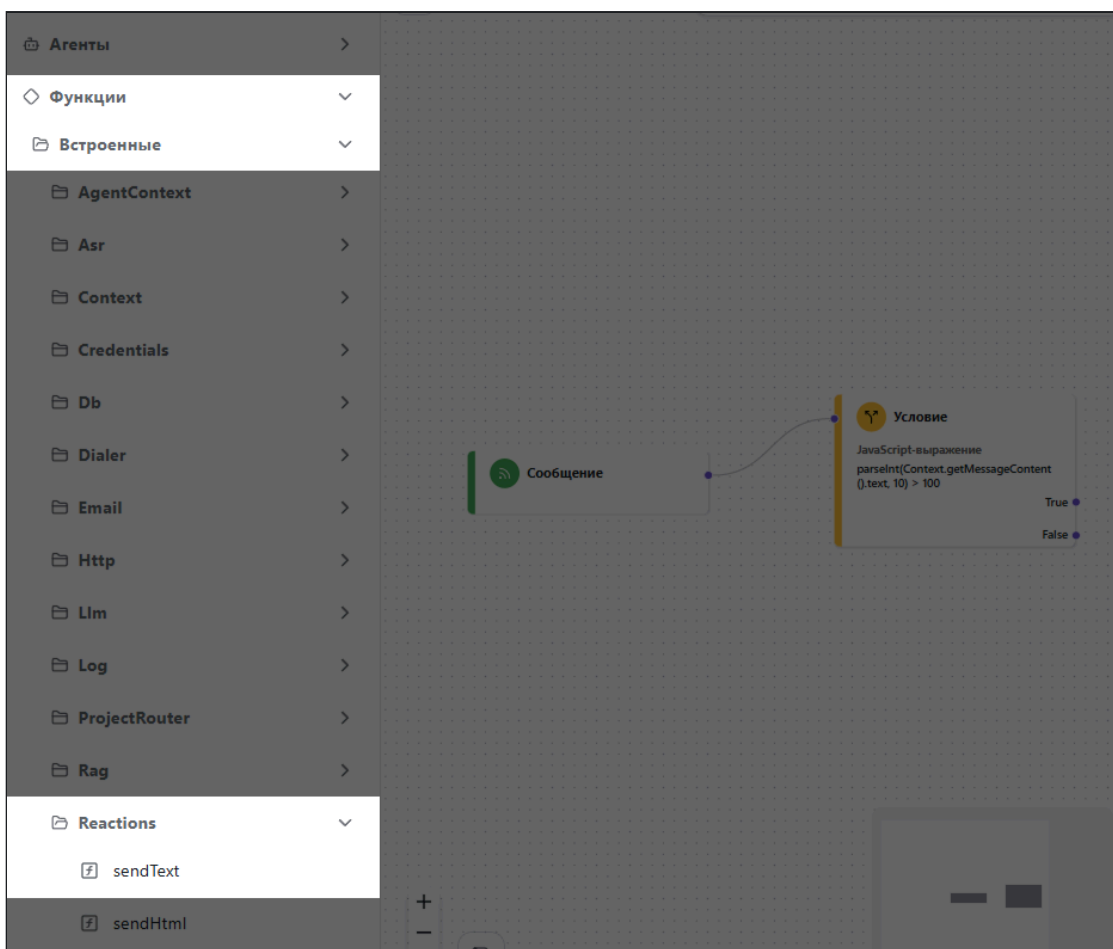


5. Дважды нажмите на блок *Условие* на холсте, чтобы открыть его.

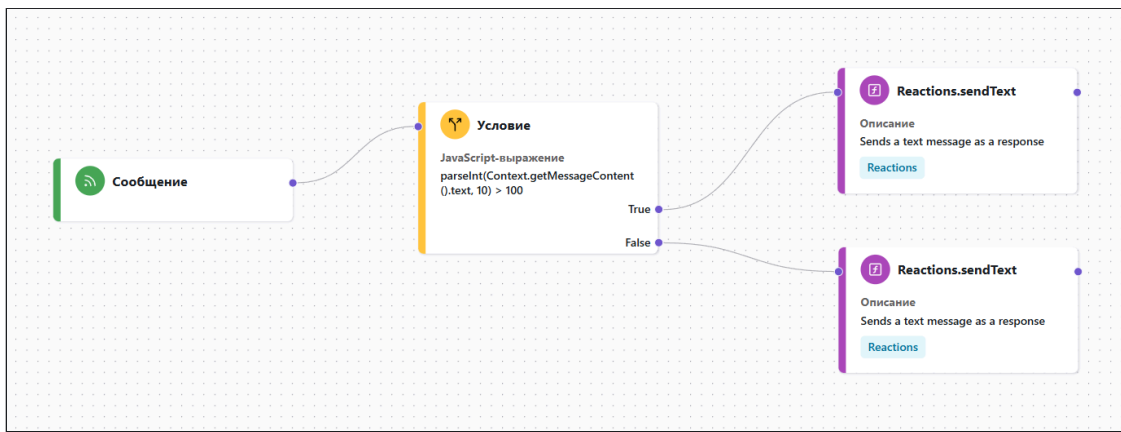
6. В поле *JavaScript-выражение* вставьте:

```
parseInt(Context.getMessageContent().text, 10) > 100
```

7. Перетащите на холст два блока *Функции* → *Встроенные* → *Reactions* → *SendText*.



8. Соедините один блок *SendText* со значением *True*, а другой со значением *False* на блоке *Условие*.



9. Откройте блок, который соединен с *True*, и в параметр *text* вставьте **Больше 100**.
10. Откройте блок, который соединен с *False*, и в параметр *text* вставьте **Меньше 100**.
11. В конструкторе нажмите *Тестировать*. Откроется окно чата, и вы сможете проверить работу вашего процесса. Отправьте в чат число, а вам придет ответ с результатом сравнения.

Рекомендации

- Используйте блок *Код* до блока *Условие* для подготовки данных.
- Проверяйте ошибки в логах, если условие ведет не по тому сценарию.

Подстановки {{ ... }}

⚠ ПРЕДУПРЕЖДЕНИЕ

В этой статье описано использование подстановок в проектах с движком [версии 2](#).
Подстановки для этой версии реализованы на основе [Pebble Templates](#).

В блоках конструктора — в параметрах функций и агентов, блоках JavaScript-кода — можно динамически подставлять данные с помощью выражений в скобках `{{ ... }}`.

Доступные данные

- `previousResult` — результат выполнения предыдущего блока: функции или агента.
- Данные запроса, отправленного клиентом (набор полей может отличаться в зависимости от канала):

Имя	Описание
<code>request.query</code>	Текст запроса
<code>request.requestId</code>	Идентификатор запроса
<code>request.botId</code>	Идентификатор канала
<code>request.channelType</code>	Тип канала (<code>telegram</code> , <code>chatwidget</code> , ...)
<code>request.channelTypeIsAsync</code>	Асинхронный ли канал
<code>request.isTestChannel</code>	Тестовый ли канал
<code>request.chatId</code>	Идентификатор чата
<code>request.sessionId</code>	Идентификатор сессии
<code>request.clientInfo.id</code>	ID клиента
<code>request.clientInfo.firstName</code>	Имя клиента

Имя	Описание
<code>request.clientInfo.lastName</code>	Фамилия клиента

Пример:

Здравствуйте, {{ request.clientInfo.firstName }}! Канал: {{ request.channelType }}

- Ключи, установленные с помощью встроенной функции `Context.set(key, value)`, доступны по имени:

// В коде:
Context.set({key: "name", value: "Alice"});
Context.set({key: "address", value: { "city": "Moscow" }});

{{ name }} → Alice
{{ address.city }} → Moscow

Подстановки, которые не удалось разрешить, остаются в строке «как есть».

Синтаксис подстановок

Синтаксис	Описание	Пример
<code>{{ variable }}</code>	Переменная, см. Доступные данные выше	<code>{{ name }}</code> → Alice
<code>{{ a.b.c }}</code>	Доступ по вложенному пути	<code>{{ user.address.city }}</code> → Berlin
<code>{{ list[0] }}</code>	Доступ по индексу массива	<code>{{ items[1] }}</code> → beta
<code>{{ map["key"] }}</code>	Доступ по строковому ключу	<code>{{ headers["Content-Type"] }}</code> → application/json
<code>{{ a + b }}</code>	Арифметика (+, -, *, /, %)	<code>{{ price * qty }}</code> → 30
<code>{{ a >= b }}</code>	Сравнение (==, !=, >, <, >=, <=)	<code>{{ age >= 18 }}</code> → true

Синтаксис	Описание	Пример
<code>{{ a and b }}</code>	Логические операторы (<code>and</code> , <code>or</code> , <code>not</code>)	<code>{{ a and b }}</code> → <code>false</code>
<code>{{ a ~ " " ~ b }}</code>	Конкатенация строк (оператор <code>~</code>)	<code>{{ first ~ " " ~ last }}</code> → <code>John Doe</code>
<code>{{ val filter }}</code>	Применение фильтра	<code>{{ name upper }}</code> → <code>ALICE</code> См. Фильтры
<code>{{ val f1 f2 }}</code>	Цепочка фильтров	<code>{{ name trim upper }}</code> → <code>ALICE</code>
<code>{% if %}...{% endif %}</code>	Условный блок	См. Условия и циклы
<code>{% for %}...{% endfor %}</code>	Цикл	См. Условия и циклы
<code>{{ "literal" }}</code>	Строковый литерал	<code>{{ "hello" }}</code> → <code>hello</code>
<code>{{ 42 }}</code>	Числовой литерал	<code>{{ 42 }}</code> → <code>42</code>

Фильтры

Встроенные фильтры Pebble

Фильтр	Описание	Пример
<code>upper</code>	Верхний регистр	<code>{{ name upper }}</code> → <code>ALICE</code>
<code>lower</code>	Нижний регистр	<code>{{ name lower }}</code> → <code>alice</code>
<code>capitalize</code>	Первая буква заглавная	<code>{{ name capitalize }}</code> → <code>Alice</code>
<code>trim</code>	Удаление пробелов по краям	<code>{{ text trim }}</code> → <code>hello</code>

Фильтр	Описание	Пример
title	Каждое слово с заглавной	{{ text title }}
default(val)	Значение по умолчанию (если null)	{{ name default("Guest") }} → Guest
first	Первый элемент коллекции	{{ items first }}
last	Последний элемент коллекции	{{ items last }}
join(sep)	Объединение коллекции в строку	{{ items join(", ") }} → a, b, c
split(sep)	Разбиение строки в список	{{ text split(",") }}
reverse	Обратный порядок	{{ items reverse }}
sort	Сортировка	{{ items sort }}
slice(from, to)	Подмассив	{{ items slice(0, 2) }}
replace(map)	Замена подстрок	{{ text replace({"old": "new"}) }}
length	Длина строки/коллекции	{{ items length }} → 3

Фильтры Agent Platform

Фильтр	Описание	Пример
json	Сериализация значения в JSON-строку	{{ obj json }} → {"key":"value"}
parseJson	Парсинг JSON-строки в объект	{{ '{"a":1}' parseJson }}
string	Приведение к строке (пустая строка для null)	{{ count string }}

Фильтр	Описание	Пример
<code>truncate(length, suffix?)</code>	Обрезка строки до указанной длины с добавлением суффикса (по умолчанию <code>...</code>), который учитывается в длине	<code>{{ text truncate(12) }}</code> → <code>Once upon...</code>

Условия и циклы

Условный блок `{% if %}`

```
{% if age >= 18 %}adult{% else %}minor{% endif %}
```

Поддерживаются вложенные `{% elseif %}`:

```
{% if score >= 90 %}отлично{% elseif score >= 70 %}хорошо{% else %}удовлетворительно{% endif %}
```

Условие может быть любым выражением, включая логические операторы:

```
{% if show %}visible{% endif %}
```

⚠ ПРЕДУПРЕЖДЕНИЕ

Шаблоны с управляющими блоками `{% %}` вычисляются целиком, а не по отдельным выражениям. Неизвестные переменные внутри таких блоков трактуются как `null/false`.

Цикл `{% for %}`

```
{% for item in items %}{{ item }},{% endfor %}
```

Внутри цикла доступна переменная `loop`:

Переменная	Описание
<code>loop.index</code>	Индекс текущей итерации (с 0)

Переменная	Описание
<code>loop.length</code>	Общее количество элементов
<code>loop.first</code>	<code>true</code> для первой итерации
<code>loop.last</code>	<code>true</code> для последней итерации

Пример с индексом:

```
{% for item in items %}{{ loop.index }}:{{ item }},{% endfor %}
```

При `items = ["x", "y"]` результат: `0:x,1:y,.`

Примеры

Инструкция агента

```
Ты – ассистент службы поддержки {{ companyName }}.
```

Текст TTS с условиями

```
{% if request.clientInfo.firstName %}Здравствуйте, {{ request.clientInfo.firstName }}!  
{% else %}Здравствуйте!{% endif %} Чем могу помочь?
```

JavaScript-код

```
const count = {{ items | length }}
```

Фильтрация и форматирование

```
Описание: {{ description | truncate(200) }}  
Данные: {{ previousResult | json }}
```

Встроенные базы данных

Используйте встроенные базы данных, чтобы хранить информацию по проекту: списки клиентов, адреса магазинов, настройки или товары.

Формат данных

Все данные хранятся в формате ключ-значение:

- **Ключ (key)** — строка, которая однозначно идентифицирует запись в базе данных.
- **Значение (value)** — JSON-объект, который содержит любые данные.

Примеры записей

Справочник

Клиенты

Для небольших списков, которые нужны целиком (например, адреса филиалов), можно использовать одну запись.

- Ключ: `pharmacy_addresses`
- Значение:

```
{
  "main_office": "ул. Примерная, дом 3",
  "branch_pushkin": "ул. Пушкина, дом 5",
  "city": "Москва"
}
```

Создание баз данных

Общая база для аккаунта

Чтобы создать базу данных, которая будет доступна во всех проектах аккаунта:

1. Перейдите на главную страницу — к списку всех проектов Agent Platform.
2. Выберите *Базы данных* в меню слева.
3. Нажмите *Создать базу данных*.

4. Введите название базы данных и нажмите *Добавить*.

Такая база имеет область видимости *ACCOUNT*.

База данных для проекта

Чтобы создать базу данных, которая будет доступна только в одном проекте:

1. Перейдите в проект.
2. В меню слева выберите *Базы данных*.

ПРИМЕЧАНИЕ

Вы можете также добавить базу данных в разделе *Интеграции*. Процесс добавления не отличается.

3. Нажмите *Создать базу данных*.
4. Введите название.
5. Выберите область видимости:
 - *PROJECT* — база данных будет доступна в этом проекте вне зависимости от сессии, канала или клиента.
 - *CHANNEL* — база привязана к конкретному каналу. Данные из разных каналов не пересекаются.
 - *CLIENT* — база привязана к конкретному клиенту. У каждого клиента будет свой независимый набор записей.
 - *SESSION* — база данных для временного хранения внутри одного диалога. Обычно ее не требуется создавать вручную. Подробнее о работе с такими базами читайте в разделе [Хранение данных внутри диалога](#).
6. Нажмите *Добавить*.

Управление записями через интерфейс

Прямо в интерфейсе платформы можно просматривать, добавлять, редактировать и удалять записи.

ОГРАНИЧЕНИЯ

Сейчас интерфейс поддерживает работу только с базами *ACCOUNT* и *PROJECT*. Записями в остальных базах можно управлять только через [код процесса](#).

Просмотр записей

Чтобы перейти к записям базы данных:

1. Перейдите в раздел *Базы данных* на главной странице или в конкретном проекте.
2. Нажмите на карточку вашей базы данных. Вы увидите список всех записей, которые есть в базе данных.

Поиск

Поиск проходит по всем записям в базе данных и возвращает те, чьи **значения (JSON)** соответствуют критериям. Для сложных запросов, например, чтобы найти числа больше определенного значения или указать логические условия, используйте **операторы MongoDB**.

Запрос должен быть в формате JSON: заключайте условия поиска в фигурные скобки `{...}`.

Примеры:

- Найти конкретного человека по email:

```
{"email": "ivanov@example.com"}
```

- Найти всех клиентов старше 25 лет. Используется оператор `$gt` (greater than):

```
{"age": {"$gt": 25}}
```

- Найти клиентов, у которых тариф базовый или которые отключили автопродление. Используется оператор `$or`:

```
{ "$or": [ {"tariff": "basic"}, {"auto_renewal": false} ] }
```

Добавление записи

1. Нажмите *Добавить запись*.
2. Укажите ключ, например `client:ivanov`.
3. Введите значение в формате JSON, например: `{"name": "Иванов Иван", "age": 30}`.
4. Нажмите *Добавить*.

Редактирование записи

1. Нажмите  у записи.
2. Укажите новое значение в формате JSON.

⚠ ПРЕДУПРЕЖДЕНИЕ

Технически ключ нельзя переименовать. Если вы измените ключ при редактировании, система создаст запись с новым ключом, а старая запись останется на месте.

3. Нажмите *Сохранить*.

Как использовать в процессе

Управлять данными прямо в процессе можно с помощью функций **Db**. Вы можете добавить блоки этих функций в процесс или подключить их к агенту.

💡 ПОДСКАЗКА

Для вызова функций **Db** в коде вам потребуется ID интеграции с вашей базой данных. Чтобы получить ID:

1. Перейдите в проект.
2. В меню слева выберите *Интеграции*.
3. На карточке вашей базы данных нажмите **:** и выберите *Скопировать ID интеграции*.

Пример

Код зависит от **версии движка** в вашем проекте.

Версия 2 **Версия 1**

```
// Получение данных конкретного клиента по ключу
const record = Db.get({
  dbIntegration: "1000123456-example-abc",
  documentKey: "client:ivanov"
});

// Обращаемся к данным внутри объекта
const email = record.value.email;

/* Структура record:
{
  "key": "client:ivanov",
  "value": { ... }
}
```


Хранение данных внутри диалога

Используйте сессионную базу данных, когда нужно запомнить информацию только на время текущего разговора. Данные доступны в рамках **сессии** и удаляются после завершения диалога.

❗ ПРИМЕЧАНИЕ

Вам не нужно создавать сессионную базу данных вручную. При вызове любой функции `SessionDb` будет создана *Default session db* — одна база на весь проект.

Данные в ней изолированы: информация из одного диалога не пересекается с другими.

Управлять данными можно через функции `SessionDb` внутри кода или подключив их к агенту.

Чтобы очистить данные текущего диалога и начать новую сессию, используйте функцию `Context.resetSession()`. Функция доступна, только если в проекте **движок версии 2**.

Пример

Разберем пример: дадим агенту 3 попытки, чтобы помочь пользователю. Если агент не справляется, он должен перевести диалог на поддержку.

Реализация зависит от **версии движка** в вашем проекте.

Версия 2 Версия 1

1. В начале диалога сохраняем стартовое значение счетчика:

```
// Создаем запись counters и устанавливаем лимит в 3 попытки
SessionDb.put({documentKey: "counters", value: { "notResolvedLimit": 3 }});
```

Сессионная база данных создается автоматически при вызове функции.

2. Если ответ агента не устроил пользователя, уменьшаем счетчик на единицу. Например, инструмент агента может содержать код:

```
// 1. Получаем текущие данные (весь объект)
let store = SessionDb.get({documentKey: "counters"});
```

```
// 2. Меняем значение нужного поля
// Остальные поля в store.value (если они есть) останутся нетронутыми
store.value.notResolvedLimit -= 1;

// 3. Сохраняем обновленный объект целиком
SessionDb.put({documentKey: "counters", value: store });
```

3. В логике процесса проверяем, есть ли еще попытки. Если счетчик на нуле — создаем заявку в поддержку:

```
let store = SessionDb.get({documentKey: "counters"});

if (store.value.notResolvedLimit <= 0) {
  // Лимит исчерпан, создаем заявку в поддержку
  // ...
}
```

ВКонтакте

Вы можете опубликовать процесс в любой группе, встрече или на публичной странице.

Чтобы использовать процесс во ВКонтакте:

1. **Настройте сообщества.**
2. **Подключите канал.**
3. **Протестируйте процесс.**

Настройка сообщества

Перед публикацией процесса во ВКонтакте необходимо настроить сообщество.

1. Откройте **vk.com** и перейдите в сообщество, в котором вы являетесь администратором.
2. В меню справа выберите *Управление → Сообщения*. Для параметра *Сообщения сообщества* установите значение *Включены*.
3. В том же меню выберите *Сообщения → Настройки для бота*. В параметре *Возможности ботов* выберите значение *Включены*.
4. Перейдите в раздел *Дополнительно → Работа с API*. Скопируйте существующий ключ или нажмите *Создать ключ*, отметив опцию *Разрешить приложению доступ к сообщениям сообщества*. Подтвердите действие и сохраните полученный ключ.

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить → Канал → ВКонтакте*.
- На холсте выберите *Сообщение → Подключить канал → ВКонтакте*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. Включите параметр *Сообщение начала диалога*, если он активирован в вашем сообществе.
3. В поле *Токен сообщества* выберите *Добавить новые*, чтобы создать новые учетные данные для ВКонтакте.
 - i. Задайте название учетных данных.

- ii. Укажите ключ, который вы получили при **настройке сообщества**, и нажмите *Добавить*.
- 4. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
- 5. Нажмите *Добавить*.

Теперь вы можете обмениваться сообщениями в этом канале в рамках процесса.

Тестирование процесса

1. Добавьте на холст триггер *Сообщение*.
2. Добавьте функцию `Reactions.sendText()` и подключите ее к триггеру *Сообщение*. Функция находится в разделе *Функции* → *Встроенные* → *Reactions*.
3. В настройках функции `Reactions.sendText()` в поле *text* укажите `Это тест бота ВК`.
4. Нажмите *Опубликовать*.
5. Дважды нажмите на триггер *Сообщение* и выберите *Открыть бота в ВКонтакте*. Откроется диалог с вашим ботом во ВКонтакте.
6. Напишите любое сообщение. В ответ вы получите `Это тест бота ВК`.

Чат-виджет

С помощью чат-виджета Agent Platform клиенты могут общаться с AI-агентом прямо на вашем сайте.

Чтобы настроить чат-виджет:

1. Подключите канал.
2. Получите скрипт виджета.
3. Разместите скрипт на сайте.

Подключение канала

Добавить канал чат-виджета в проекте можно двумя способами:

- В проекте перейдите в раздел *Интеграции* и выберите *Подключить* → *Канал* → *Чат-виджет*.
- На холсте в триггере *Сообщение* нажмите *Подключить канал* → *Чат-виджет*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
3. Нажмите *Добавить*.

Получение скрипта виджета

Получить скрипт чат-виджета также можно двумя способами:

- В разделе *Интеграции* наведите курсор на карточку чат-виджета и нажмите *:* → *Получить скрипт виджета*.
- На холсте в триггере *Сообщение* нажмите *Получить скрипт виджета* в карточке чат-виджета.

Скрипт будет скопирован в буфер обмена.

Размещение чат-виджета на сайте

В коде

В конструкторе сайтов

Вставьте полученный скрипт в элемент `<head>` на всех страницах вашего сайта, где вы хотите разместить виджет с вашим процессом. Например, так:

```
<head>  
  <script src="https://bot.jaicp.com/chatwidget/ae6deeca-be8y-43f0-b218-  
2b7k4d1aaaae/justwidget.js" async></script>  
</head>
```

Если вы столкнулись с трудностями, обратитесь к вашему веб-разработчику.

Чат-виджет появится на соответствующей странице.

Chat API

Chat API позволяет интегрировать чат с Agent Platform в любые приложения, например сайты и игры.

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *Chat API*.
- На холсте дважды нажмите на триггер *Сообщение* и выберите *Подключить канал* → *Chat API*.

Укажите параметры:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. Укажите **URL обратного вызова**, если будете его использовать.
3. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
4. Нажмите *Добавить*.

Токен для доступа к Chat API

После создания канала получить токен и адрес для запросов можно двумя способами:

- В разделе *Интеграции* нажмите *:* → *Получить вебхук* у канала Chat API.
- На холсте нажмите триггер *Сообщение*, затем *Получить вебхук*.

Токен — это часть адреса после `https://bot.jaicp.com/chatapi/`. Пример токена:

`ZaBcdEfG:123tf0bc456708ea48a89ee1b537211141dd0f41b`.

Методы API

💡 ПОДСКАЗКА

Подробнее о методах, параметрах запроса, форматах ответов и возвращаемых ошибках смотрите в [спецификации Chat API](#).

Отправка сообщений клиента

Запросы, которые клиент отправляет в чат, ваше приложение может передавать на сервер Agent Platform с помощью синхронных или асинхронных вызовов.

Синхронная отправка

Предоставляются следующие методы для синхронной отправки запроса клиента:

- `POST /chatapi/{token}`

Метод позволяет передавать подробную информацию о запросе, в том числе `cid`.

Оptionальное поле `cid` — идентификатор соединения. Это произвольная строка, определяющая текущее соединение с чат-приложением. Она может быть далее использована при **получении событий в чате**, чтобы фильтровать только события во время данного соединения.

- `GET /chatapi/{token}`

Упрощенный метод, который поддерживает несколько параметров запроса: `clientId`, `query` и `event`.

❗ ПРИМЕЧАНИЕ

Возможна отправка как текстового запроса, так и события в чат-приложении.

Асинхронная отправка

Метод `POST /chatapi/{token}/async` позволяет асинхронно отправлять запрос клиента или событие в чат-приложении. В ответ придет только идентификатор сообщения.

Асинхронность позволяет обрабатывать несколько запросов одновременно: чат-приложение может отправлять новые запросы, не ожидая, пока сервер закончит обрабатывать предыдущие.

Ответ процесса можно получить двумя способами:

- Чат-приложение может запрашивать **асинхронные события** с сервера Agent Platform (long polling).
- Дополнительно, если в настройках канала Chat API задан **URL обратного вызова**, Agent Platform отправляет ответ на него.

Получение асинхронных событий в чате

Метод `GET /chatapi/{token}/events` предназначен для получения асинхронных событий от сервера, например:

- Изменение **состояния** другого экземпляра чат-приложения.
- Запрос клиента, отправленный в другом экземпляре.
- Ответ процесса на запрос из другого экземпляра.

❗ ПРИМЕЧАНИЕ

Метод реализует стратегию **long polling**: если нет подходящих событий, он блокируется в ожидании следующего события.

Максимальное количество событий в ответе на запрос — 250. Если нужно обработать больше, используйте метод для **получения истории переписки**.

Получение истории переписки

Метод `GET /chatapi/{token}/client/{clientId}/history` позволяет получить историю переписки с клиентом за указанный период либо за все доступное время.

Сохранение и загрузка состояния чат-приложения

Следующие методы позволяют сохранить и загрузить состояние чат-приложения во время диалога с клиентом:

- `GET /chatapi/{token}/client/{clientId}/state`
- `POST /chatapi/{token}/client/{clientId}/state`

❗ ПРИМЕЧАНИЕ

В теле запроса к методу `POST` передается произвольный объект. Последующий запрос к методу `GET` вернет ранее переданный объект. Его содержимое не проверяется.

Управление статусами сообщений

Вы можете просматривать статусы сообщений, которые бот отправил в канале Chat API.

В Chat API поддерживаются следующие статусы сообщений:

Статус	Chat API
Отправлено	<code>SENT</code>

Статус	Chat API
Доставлено	DELIVERED
Прочитано	READ
Не отправлено	NOT_SENT

Получение статуса сообщения

Метод `GET /chatapi/{token}/client/{clientId}/message/{questionId}/status` позволяет получить статус сообщения.

Параметры:

- `token` — токен канала Chat API.
- `clientId` — идентификатор клиента. Для получения используйте функцию `Context.getClientInfo().id`.
- `questionId` — идентификатор сообщения. Чтобы получить идентификатор, воспользуйтесь методом `GET /chatapi/{token}/client/{clientId}/history`.

Обновление статуса сообщения

Метод `POST /chatapi/{token}/client/{clientId}/message/{questionId}/status` позволяет изменить статус сообщения. В теле запроса укажите новое значение для поля `status`.

Обновление статусов сообщений

Метод `POST /chatapi/{token}/client/{clientId}/message-statuses` позволяет изменить статусы нескольких сообщений. В теле запроса укажите поле `statuses` — массив JSON-объектов.

Получение количества непрочитанных сообщений

Метод `GET /chatapi/{token}/client/{clientId}/message-not-read-count` позволяет узнать, сколько сообщений не прочитал клиент.

URL обратного вызова

Чтобы указать URL для обратного вызова:

1. Для канала в разделе *Интеграции* или в триггере *Сообщение* нажмите `:` → *Редактировать*.

2. Введите адрес в поле *URL обратного вызова*. Нажмите *Сохранить*.

Agent Platform отправляет событие на URL обратного вызова в следующих случаях:

- Подготовлен ответ на асинхронный запрос.
- Вы провели рассылку.

❗ ПРИМЕЧАНИЕ

Если запрос на URL обратного вызова не был успешно выполнен, он будет отправлен повторно еще **три** раза. Время ожидания между ответами при асинхронных запросах составляет **три** секунды.

На URL обратного вызова придет массив JSON-объектов:

```
{
  "token": "token",
  "clientId": "test",
  "questionId": "12345",
  "data": {
    "nlpClass": "/CatchAll",
    "confidence": -0.010999999999999979,
    "replies": [ // Ответ клиенту
      {
        "type": "text",
        "text": "Скажите что-то осмысленное",
        "state": "/CatchAll"
      }
    ],
    "answer": "Скажите что-то осмысленное",
    "newSessionStarted": false,
    "debugData": [],
    "endSession": false
  },
  "timestamp": "2022-09-28T12:32:13.262",
  "blockForm": false
}
```

Как подключить edna Chat Center

edna Chat Center — платформа для ведения продаж и клиентской поддержки через цифровые каналы. Позволяет подключать чат-ботов, настраивать маршрутизацию обращений и использовать их в веб-интерфейсе и мессенджерах.

Чтобы опубликовать процесс в edna Chat Center:

1. Подключите канал edna Chat Center в проект.
2. Настройте чат-бота в аккаунте edna Chat Center.
3. Завершите настройку подключения в Agent Platform.
4. Протестируйте процесс.

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *edna Chat Center*.
- На холсте выберите *Сообщение* → *Подключить канал* → *edna Chat Center*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. Укажите *URL клиента* — это URL вида `https://YourAddress.edna.io`.
3. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
4. Оставьте поле *Токен доступа* пустым. Его вы заполните после настройки **чат-бота в edna Chat Center**.
5. Нажмите *Добавить*.
6. После создания канала перейдите в раздел *Интеграции*. Наведите курсор на карточку интеграции и нажмите *⋮* → *Получить вебхук*.

Настройка чат-бота в edna Chat Center

1. Войдите в ваш аккаунт edna Chat Center с правами администратора.
2. Перейдите в меню *Интеграции* → *Чат-бот*.
3. Нажмите **+**, чтобы создать нового бота.
4. Укажите:
 - *Имя чат-бота* — так бот будет называться в edna.
 - Вставьте скопированный адрес вебхука во все три поля:
 - *URL бекенда чат-бота для обработки сообщений клиента.*
 - *URL бекенда чат-бота для инициации диалога с клиентом.*
 - *URL бекенда чат-бота для обработки закрытия веб-формы в чате* (необязательно).
 - *Таймаут ожидания ответа* — укажите при необходимости. Если таймаут не указан, применятся настройки по умолчанию.
5. Нажмите *Сохранить*.
6. Скопируйте токен, нажав *Token* напротив бота. Он будет показан только один раз — сохраните его.
7. Перейдите в *Маршрутизация* → *Маршруты* и нажмите **+**:
 - Укажите название маршрута.
 - Выберите нужный сегмент (или создайте новый).
 - В типе маршрута выберите *Чат-бот*, затем укажите только что созданного бота.
 - Нажмите *Сохранить*.

Завершение настройки в Agent Platform

1. Вернитесь в раздел *Интеграции* в Agent Platform.
2. Наведите курсор на карточку интеграции с edna Chat Center. Нажмите **:** → *Редактировать*.
3. В поле *Токен доступа* выберите *Добавить новые*, чтобы создать новые учетные данные.
4. Задайте название учетных данных.
5. *Токен* — вставьте скопированный токен. Если в поле уже есть значение, замените его. Нажмите *Добавить*.
6. Сохраните изменения.

Тестирование процесса

1. Опубликуйте процесс в канал edna.
2. Перейдите в окно диалога с ботом в edna Chat Center и напишите ему.

После этого должен начаться ваш диалог. Бот должен ответить так, как вы настроили в процессе Agent Platform.

Mattermost

Mattermost — мессенджер, который используется в качестве внутреннего чата в организациях и компаниях. Он позволяет создавать группы и каналы, обмениваться файлами и настраивать различные интеграции.

Чтобы запустить ваш процесс в Mattermost:

1. **Создайте бота в Mattermost.**
2. **Подключите канал.**
3. **Протестируйте процесс.**

Создание бота в Mattermost

1. **Создайте аккаунт бота в Mattermost.**
2. Во время создания аккаунта вы получите токен бота. Сохраните токен, он нужен для подключения бота к Agent Platform.
3. **Добавьте бота в вашу команду.**

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *Mattermost*.
- На холсте выберите *Сообщение* → *Подключить канал* → *Mattermost*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. Укажите адрес сервера, на котором установлен Mattermost. Например:
`https://mattermost.example.com`.
3. В поле *Токен бота* выберите *Добавить новые*, чтобы создать новые учетные данные.
 - i. Задайте название учетных данных.
 - ii. *Токен* — укажите токен, который вы получили при **создании бота**, и нажмите *Добавить*.
4. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.

5. Нажмите *Добавить*.

Теперь вы можете получать и отправлять сообщения в этом канале.

Тестирование процесса

1. Добавьте бота Mattermost в канал или группу. Также вы можете общаться с агентом в личных сообщениях.
2. Напишите боту. После этого должен начаться ваш диалог с агентом.

MAX

MAX — это приложение для обмена сообщениями, которое работает на смартфонах и компьютерах. В него также встроена платформа мини-приложений, конструкторы чат-ботов и платежная система.

Через Agent Platform вы можете опубликовать процесс в виде бота для мессенджера MAX.

Чтобы использовать ваш процесс в MAX:

1. **Создайте бота и получите токен.**
2. **Подключите канал.**
3. **Протестируйте процесс.**

Создание бота и получение токена

Прежде чем опубликовать процесс в MAX, нужно создать бота на стороне MAX и получить его токен.

1. Зарегистрируйтесь в **MAX для партнеров**. Переходите именно по этой ссылке — она дает клиентам Just AI доступ к верификации.
2. **Создайте и верифицируйте профиль организации.**
3. **Создайте чат-бота и пройдите модерацию.**
4. Когда бот пройдет модерацию, получите токен — он понадобится при **подключении канала**:
 - i. Если у вас несколько ботов, в левом верхнем углу выберите нужный.
 - ii. На панели *Интеграция* нажмите *Получить токен*.
 - iii. Скопируйте токен.

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *MAX*.
- На холсте выберите *Сообщение* → *Подключить канал* → *MAX*.

Укажите настройки:

1. В поле *Название канала* задайте название. Оно будет отображаться в списке интеграций.

2. В поле *Токен бота* выберите учетные данные с токеном бота MAX или создайте новые:
 - i. Выберите *Добавить новые*.
 - ii. В поле *Название* задайте название учетных данных.
 - iii. В поле *Токен* вставьте токен, который вы получили при **создании бота**.
 - iv. В поле *Действует до* укажите дату окончания действия токена, если она есть.
 - v. Нажмите *Добавить*.
3. Включите *Автоматическую публикацию*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
4. Нажмите *Добавить*.

Теперь вы можете получать и отправлять сообщения в этом канале в рамках своего процесса.

Тестирование процесса

Чтобы перейти к боту из Agent Platform, используйте один из способов:

- Через триггер на холсте:
 - i. На холсте дважды нажмите на триггер *Сообщение*. Если триггера еще нет на холсте, **добавьте** его.
 - ii. Нажмите *Открыть бота в MAX*  у нужного канала.
- Через список интеграций:
 - i. В проекте перейдите в раздел *Интеграции*.
 - ii. В списке каналов найдите нужный канал.
 - iii. Нажмите **:** →  *Открыть бота в MAX*.

Вы также можете найти вашего бота в списке контактов или каналов в мессенджере MAX.

Отправьте боту активационную фразу, предусмотренную вашим процессом. После этого начнется диалог с ботом.

Prompter API

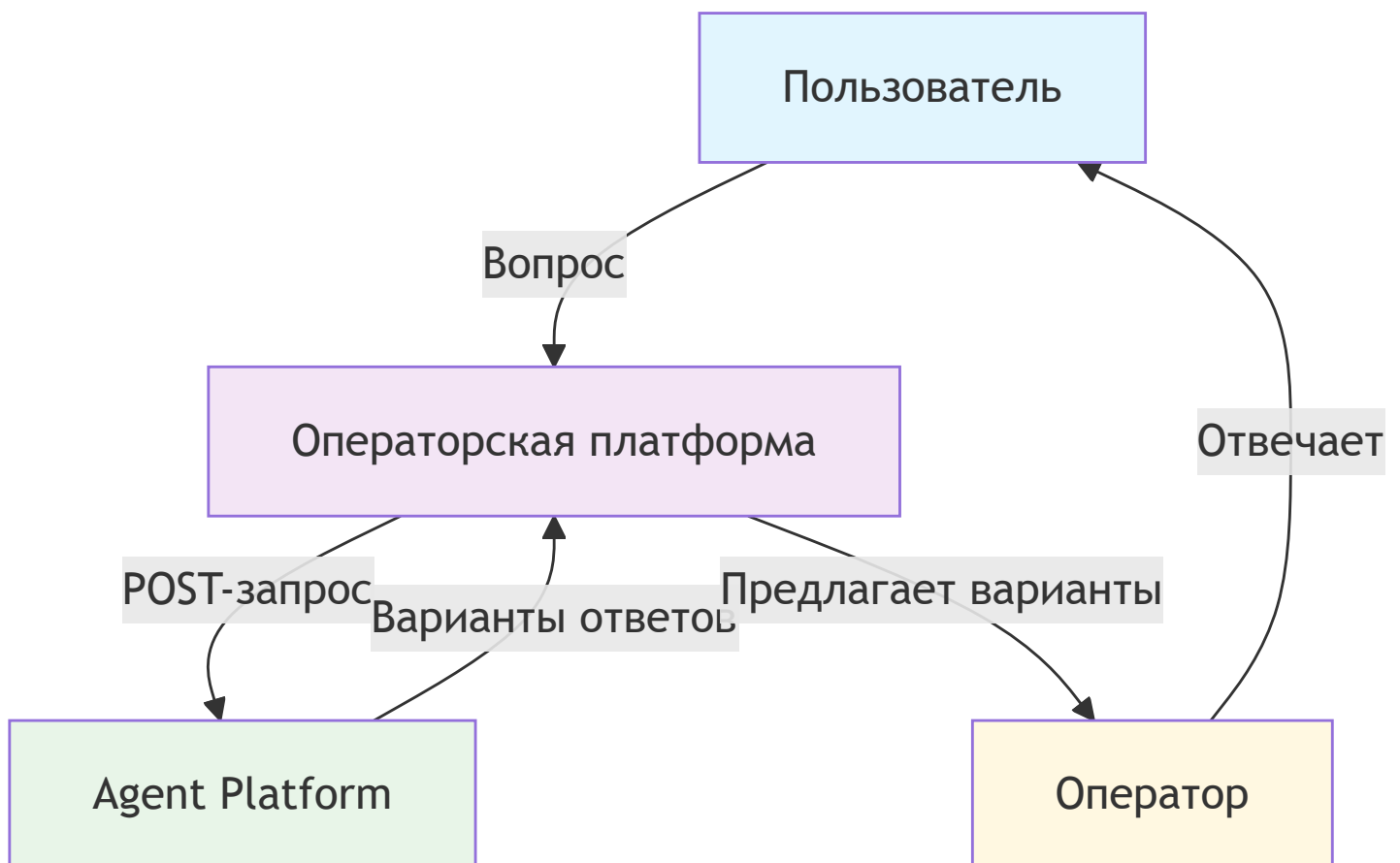
Суфлер — помощник, который анализирует вопросы клиентов и подсказывает операторам наиболее подходящие варианты ответов. Он не общается с клиентом напрямую, а помогает оператору быстрее реагировать на запросы.

Prompter API позволяет:

- Подключить суфлеров в [Aimychat](#) для автоматической поддержки операторов.
- Интегрировать суфлеров в сторонние сервисы через API.

Как работает интеграция

Когда пользователь отправляет вопрос в операторскую платформу, система передает этот запрос через вебхук Prompter API в Agent Platform. Процесс в Agent Platform анализирует вопрос, формирует подходящие варианты ответов и возвращает их обратно в операторскую платформу. Оператор получает готовые варианты ответов и может выбрать наиболее подходящий для отправки пользователю.



Подключение Prompter API

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *Prompter API*.
- На холсте выберите *Сообщение* → *Подключить канал* → *Prompter API*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
3. Нажмите *Подключить*.

После создания канала получить токен и адрес вебхука можно двумя способами:

- Нажмите *⋮* → *Получить вебхук* у канала на вкладке *Интеграции*.
- Нажмите *Получить вебхук* в триггере *Сообщение* на холсте. Токен — это часть URL вебхука после `https://bot.jaicp.com/chatapi/webhook/prompter_api/`

Интеграция в сторонние сервисы

Вы можете подключить любой сторонний сервис. Он должен использовать метод `POST` `/chatapi/webhook/prompter_api/{token}`. Этот метод отправляет в Agent Platform вопрос клиента и получает подходящие варианты ответа.

Параметры запроса

Параметр пути:

- `token` — токен канала Prompter API, который можно получить после **подключения канала**.

Параметры тела запроса:

- `clientInfo` — объект с информацией о клиенте.
 - `id` — уникальный идентификатор клиента в вашей системе.
 - `firstName` и `lastName` — имя и фамилия клиента (необязательные параметры).
- `chatId` — уникальный идентификатор диалога в вашей системе.
- `text` — текст вопроса клиента.

Ответ на запрос

В ответ на запрос платформа возвращает JSON-объект с вариантами ответов:

```
{
  "clientInfo": {
    "id": "testClientId"
  },
  "chatId": "testChatId",
  "questionId": "questionId",
  "answerOptions": [
    {
      "messages": [
        {
          "type": "TEXT",
          "text": "Ответ на вопрос клиента",
          "markup": "plain"
        }
      ]
    }
  ]
}
```

💡 ПОДСКАЗКА

Подробнее о методе, параметрах запроса и формате ответа смотрите в [спецификации Prompter API](#).

Aimychat

Для подключения суфлера в **Aimychat** можно воспользоваться вебхуком:

1. Перейдите в раздел *Настройки* → вкладка *Суфлеры*.
2. Добавьте нового суфлера и укажите адрес вебхука Prompter API.
3. На вкладке *Группы* выберите нужную группу или создайте новую. В окне редактирования группы перейдите на вкладку *Суфлеры*, выберите ваш суфлер и нажмите *Добавить*.

Вебхук представляет собой эндпоинт на стороне Agent Platform, который принимает POST-запросы от Aimychat с вопросами клиентов.

Telegram

Через Agent Platform вы можете опубликовать процесс в виде:

- обычного Telegram-бота;
- Telegram-бота для **бизнес-аккаунтов**.

Чтобы использовать ваш процесс в Telegram:

1. **Создайте бота.**
2. **Подключите канал.**
3. **Протестируйте процесс.**

Создание бота

Прежде чем опубликовать процесс в Telegram, нужно создать бота на стороне Telegram.

1. Откройте Telegram и в поиске контактов введите `BotFather`.
2. Отправьте команду `/newbot` в чат с BotFather.
3. Придумайте имя бота и отправьте его в чат. Имя будет отображаться в контактах и чатах.
4. Придумайте `username` для бота и отправьте его в чат.
 - `username` — короткое имя, которое используется для упоминаний бота и в ссылках на профиль.
 - Может состоять из букв латинского алфавита, подчеркиваний и цифр.
 - Должно заканчиваться на `bot`, например: `test_bot` или `TestBot`.
 - Может быть от 5 до 32 символов.
5. BotFather отправит ссылку на вашего бота и токен. Скопируйте токен, он потребуется при **подключении канала**. Токен представляет собой набор символов вида `123456789:AAHdqTcvCH1vGWJxfSeofSAs0K5PALDsaw`.
6. Если вы хотите подключить процесс к бизнес-аккаунту:
 - i. Отправьте команду `/mybots` в чат с BotFather.
 - ii. Выберите бота в списке.
 - iii. Перейдите в *Bot Settings* → *Business Mode*.
 - iv. Нажмите *Turn on*.
 - v. Добавьте бота в **бизнес-аккаунт**.

Если вы потеряли токен, отправьте команду `/token` в чат с BotFather. BotFather создаст новый токен.

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *Telegram*.
- На холсте выберите *Сообщение* → *Подключить канал* → *Telegram*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. В поле *Токен бота* выберите *Добавить новые*, чтобы создать новые учетные данные для Telegram.
 - i. Задайте название учетных данных.
 - ii. *Токен* — укажите токен, который вы получили при *создании бота*, и нажмите *Добавить*.
3. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
4. Нажмите *Добавить*.

Теперь вы можете получать и отправлять сообщения в этом канале в рамках своего процесса.

Тестирование процесса

Перейдите в окно диалога с ботом в Telegram и напишите ему. После этого должен начаться ваш диалог с ботом согласно вашему процессу.

WhatsApp

❗ ПРИМЕЧАНИЕ

Этот вид подключения может работать нестабильно в связи с замедлением WhatsApp в России.

Через Agent Platform вы можете опубликовать процесс в WhatsApp Business. Чтобы использовать ваш процесс в WhatsApp:

1. **Создайте приложение в аккаунте разработчика.**
2. **Настройте приложение.**
3. **Подключите к каналу.**
4. **Настройте вебхук в приложении.**
5. **Протестируйте в мессенджере.**

Создание приложения

❗ ПРИМЕЧАНИЕ

Чтобы настроить интеграцию, вам понадобится аккаунт разработчика.

1. Войдите в **аккаунт разработчика**.
2. Откройте **Панель приложений**.
3. Нажмите *Создать приложение*.
4. Заполните информацию о приложении:
 - *Название приложения*;
 - *Контактный электронный адрес приложения для связи*;
5. На экране выбора сценария использования выберите *Бизнес-переписка* и *Общайтесь с клиентами в WhatsApp*. Нажмите *Далее*.
6. Выберите существующее бизнес-портфолио или выполните подсказки, чтобы создать новое.
7. Завершите создание приложения.

Настройка приложения

Для стабильной работы бота необходимо получить *ID номера телефона* и сгенерировать *постоянный маркер* (токен). Временный маркер, который выдается на странице *Настройка API*, действует всего 24 часа.

Генерация постоянного маркера

1. Перейдите на страницу [Настройки компании](#).
2. В меню слева выберите *Системные пользователи*.
3. Нажмите *Добавить* в правом верхнем углу и следуйте инструкциям, чтобы создать нового системного пользователя. Если у вас уже есть системный пользователь, можно использовать его.
4. Нажмите *Назначить объекты*.
5. В открывшемся окне:
 - Выберите свое приложение и активируйте переключатель *Управление приложением* в разделе *Полный доступ*.
 - Выберите свой аккаунт WhatsApp и активируйте переключатель *Управление аккаунтами WhatsApp Business* в разделе *Полный доступ*.
 - Нажмите *Назначить объекты*.
6. Вернувшись в список системных пользователей, нажмите *Сгенерировать маркер*.
7. Выберите ваше приложение из списка.
8. Отметьте галочками разрешения:
 - `business_management`
 - `whatsapp_business_messaging`
 - `whatsapp_business_management`
9. Скопируйте маркер и сохраните его для использования на последующих этапах.

WARNING

Сохраните маркер в надежном месте (например, в менеджере паролей). Facebook покажет его только один раз. Если вы закроете окно, токен придется генерировать заново.

Получение ID номера телефона

1. Вернитесь в раздел [Мои приложения](#) и выберите ваше приложение.
2. В меню слева разверните раздел *WhatsApp* и выберите *Настройка API*.
3. В блоке *Отправка и получение сообщений*:

- В поле *От* выберите нужный номер телефона. Если у вас еще нет номера, в меню слева выберите *WhatsApp* → *Конфигурация*, добавьте номер и подтвердите его через SMS. Затем вернитесь в раздел *Настройка API*
- Скопируйте ID, который отобразится под номером.

❗ ПРИМЕЧАНИЕ

ID номера телефона понадобятся вам при подключении канала в Agent Platform.

Подключение канала

Добавить новый канал в проекте можно несколькими способами:

- В проекте перейдите в раздел *Интеграции*, выберите *Подключить* → *Канал* → *WhatsApp*.
- На холсте выберите *Сообщение* → *Подключить канал* → *WhatsApp*.

Укажите настройки:

1. Задайте название. Оно будет отображаться в списке интеграций.
2. *ID номера телефона* — вставьте идентификатор, который вы скопировали ранее.
3. В поле *Учетные данные Business API* выберите *Добавить новые*, чтобы создать новые учетные данные для WhatsApp.
 - i. Задайте название учетных данных.
 - ii. *Токен* — вставьте *Постоянный маркер*, который вы скопировали ранее, и нажмите *Добавить*.
4. В *Учетные данные Business API* выберите учетные данные, которые только что создали.
5. Включите *Автоматический деплой*, чтобы изменения автоматически публиковались в канале при нажатии кнопки *Опубликовать* на холсте. В противном случае их нужно публиковать вручную кнопкой *Опубликовать* в триггере *Сообщение*.
6. Нажмите *Добавить*.

После создания канала в списке интеграций появится кнопка *Получить вебхук*. Нажмите на нее и скопируйте данные — они понадобятся на следующем шаге.

Настройка вебхука

Чтобы завершить настройку интеграции в приложении:

1. Выберите ваше приложение в **аккаунте разработчика**.

2. В меню слева разверните раздел *WhatsApp* и выберите *Настройка*.
3. В разделе *Webhook* нажмите *Редактировать*.
4. Заполните данные в окне *Редактировать URL обратного вызова webhook*:
 - В поле *URL обратного вызова* вставьте вебхук, скопированный из Agent Platform.
 - В поле *Подтверждение маркера* укажите `justai_verify_token`.
5. Нажмите *Подтвердить и сохранить*.
6. В этом же разделе в блоке *Поля Webhook* нажмите *Настроить*.
7. Найдите в списке поле `messages` и нажмите *Оформить подписку*.
8. Нажмите *Готово*.

Тестирование бота

Настройка интеграции завершена. Чтобы протестировать бота, отправьте сообщение в бизнес-аккаунт WhatsApp.

SIP-подключение

Используйте SIP-подключение, чтобы принимать в Agent Platform звонки, которые поступают на ваш SIP-транк. Подробнее про телефонию читайте в статье [Как настроить звонки](#).

Основные настройки

- *Учетные данные* — **создайте** или выберите учетные данные типа *SIP-аккаунт*.
- *Имя хоста или IP-адрес* — укажите адрес SIP-сервера вашего провайдера.
- *Порт* — укажите порт SIP-сервера.
- *Принимать входящие звонки* — включите, чтобы по этому подключению могли поступать входящие вызовы.

Расширенные настройки

- *Авторизация по username* — заполните, если username для авторизации отличается от логина в **учетных данных типа SIP-аккаунт**.
- *Включить регистрацию* — включите, если ваш провайдер требует регистрацию для приема входящих звонков.
- *Сопоставление по IP и порту* — включите опцию, если в одном подключении нужно обрабатывать входящие вызовы на несколько DNID (dialed number identifier, идентификатор набранного номера).
- *Игнорировать порт* — включите для авторизации только по IP, без учета порта.
- *Отправлять локальный домен в заголовке FROM* — включите опцию, чтобы в SIP-заголовке `From` передавался адрес локального домена, а не SIP URI.
- *Проверять соединение* — включите опцию и заполните *Период проверки в секундах*, если нужна проверка подключения.
- *Таймаут регистрации* — укажите время в секундах, через которое Agent Platform должна повторно регистрироваться на SIP-сервере. Значение используется в заголовке `Expires` у SIP-запроса `REGISTER`.

ПРИМЕЧАНИЕ

Параметры *Таймаут регистрации* и *Проверять соединение* не зависят друг от друга.

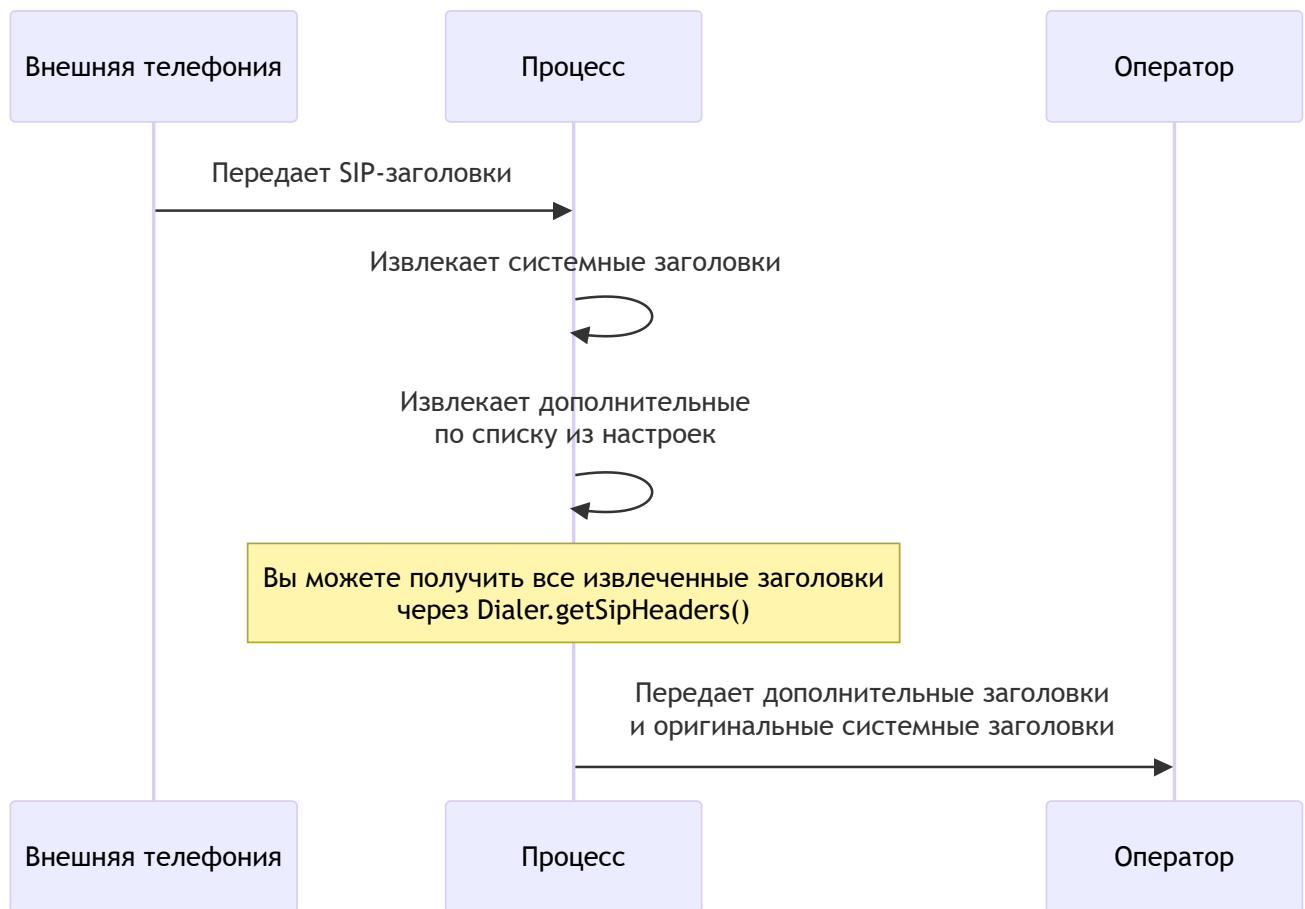
- *Кодеки* — выберите кодеки, которые будут использоваться для передачи аудио во время звонков.

SIP-заголовки

Укажите на этой вкладке дополнительные заголовки, чтобы извлекать их из сообщения **INVITE** при входящем звонке. Названия заголовков чувствительны к регистру.

Смотрите статью [SIP-заголовки](#), чтобы узнать подробнее о работе с системными и дополнительными заголовками.

SIP-заголовки



Как работать с SIP-заголовками в процессе:

- При входящем звонке процесс извлекает SIP-заголовки из сообщения `INVITE`:
 - Системные заголовки: `call-id`, `cseq`, `contact`, `from`, `max-forwards`, `refer-to`, `to`, `via`.
 - Дополнительные заголовки. Укажите их на вкладке *SIP-заголовки* в *настройках SIP-подключения*.
- В процессе вы можете получить извлеченные заголовки с помощью `Dialer.getSipHeaders()`:

```
{
  "cseq": "12345 INVITE",
  "contact": "<sip:userA@hostA>",
  "call-id": "abc123",
  "max-forwards": "70",
  "from": "<sip:userA@domainA>;tag=tagA",
  "to": "<sip:userB@hostB>",
  "via": "SIP/2.0/UDP hostB:5060;branch=z9hG4bKbranchA",
  "X-Example": "Example" // Дополнительный заголовок
}
```

- Системные заголовки всегда передаются в нижнем регистре.

- Дополнительные будут в том же регистре, в котором вы их указали на вкладке *SIP-заголовки* в **настройках SIP-подключения**.

При **перевode звонка на оператора**:

- Платформа передает оператору все извлеченные заголовки, как системные, так и дополнительные.
- В параметре `headers` функции `Dialer.transferCall()` вы можете передать новые или переопределить существующие заголовки.

⚠ ПРЕДУПРЕЖДЕНИЕ

- Системные заголовки нельзя переопределить через `headers` — платформа проигнорирует ваши изменения.
- Платформа автоматически передает все извлеченные дополнительные заголовки, даже если вы не указали их в `headers`.

Учетные данные для провайдеров ASR/TTS

Эти учетные данные нужны для подключения [моделей ASR и голосов TTS](#).

ASR и TTS используются:

- Во время звонков. Подробнее о телефонии читайте в статье [Как настроить звонки](#).
- В текстовых каналах [для распознавания или генерации аудиофайлов](#).

3i VoxKit

Токен — вставьте [токен доступа к API 3i VoxKit](#).

Audiogram

Получите доступ к сервису на сайте [Audiogram](#). В Agent Platform нужны следующие данные:

- *IAM account*
- *IAM workspace*
- *Client ID*
- *Client secret*

Azure

- *Ключ ресурса* — вставьте ключ ресурса *Речь (Speech)*.
- *Регион ресурса* — выберите регион, в котором создан ресурс.

Вы можете посмотреть ключ и регион ресурса на [портале Azure](#).

Google Cloud

Ключ в формате JSON — вставьте содержимое файла с [ключом сервисного аккаунта Google Cloud](#).

Пример:

```
{
  "type": "service_account",
  "project_id": "example-project-123456",
  "private_key_id": "1abc23...",
  "private_key": "-----BEGIN PRIVATE KEY-----\nABCDef...\n-----END PRIVATE KEY-----\n",
  "client_email": "user@example.com",
  "client_id": "12345...",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url":
    "https://www.googleapis.com/robot/v1/metadata/x509/user%40example.com",
  "universe_domain": "googleapis.com"
}
```

ElevenLabs

Ключ API — вставьте **API-ключ**, полученный в личном кабинете ElevenLabs.

❗ ПРИМЕЧАНИЕ

Сайт ElevenLabs недоступен для российских IP-адресов.

Права доступа для ключа API

Для работы интеграции настройте права доступа в ElevenLabs. В разделе *Endpoints*:

1. Для *Text to Speech* выберите *Access*. В интерфейсе появится настройка *Voices*.
2. Для *Voices* выберите *Read*.

SaluteSpeech

- *Client ID*.
- *Client Secret*.
- *Scope* — выберите версию API:
 - *SALUTE_SPEECH_PERS* — для физических лиц.
 - *SALUTE_SPEECH_CORP* — для юридических лиц, у которых проект SaluteSpeech создан на условиях постоплаты.
 - *SALUTE_SPEECH_B2B* — для юридических лиц, у которых проект SaluteSpeech создан на условиях предоплаты.

- *SBER_SPEECH* — устаревшее значение для юридических лиц.

Все эти данные вы можете получить в [личном кабинете SaluteSpeech](#).

T-Bank VoiceKit

Ключ API и Секретный ключ — вставьте значения `API_KEY` и `SECRET_KEY` из [личного кабинета T-Bank VoiceKit](#).

Yandex SpeechKit

Чтобы подключить Yandex SpeechKit, создайте [сервисный аккаунт](#) и [авторизованный ключ](#).

- *Закрытый ключ сервисного аккаунта* — вставьте закрытую часть авторизованного ключа в формате:

```
-----BEGIN PRIVATE KEY-----EXAMPLE1234567G9w0BAQEFAASCBKgwggS
cmQxJjAkBgNVBAoTHVByb2dyZXNzIFNvZnR3YXJlIENvcnBvcnF0aW9uMSAwHgYD
VQQDDCqLmF3cy10ZXN0LnByb2dyZXNzLmNvbTCCASIwDQYJKoZIhvcNAQEBBQAD
...
EXAMPLE1234567YWxzaGEyZzIuY3JsMIGgBggrBgEFBQcBAQSBkzCBkDBNBggrBgEF
BQcwAoZBaHR0cDovL3N1Y3VyZS5nbG9iYWxzaWduLmNvbS9jYWNLcnQvZ3Nvcmdh
z3P668YfhUbKdRF6S42Cg6zn-----END PRIVATE KEY-----
```

- *ID сервисного аккаунта* — вставьте ID сервисного аккаунта. Его можно получить в списке [сервисных аккаунтов](#).
- *ID открытого ключа* — вставьте ID открытого ключа. Он отображается напротив ранее созданного авторизованного ключа.
- *ID каталога* — вставьте ID каталога, в котором создан сервисный аккаунт.

Модели ASR и голоса TTS

Эти интеграции нужны для распознавания и синтеза речи:

- Во время звонков. Подробнее о телефонии читайте в статье [Как настроить звонки](#).
- В текстовых каналах [для распознавания или генерации аудиофайлов](#).

Учетные данные для этих интеграций описаны в статье [Учетные данные для ASR/TTS](#).

Модели ASR

3i VoxKit

Модель — идентификатор модели для распознавания речи.

Audiogram

Сервис распознает речь только на русском языке. У интеграции нет дополнительных настроек.

Azure

Язык — введите код языка, на котором говорят ваши клиенты. Например, `ru-RU` для русского языка. Полный список языков смотрите в документации [Azure](#).

Google Cloud

- *Язык* — выберите язык, на котором будут говорить ваши клиенты.
- *Модель* — название модели для распознавания речи.

В документации Google Cloud вы можете посмотреть список поддерживаемых [языков](#) и [моделей](#).

SaluteSpeech

Сервис распознает речь только на русском языке. У интеграции нет дополнительных настроек.

T-Bank VoiceKit

Сервис распознает речь только на русском языке. У интеграции нет дополнительных настроек.

Yandex SpeechKit

- *Язык* — код языка, на котором говорят ваши клиенты.
- *Модель* — **версия модели** для распознавания речи.
- *Числа прописью* — если опция включена, то в распознанном тексте числа записываются словами. Например, `тринадцать` вместо `13`.
- *Подавление шума* — опция уменьшает чувствительность модели к фоновым шумам.

Голоса TTS

3i VoxKit

Голос — выберите голос для синтеза речи.

Audiogram

- *Модель* — модель для синтеза речи. Сейчас доступна только модель *high_quality*.
- *Голос* — голос для синтеза речи:
 - Женские голоса:
 - *borisova*
 - *kishchik*
 - Мужские голоса:
 - *gandzhaev*
 - *gavrilov*
- *Эмоция* — эмоциональная окраска голоса:
 - *neutral* — нейтральная
 - *happy* — радостная

Azure

Голос — введите идентификатор голоса. Полный список голосов смотрите в документации [Azure](#). Agent Platform поддерживает только нейронные голоса (*Neural*).

ElevenLabs

- *Модель*:
 - `eleven_turbo_v2_5` — модель с хорошим балансом между качеством речи и скоростью генерации.

- `eleven_flash_v2_5` — быстрая модель с минимальной задержкой.
- `eleven_multilingual_v2` — модель с высоким качеством синтеза, но имеет более высокую задержку и цену.
- *Сходство с оригиналом* — определяет, насколько точно ИИ должен имитировать оригинальный голос.
- *Скорость* — чем выше значение, тем быстрее будет речь.
- *Стабильность* — определяет стабильность голоса и вариативность синтеза. При низких значениях голос звучит эмоционально, при высоких — монотонно.
- *Экспрессия* — подчеркивает уникальные черты голоса и делает голос более выразительным. Значения выше 0 требуют больше ресурсов и могут увеличить задержку.
- *Голос* — идентификатор голоса для синтеза речи. Вы можете получить идентификатор (Voice ID) в личном кабинете ElevenLabs. Пример: `12a34Bcd5EfgHi6jkLMN`.

❗ ПРИМЕЧАНИЕ

Сходство с оригиналом, *Стабильность* и *Экспрессия* соответствуют параметрам *Similarity*, *Stability* и *Style Exaggeration* в ElevenLabs.

Языки

- ElevenLabs поддерживает **множество языков**.

❗ ПРИМЕЧАНИЕ

Сайт ElevenLabs недоступен для российских IP-адресов.

- Провайдер сам определяет язык синтеза на основе полученного текста. Вам не нужно указывать язык вручную.
- Некоторые голоса могут иметь акценты.

Google Cloud

- *Высота голоса* — например, значение `20` повысит тон голоса на `20` полутонов.
- *Скорость* — чем выше значение, тем быстрее будет речь.
- *Голос* — название голоса для синтеза речи.
- *Громкость* — измеряется в децибелах (dB). Не рекомендуется устанавливать значение выше `10`.

В документации Google Cloud вы можете посмотреть список поддерживаемых **голосов**.

SaluteSpeech

Голос — введите код голоса для синтеза речи, например `Nec_8000`. Agent Platform поддерживает только голоса с частотой 8 кГц — коды таких голосов заканчиваются на `_8000`.

Смотрите список голосов в документации [SaluteSpeech](#).

Yandex SpeechKit

- *Язык* — код языка, на котором будет синтезироваться речь.
- *Манера речи* — соответствует **амплуа** из документации Yandex SpeechKit. Это характеристика звучания голоса — например, диктор может говорить более дружелюбно или шепотом.
- *Скорость* — чем выше значение, тем быстрее будет речь. `1` — нормальная скорость голоса.
- *Голос* — название голоса для синтеза речи. Agent Platform поддерживает голоса v3.
- *Громкость* — громкость относительно цифровой полной шкалы **LKFS/LUFS**. Рекомендуемый диапазон значений: от `-20` до `-16`.

❗ К СВЕДЕНИЮ

Список доступных голосов v3, языков и амплуа смотрите в документации [Yandex SpeechKit](#).

Сравнение скорости ASR и TTS в телефонии

Задержка ответа — ключевой параметр в голосовых каналах. Чтобы вы могли выбрать самые быстрые сервисы для своего проекта, мы замерили скорость работы ASR и TTS в агентах.

❗ ON-PREMISE

Если Agent Platform установлена в контур вашей компании, то на скорость работы также могут влиять:

- расположение серверов;
- настройки VAD (Voice Activity Detection) — они указываются в конфигурации вашей установки.

ASR

Время от начала распознавания до получения первого фрагмента текста:

Провайдер	Среднее время (мс)
Google Cloud	485
Yandex SpeechKit	910
T-Bank VoiceKit	1050
SaluteSpeech (Сбер)	1640
3i VoxKit	1910

TTS

Время от начала синтеза до получения аудио:

Провайдер	Голос	Среднее время (мс)
Google Cloud	ru-RU-Wavenet-A	275
Audiogram	borisova	320
Yandex SpeechKit	alena (neutral)	355
ElevenLabs	Голоса модели Turbo v2.5	370
Azure	ru-RU-SvetlanaNeural	730
SaluteSpeech (Сбер)	Nec_8000	1015

ASR и TTS в текстовых каналах

Вы можете использовать ASR и TTS не только в звонках, но и в текстовых каналах. В Agent Platform доступны функции:

- `Asr.recognize` — получает файл по URL и распознает речь в нем.
С помощью этой функции вы можете, например, распознать речь в записи совещания. Полученный текст можно передать в агента и попросить составить краткое содержание встречи.
- `Tts.synthesize` — синтезирует речь из текста и возвращает URL, по которому можно скачать аудиофайл.
Например, вы можете попросить агента подготовить текст для подкаста и далее озвучить его с помощью этой функции.

ASR

Как настроить интеграцию

Функция `Asr.recognize` использует интеграцию типа *Модель ASR*.

Чтобы создать такую интеграцию:

1. Добавьте учетные данные для провайдера ASR/TTS:
 - i. В проекте перейдите в раздел *Учетные данные*.
 - ii. Добавьте новые учетные данные с типом *Провайдер ASR/TTS*.
Подробнее о нужных данных читайте в статье [Учетные данные для ASR/TTS](#).
2. Перейдите в раздел *Интеграции*.
3. Создайте интеграцию с типом *Модель ASR*.
Подробнее о настройках для интеграции читайте в статье [Модели ASR и голоса TTS](#).

Вызов функции

Функция `Asr.recognize` принимает следующие параметры:

- `asrIntegrationKey` — идентификатор интеграции *Модель ASR*. В разделе *Интеграции* вы можете скопировать этот идентификатор на карточке интеграции. Пример: `1000001-name-abC`.

- `audioUrl` — URL аудиофайла, в котором нужно распознать речь. Файл должен быть доступен из интернета без авторизации.

Функция возвращает объект с результатами распознавания. Пример:

```
{
  "metadata":{
    "initiatedAt":1763637249747,
    "requestId":"1abc3def-1a23-4567-8901-164aaa3d6662"
  },
  "settings":{
    "type":"YANDEX",
    "model":"general",
    "languageCode":"ru-RU",
    "numbersAsWords":true,
    "sensitivityReduction":true
  },
  "result":{
    "text":"хочу отменить заказ"
  }
}
```

Здесь:

- `metadata` — метаданные запроса.
- `settings` — настройки интеграции ASR, которые использовались для распознавания.
- `result.text` — результат распознавания.

TTS

Как настроить интеграцию

Функция `Tts.synthesize` использует интеграцию типа *Голос TTS*.

Чтобы создать такую интеграцию:

1. Добавьте учетные данные для провайдера ASR/TTS:
 - i. В проекте перейдите в раздел *Учетные данные*.
 - ii. Добавьте новые учетные данные с типом *Провайдер ASR/TTS*.
Подробнее о нужных данных читайте в статье [Учетные данные для ASR/TTS](#).
2. Перейдите в раздел *Интеграции*.
3. Создайте интеграцию с типом *Голос TTS*.

Подробнее о настройках для интеграции читайте в статье [Модели ASR и голоса TTS](#).

Вызов функции

Функция `Tts.synthesize` принимает следующие параметры:

- `ttsIntegrationKey` — идентификатор интеграции *Голос TTS*. В разделе *Интеграции* вы можете скопировать этот идентификатор на карточке интеграции. Пример: `1000001-name-abC`.
- `text` — текст для синтеза речи.
- `languageCode` — код языка, на котором будет синтезироваться речь. Возможные значения: `DE`, `KK` или `KZ`, `RU`, `US`, `UZ`. Ваш TTS-провайдер должен поддерживать выбранный язык.
- `audioFormat` — формат аудиофайла:
 - `MP3` — для провайдера ElevenLabs.
 - `WAV` — для всех остальных провайдеров.

Функция возвращает объект с результатами синтеза речи. Пример:

```
{
  "settings": {
    "type": "YANDEX",
    "languageCode": "ru-RU",
    "voice": "alena",
    "speakingRate": 1,
    "volume": -19,
    "role": "neutral",
    "useVariables": false
  },
  "result": {
    "url": "https://a12345b6-1234-41ec-ab00-790c5c412a3b.example.com/tts-cache/a123b45c-a123-45bc-d678-eef86d26abc1.wav"
  }
}
```

Здесь:

- `settings` — настройки интеграции TTS, которые использовались для синтеза речи.
- `result.url` — публичный URL, по которому можно скачать сгенерированный аудиофайл. Файл хранится на сервере бессрочно.

Перевод звонка на оператора

Если помощи агента оказалось недостаточно, звонок клиента можно перевести на оператора.

Как перевести звонок

Вызовите функцию `Dialer.transferCall()` **любым способом**: например, добавьте ее как отдельный шаг процесса или подключите в качестве **инструмента к агенту**.

```
Dialer.transferCall({
  phoneNumber: "71234567890",
  method: "invite",
  continueCall: true,
  continueRecording: true
});
```

Параметр	Описание
phoneNumber	Номер телефона оператора. Обязательный параметр.
timeout	Время ожидания ответа в секундах. По умолчанию — 60.
method	- invite — соединение проходит через платформу, можно записывать разговор с оператором. - refer — напрямую через SIP-провайдера. Параметры continueCall, continueRecording и headers игнорируются.
continueCall	Вернуть звонок клиента в процесс после разговора с оператором.
continueRecording	Записывать разговор с оператором. Записи доступны в разделе <i>История запусков</i> .
headers	Дополнительные SIP-заголовки. Передаются объектом, например: {"Example-Header": "value1", "Another-Header": "value2"}.

Результат перевода

После попытки перевода платформа отправляет событие `transfer` в триггер *Сообщение*.

Вызовите `Dialer.getTransferStatus()`, чтобы узнать результат:

Статус	Описание
<code>SUCCESS</code>	Оператор ответил на звонок.
<code>FAIL</code>	Перевод не удался: линия занята, номер недоступен.
<code>TIMEOUT</code>	Оператор не ответил в течение заданного времени.
<code>NOT_AVAILABLE</code>	Перевод еще не вызывался в этой сессии.

Варианты перевода

Без возврата в процесс

Звонок продолжается между клиентом и оператором, процесс в Agent Platform завершается.

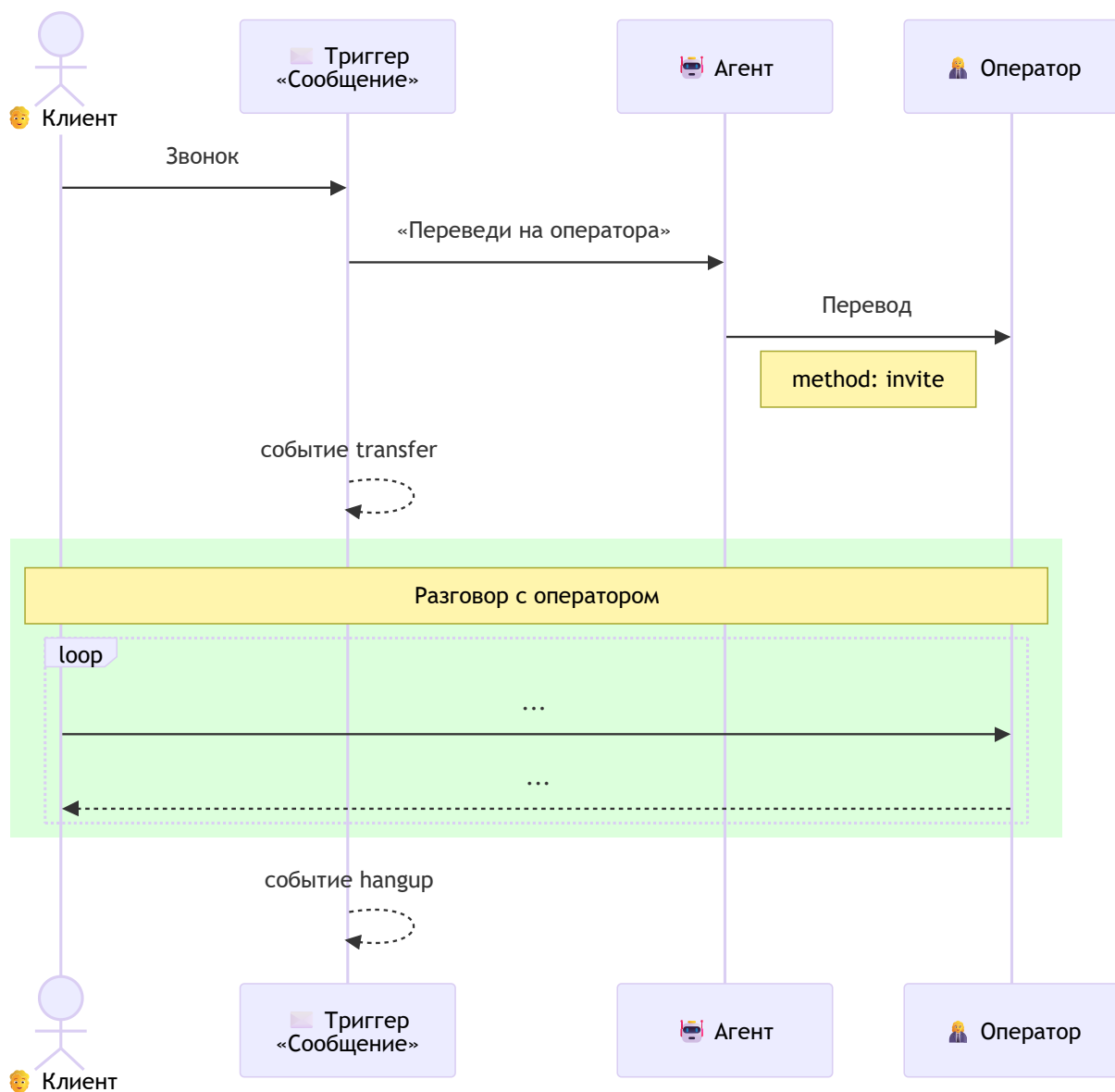
`invite` `refer`

Соединение идет через платформу. Можно записывать разговор с оператором.

```
Dialer.transferCall({
  phoneNumber: "71234567890",
  method: "invite",
  continueCall: false,
  continueRecording: true
});
```

События:

- `transfer` — приходит сразу, когда оператор ответил или перевод не удался.
- `hangup` — приходит после завершения разговора с оператором.



С возвратом в процесс

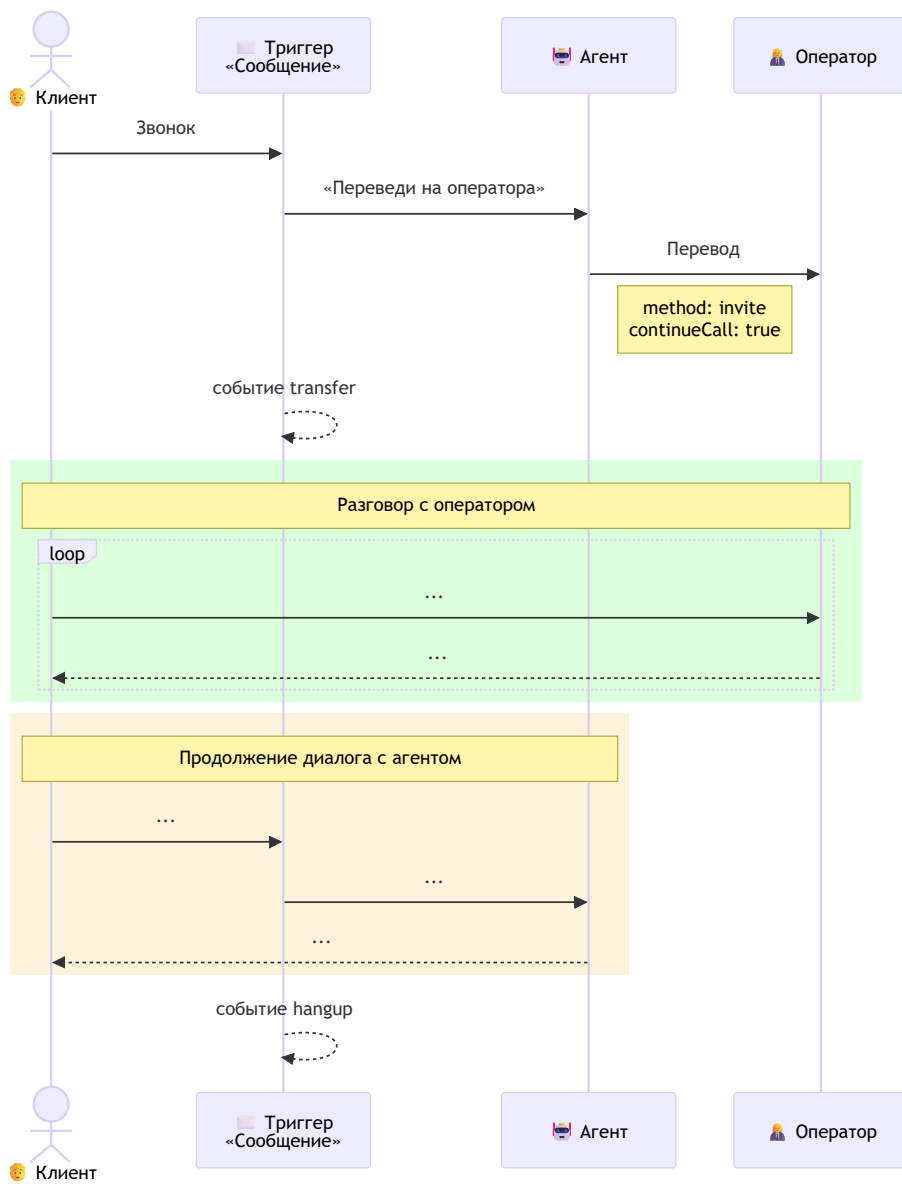
После разговора с оператором клиент возвращается в процесс. Если оператор не ответил — клиент тоже возвращается.

```

Dialer.transferCall({
  phoneNumber: "71234567890",
  method: "invite",
  continueCall: true
});
  
```

События:

- `transfer` — приходит сразу, когда оператор ответил или перевод не удался.
- `hangup` — приходит, если клиент положил трубку в любой момент разговора.



Подключение LLM

Интеграции с большими языковыми моделями (LLM) позволяют добавить в процесс **AI-агентов**, которые могут отвечать на любые вопросы и поддерживать диалог с пользователем. Интеграции с LLM также необходимы для использования встроенных функций из коллекции **Llm**.

Добавить интеграцию можно несколькими способами:

- На холсте в блоке *Агент*: в поле *Настройки* → *Основные* → *LLM* выберите *Создать интеграцию*.
- На холсте в блоке *Llm.sendRequest*, *Llm.sendText* или *Llm.summarizeText*: в поле *llmModelKey* выберите *Создать интеграцию*.
- В разделе *Интеграции*: нажмите *Подключить* → *LLM*.

Agent Platform поддерживает несколько способов доступа к LLM:

- **Через Just AI** — без регистрации у LLM-провайдеров.
- **Прямой доступ** — позволяет подключить модели ведущих провайдеров, таких как OpenAI, DeepSeek и Google, используя ваши учетные данные (API-ключи).
- **Своя LLM** — подходит для подключения моделей, которые поддерживают API, совместимый с форматом **OpenAI Chat Completions API**. Модели могут быть развернуты на ваших серверах или в облаке.

❗ ПРИМЕЧАНИЕ

Сайт OpenAI недоступен для российских IP-адресов.

Через Just AI Прямое подключение Своя LLM

1. В поле *Доступ к LLM* выберите *Через Just AI*.
2. Выберите провайдера и модель.
3. Отредактируйте название интеграции, если требуется.
4. Нажмите *Добавить*.

❗ К СВЕДЕНИЮ

Запросы к LLM через Just AI тарифицируются в **J-токенах**. Стоимость зависит от выбранной модели. Чтобы оценить расходы исходя из вашей нагрузки, воспользуйтесь [калькулятором стоимости](#).

Подключение баз знаний (RAG)

Использование баз знаний (RAG, от англ. Retrieval-Augmented Generation) — один из способов наделить AI-агента экспертизой в конкретной области.

Базы знаний позволяют агенту находить ответы в документах, статьях и других источниках. Благодаря этому он генерирует точные и релевантные ответы, основанные на ваших данных, а не на общих знаниях модели, которых может быть недостаточно.

Платформа Agent Platform имеет встроенную интеграцию с базами знаний, которые размещаются в [Jay Knowledge Hub](#) — платформе для создания, хранения и управления знаниями от компании Just AI. Если у вас есть аккаунт в Agent Platform, вы можете использовать его для доступа к Jay Knowledge Hub без дополнительной регистрации.

Создание подключения

Подготовьте в Jay Knowledge Hub базу знаний с документами, которые агент будет использовать для поиска ответов. Задайте необходимые настройки поиска релевантных фрагментов и генерации ответов — они будут использоваться при вызове функций из Agent Platform. Подробнее о создании базы знаний смотрите в [документации Jay Knowledge Hub](#).

❗ К СВЕДЕНИЮ

Agent Platform и Jay Knowledge Hub поддерживают работу в нескольких аккаунтах — группах проектов. Вы можете использовать одинаковые или разные аккаунты в Agent Platform и Jay Knowledge Hub. Это будет влиять на способ подключения.

📌 Выбор аккаунта в Agent Platform

1. В левом меню перейдите в раздел *Интеграции*.
2. Нажмите *Подключить* и выберите *База знаний RAG*.
3. В поле *Доступ к базе знаний* выберите, как вы будете подключаться к базе знаний:
 - *Текущий аккаунт* — если база знаний находится в том же аккаунте Jay Knowledge Hub, что и проект в Agent Platform.
 - *Другие учетные данные* — если база знаний находится в другом аккаунте Jay Knowledge Hub.

4. Заполните параметры в зависимости от выбранного способа подключения:

Текущий аккаунт **Другие учетные данные**

- Выберите нужный проект с базой знаний из списка.
- Укажите название интеграции — оно будет отображаться на странице с интеграциями.

5. Нажмите *Сохранить*.

Интеграция появится в списке, и вы сможете выбрать ее в настройках функций **Rag** на холсте.

Использование в процессе

После создания интеграции вы сможете использовать встроенные функции **Rag** в конструкторе процессов:

- **retrieveChunks** — ищет в базе знаний фрагменты документов, релевантные запросу. Полученные фрагменты можно передать агенту для генерации ответа или дополнительной обработки. В этом случае функцию нужно использовать как **инструмент**.
- **generateAnswer** — генерирует ответ на вопрос на стороне Jay Knowledge Hub, используя базу знаний. Эту функцию лучше использовать как **отдельный шаг в процессе**.

Режим «Функция»

Базовый режим, чтобы использовать функцию как отдельный шаг в процессе.

- **Как работает:** система вызывает выбранную функцию с заданными параметрами. Для получения результата в следующем блоке используйте метод `Context.getLastFunctionResult()`.
- **Когда использовать:** для задач, где вызов внешней системы — это четкий шаг в процессе. Например: «получить курс валют», «рассчитать доставку», «проверить статус заказа».

Настройка

1. В левом меню перейдите в *Конструктор* → .
2. Выберите *Функции* → *Встроенные* → *Rag*.
3. Перетащите функцию на холст и укажите параметры. Вы можете задать их вручную или использовать динамическую подстановку данных, например:
 - Для **движка версии 2**: `ID этого канала: {{ request.botId }}`.
 - Для версии 1: `{{ 'ID этого канала: ' + Context.getBotId() }}`.

4. Если в вашем проекте движок версии 2, вы можете задать параметры на вкладке *Поведение*:

- *Время ожидания ответа*: до 60 минут. Если выполнение функции не завершится за это время, будет зафиксирована ошибка.
- *Повторные попытки при ошибке*: до 10 повторных вызовов.
Например, если указать таймаут 20 секунд и количество повторных попыток 3, то суммарное время ожидания может составить до 80 секунд: $20 \times (1 + 3)$.
- *Возвращать указанное значение вместо ошибки*: если функция не выполнится успешно после всех попыток, процесс не прервется, а на выход блока будет передано заданное значение (число, строка, JSON-объект, true/false или null).

Пример

1. Добавьте на холст:

- триггер *Сообщение*;
- функцию `Rag.generateAnswer()`;
- функцию `Reactions.sendText()`.

Соедините их последовательно.

2. В настройках функции `Rag.generateAnswer()`:

- выберите из списка интеграцию с базой знаний;
- в параметре *query* укажите значение
 - для движка версии 2: `{% if request.query %} {{ request.query }} {% else %}Чем ты можешь помочь?{% endif %}`;
 - для версии 1: `{{ Context.getMessageContent().text || "Чем ты можешь помочь?" }}`.

3. В настройках функции `Reactions.sendText()` укажите значение:

- для движка версии 2: `{{ previousResult.response }}`;
- для версии 1: `{{ Context.getLastFunctionResult().response }}`.

Такой процесс передает запрос пользователя в базу знаний, а ответ базы знаний возвращает пользователю.

Режим «Инструмент»

Используется совместно с блоком *Агент*, позволяя AI самостоятельно решать, когда вызывать функцию.

- **Как работает:** вы не вызываете функцию напрямую — она становится «инструментом» для агента. Агент сам решает, когда и как ее использовать, исходя из диалога с пользователем.
- **Когда использовать:** для гибких систем, где агент взаимодействует с внешним миром: например, «забронируй переговорную» или «какой у меня баланс?».

💡 ПОДСКАЗКА

Подробнее о настройке функций в режиме инструмента смотрите в [разделе про блок «Агент»](#).

Вызов в коде

Функцию можно вызвать:

- Из выражений в блоках *Код* и *Условие*. Пример смотрите в разделах [Код](#) и [Условие](#).
- Из кода других (пользовательских) функций.
- Из параметров других JavaScript-функций (только если в вашем проекте [движок версии 1](#)).

▼ Пример

 **Reactions.sendText** 

Отмена Сохранить

Sends a text message as a response

☐ **Режим инструмента.** Функцию вызывает агент. Ответ не отправляется напрямую пользователю

Параметры функции

text: string
The text content to be sent

При вызове функций указывайте коллекцию, например:

```
// Вызов функции generateAnswer из коллекции Rag
Rag.generateAnswer();
```

❗ ПРИМЕЧАНИЕ

Из JavaScript-кода нельзя вызывать Python-функции, и наоборот.

Сохранение результата в контекст

Результат выполнения блока функции можно записывать в служебную память, используемую агентами. Подробнее смотрите в разделе [Управление контекстом агентов](#).

Подключение MCP-серверов

Интеграция с **MCP-серверами** (от англ. Model Context Protocol) — ключевой способ расширить возможности вашей системы. MCP-серверы позволяют AI-агентам и процессам безопасно взаимодействовать с внешними системами, базами данных и сервисами. После подключения MCP-сервера вы сможете **использовать его функции** как **шаги в бизнес-процессах** или как **инструменты для AI-агента**.

Платформа Agent Platform поддерживает подключение к разным типам MCP-серверов:

- Публичные реестры и маркетплейсы (например, **Smithery**).
- Приватные self-hosted инсталляции.
- Сервисы с прямым доступом по URL.

❗ ОСОБЕННОСТИ РАБОТЫ С MCP-СЕРВЕРАМИ

- Стабильность и доступность: MCP-серверы — это внешние сервисы. Платформа Agent Platform не отвечает за их работу и доступность.
- Потребление контекста: вызов функций и их описание занимают значительную часть контекстного окна, что может ухудшить качество работы моделей с небольшим контекстом. Рекомендуем использовать модели с большим контекстным окном.
- Стоимость: использование функций MCP-серверов может быть платным. Уточняйте тарифы у провайдера сервиса.

Создание подключения

❗ К СВЕДЕНИЮ

Платформа Agent Platform находится в активной разработке и тексты интерфейса могут меняться.

Чтобы добавить MCP-сервер в проект, выполните следующие шаги:

1. В левом меню перейдите в раздел *Интеграции*.
2. Нажмите *Подключить* и выберите *MCP*.
3. Заполните параметры:
 - *Название интеграции* — укажите понятное имя, которое будет отображаться в списке интеграций. Например, **Калькулятор тарифов** или **API базы знаний**.

- *Адрес MCP-сервера* — введите адрес, по которому доступен сервер.
- *Протокол подключения* — выберите протокол, который поддерживает MCP-сервер:
 - `Streamable HTTP` — современный HTTP-транспорт MCP: клиент и сервер обмениваются сообщениями через единый HTTP-эндпоинт.
 - `SSE` — транспорт MCP через HTTP+SSE: сервер отправляет сообщения клиенту через поток Server-Sent Events.

Поддерживаемый протокол должен быть указан в документации MCP-сервера.

- *Имя коллекции* — укажите имя для группы функций, полученных с сервера. Оно будет использоваться как префикс и поможет избежать конфликтов при нескольких MCP-интеграциях.

По нему вы также сможете найти функции в конструкторе.

- *Учетные данные* — если требуется аутентификация (например, API-ключ), выберите **учетные данные** из списка или создайте новые. Если доступ открыт, оставьте поле пустым.

4. Нажмите *Добавить*.

Платформа автоматически подключится к серверу, получит список доступных функций и их параметры. Теперь вы можете использовать их в процессах.

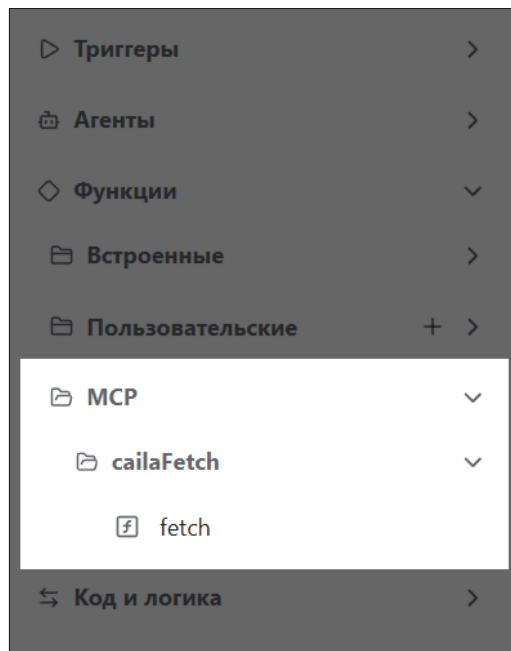
Пример: подключение MCP Fetch

Рассмотрим подключение на примере **MCP-сервера Fetch** из каталога Caila. Этот сервер предоставляет функцию для получения и обработки веб-контента: содержимого веб-страниц, API-эндпоинтов и других HTTP-ресурсов.

При добавлении интеграции укажите следующие параметры:

- *Название* — `CailaFetch`.
- *Адрес MCP-сервера* — `https://just-ai-mcp-fetch.app.caila.io/mcp`.
- *Протокол подключения* — `Streamable HTTP`.
- *Имя коллекции* — `cailaFetch`.
- *Учетные данные* — *Без учетных данных*.

После сохранения интеграции она появится в списке, а в конструкторе процессов вы сможете найти функцию `cailaFetch.fetch`.



Использование в процессе

После подключения вы можете использовать функции MCP на холсте.

Режим «Функция»

Базовый режим, чтобы использовать функцию как отдельный шаг в процессе.

- **Как работает:** система вызывает выбранную функцию с заданными параметрами.
Для получения результата в следующем блоке используйте метод `Context.getLastFunctionResult()`.
- **Когда использовать:** для задач, где вызов внешней системы — это четкий шаг в процессе.
Например: «получить курс валют», «рассчитать доставку», «проверить статус заказа».

Настройка

1. В левом меню перейдите в *Конструктор* → .
2. Выберите *Функции* → *MCP*.
3. Перетащите функцию на холст и укажите параметры. Вы можете задать их вручную или использовать динамическую подстановку данных, например:
 - Для **движка версии 2**: `ID этого канала: {{ request.botId }}`.
 - Для версии 1: `{{ 'ID этого канала: ' + Context.getBotId() }}`.
4. Если в вашем проекте движок версии 2, вы можете задать параметры на вкладке *Поведение*:
 - *Время ожидания ответа*: до 60 минут. Если выполнение функции не завершится за это время, будет зафиксирована ошибка.
 - *Повторные попытки при ошибке*: до 10 повторных вызовов.

Например, если указать таймаут 20 секунд и количество повторных попыток 3, то суммарное время ожидания может составить до 80 секунд: $20 \times (1 + 3)$.

- *Возвращать указанное значение вместо ошибки*: если функция не выполнится успешно после всех попыток, процесс не прервется, а на выход блока будет передано заданное значение (число, строка, JSON-объект, true/false или null).

Режим «Инструмент»

Используется совместно с блоком *Агент*, позволяя AI самостоятельно решать, когда вызывать функцию.

- **Как работает**: вы не вызываете функцию напрямую — она становится «инструментом» для агента. Агент сам решает, когда и как ее использовать, исходя из диалога с пользователем.
- **Когда использовать**: для гибких систем, где агент взаимодействует с внешним миром: например, «забронируй переговорную» или «какой у меня баланс?».

Подтверждение перед вызовом

Если функция проводит платеж, удаляет важные данные или выполняет другое критическое действие, запросите у клиента подтверждение перед ее выполнением.

💡 ПОДСКАЗКА

Подробнее о настройке функций в режиме инструмента смотрите в [разделе про блок «Агент»](#).

Вызов в коде

Функцию можно вызвать:

- Из выражений в блоках *Код* и *Условие*. Пример смотрите в разделах [Код](#) и [Условие](#).
- Из кода других (пользовательских) функций.
- Из параметров других JavaScript-функций (только если в вашем проекте [движок версии 1](#)).

▼ Пример

📄

Reactions.sendText

✎

ОтменаСохранить

Sends a text message as a response

☐

Режим инструмента. Функцию вызывает агент. Ответ не отправляется напрямую пользователю

Параметры функции

text: string

The text content to be sent

{{ID этого канала: ' + Context.getBotId()}}

При вызове функций указывайте коллекцию, например:

```
// Вызов функции websearch из коллекции SearchMCP
SearchMCP.websearch();
```

📘 ПРИМЕЧАНИЕ

Из JavaScript-кода нельзя вызывать Python-функции, и наоборот.

Сохранение результата в контекст

Результат выполнения блока функции можно записывать в служебную память, используемую агентами. Подробнее смотрите в разделе [Управление контекстом агентов](#).

Подключение IMAP-сервера

Чтобы использовать триггер **IMAP Email**, необходимо создать интеграцию «IMAP-сервер».

1. В разделе *Интеграции* выберите *Подключить* → *IMAP-сервер*.
2. Задайте *Название*, чтобы эту интеграцию было легко находить.
3. Укажите *Адрес сервера*, *Порт* и использовать ли *SSL-подключение*. Эти данные вы можете найти в документации вашего email-сервиса.
4. Выберите или создайте **учетные данные**.
5. Нажмите *Добавить*.

❗ ПРИМЕЧАНИЕ

После добавления интеграции Agent Platform проверит, что подключение работает. Если с ними возникнет какая-то проблема, в правом нижнем углу окна появится ошибка.

Примеры настроек для популярных сервисов

При настройке интеграции вам понадобятся параметры IMAP-сервера. Обычно используются порт **993** и шифрование **SSL/TLS**.

Сервис	Адрес сервера (Host)	Порт	Особенности пароля
Gmail	imap.gmail.com	993	Требуется включить 2FA и создать пароль приложения .
Yandex	imap.yandex.ru	993	Требуется включить доступ по IMAP в настройках и создать пароль приложений .
Outlook / Office365	outlook.office365.com	993	Для учетных записей, отличных от Майкрософт , вам может потребоваться обратиться к администратору за информацией.
Mail.ru	imap.mail.ru	993	Используйте пароль для внешних приложений .

Подключение SMTP-сервера

Интеграция с SMTP-сервером позволяет использовать встроенную функцию `Email.send()` для отправки email-сообщения через указанный сервер.

Agent Platform поддерживает два типа подключения к SMTP-серверу:

- **Через сервер Just AI** — не требует настройки сервера. Вы просто создаете интеграцию и можете сразу отправлять письма. Сообщения будут отправляться от имени `notification@just-ai.com`.
- **Прямое подключение** — позволяет использовать ваш корпоративный SMTP-сервер.

Через сервер Just AI Прямое подключение

1. В разделе *Интеграции* выберите *Подключить* → *SMTP-сервер*.
2. Выберите тип подключения *Через сервер Just AI*.
3. Задайте *Название интеграции*, чтобы ее было легко находить.
4. Нажмите *Добавить*.

Учетные данные для интеграций

Когда вы создаете интеграцию с LLM, каналом или другим внешним сервисом, то вы можете использовать свои учетные данные для аутентификации.

В Agent Platform есть специальное хранилище для ваших паролей, токенов и логинов — раздел *Учетные данные*. Чтобы перейти в него, откройте проект и выберите раздел на левой панели.

Раздел позволяет:

- **Безопасно хранить данные**

Учетные данные не будут видны в явном виде в блоках или коде.

- **Легко использовать данные**

Если вы добавили данные в хранилище, то сможете выбирать их в настройках интеграций или получать в процессах с помощью встроенной функции `Credentials.get()`.

Управление учетными данными

Добавление

1. Нажмите *Добавить*.
2. Выберите тип учетных данных. Набор данных отличается в зависимости от типа.

ПРИМЕЧАНИЕ

Каждая интеграция поддерживает один определенный тип учетных данных:

- Например, если вы хотите настроить интеграцию с LLM, то вам нужно создать учетные данные типа *LLM*.
- Если подходящей встроенной интеграции нет, выберите тип *Другое*. Далее напишите свою функцию для обращения к внешнему сервису и используйте в ней учетные данные с помощью `Credentials.get()`.
- Данные типа *Провайдер ASR/TTS* можно использовать как для ASR, так и для TTS. Если у вас провайдеры ASR и TTS разные, создайте по одному набору учетных данных для каждого провайдера.

3. Укажите нужные данные и настройте срок действия.

4. Нажмите *Добавить*.

Редактирование

1. Нажмите  в таблице с вашими учетными данными.
2. Внесите изменения.
3. Нажмите *Сохранить*.

Во всех связанных интеграциях сразу будут использоваться новые данные. Также обновленные данные будет возвращать `Credentials.get()`.

Удаление

1. Нажмите  в таблице с вашими учетными данными.
2. Подтвердите удаление.

ПРЕДУПРЕЖДЕНИЕ

Связанные интеграции сразу перестанут работать — это может привести к ошибкам в процессах.

Как использовать

Выбор данных во встроенных интеграциях

Сохраненные учетные данные можно выбрать в двух местах:

- В разделе *Интеграции* при подключении нового сервиса.
- Прямо в процессе — например, при настройке LLM в блоке *Агент*.

Новая интеграция

← LLM

Доступ к LLM

Прямой доступ

Провайдер

OPEN_AI

Модель

gpt-4o

Название

OPEN_AI - gpt-4o

Учетные данные

Мой ключ OpenAI [LLM]

Добавить новые

В обоих случаях вам не придется вводить токен или ключ API вручную — просто выберите нужные данные из выпадающего списка.

Подключение к любому другому сервису

Например, вы хотите, чтобы ваш бот мог сообщать погоду, используя сторонний сервис [WeatherAPI.com](https://weatherapi.com). Однако встроенной интеграции с этим сервисом в Agent Platform нет и вам нужно обращаться к нему напрямую по API.

Для таких задач используйте функцию `Credentials.get()`. Она позволяет безопасно получить сохраненный API-ключ прямо в коде и использовать его для отправки запросов.

Шаг 1: Сохраните API-ключ

1. Зарегистрируйтесь на [WeatherAPI.com](https://weatherapi.com) и получите бесплатный API-ключ.
2. В Agent Platform перейдите в раздел *Учетные данные*.
3. Добавьте новые учетные данные с типом *Другое* и авторизацией по токenu. В поле *Токен* вставьте ваш API-ключ.
4. Новые учетные данные появятся в таблице в разделе *Учетные данные*. Скопируйте ID из таблицы. Предположим, что ID выглядит так: `1000111111-weather-abc`.

Шаг 2: Напишите функцию для запроса погоды

Теперь вы можете написать простую функцию, которая будет обращаться к API погоды. В ней используйте `Credentials.get()`, чтобы безопасно получить ключ. Реализация зависит от [версии движка](#) в вашем проекте.

```
async function getWeather({ city }) {  
  // Получаем объект с учетными данными по ID из таблицы  
  const credentials = Credentials.get({credentialKey: '1000111111-weather-abc'});  
  
  // Формируем запрос  
  const requestConfig = {  
    url: 'https://api.weatherapi.com/v1/current.json',  
    params: {  
      // Подставляем токен из наших учетных данных  
      key: credentials.token,  
      q: city  
    }  
  };  
  
  // Выполняем запрос с помощью встроенной функции Http.get  
  const response = await Http.get(requestConfig);  
  
  // Извлекаем температуру из ответа и возвращаем результат  
  return `Температура: ${response.body.current.temp_c}°C`;  
}
```

Таким образом, ваш API-ключ надежно хранится в Agent Platform и не виден в коде функции.

Клиенты

Чтобы изучить поведение клиентов, перейдите в проект и выберите *Клиенты* в меню слева. В этом разделе представлен список клиентов с их статистикой и историей диалогов.

❗ ПРИМЕЧАНИЕ

- Клиент определяется на уровне типа канала в рамках проекта. Если удалить канал, данные останутся доступны в течение срока хранения. Если создать канал заново — клиент останется тем же.
- Для запусков процесса по триггерам *Вебхук* или *Планировщик* регистрируется служебный клиент с именем `workflow-thread_<ID триггера>`, не связанный с реальным пользователем.

Поиск и фильтрация

Чтобы найти нужных клиентов, используйте поисковую строку и фильтры.

Поиск работает по:

- ID клиента;
- имени клиента, полученному из канала (может отсутствовать);
- тексту сообщения от клиента.

Отфильтровать список клиентов можно по следующим параметрам:

- *Дата начала* — выберите из списка период, в который попадает первое событие сессии клиента, или укажите диапазон дат и времени.
- *Канал* — выберите из списка один из подключенных каналов.

Экспорт данных

Чтобы выгрузить данные о клиентах в файл, нажмите *Скачать CSV*.

Список клиентов в файле зависит от примененных фильтров. Чтобы скачать отчет по всем клиентам, сбросьте фильтры.

Формирование отчета занимает некоторое время. Скачивание начнется автоматически.

Детализация по клиенту

Нажмите на строку клиента в таблице, чтобы открыть детализацию. В ней доступны две вкладки с возможностью поиска и фильтрации:

- *Сессии* — список сессий клиента. Чтобы прочитать сообщения в диалоге, нажмите на строку сессии.
- *История запусков* — список запусков процесса, связанных с этим клиентом. Чтобы прочитать журнал выполнения, нажмите на строку запуска.

ПОДСКАЗКА

Прослушивание аудиозаписей для запусков из телефонных каналов доступно в истории запусков.

Срок хранения

Данные о клиентах и их сессиях хранятся в течение года. Журналы запусков хранятся 3,5 месяца.

Сессии

Сессия — это последовательность запусков процесса в рамках одного диалога с клиентом.

❗ ПРИМЕЧАНИЕ

В телефонном канале один звонок — это одна сессия. В текстовых каналах платформа не завершает сессию автоматически, однако некоторые каналы могут сами закрыть сессию по таймауту.

Чтобы проанализировать сессии, перейдите в проект и выберите *Сессии* в меню слева.

Поиск и фильтрация

Чтобы найти нужные сессии, используйте поисковую строку и фильтры.

Поиск работает только по ID сессии.

Отфильтровать список сессий можно по следующим параметрам:

- *Дата начала* — выберите из списка период, в который попадает первое событие сессии, или укажите диапазон дат и времени.
- *ID клиента* — выберите идентификатор клиента из списка.

Экспорт данных

Чтобы выгрузить данные о сессиях в файл, нажмите *Скачать CSV*.

Список сессий в файле зависит от примененных фильтров. Чтобы скачать отчет по всем сессиям, сбросьте фильтры.

Формирование отчета занимает некоторое время. Скачивание начнется автоматически.

Детализация сессии

Нажмите на строку сессии в таблице, чтобы открыть историю событий в формате чата. В конце каждого запуска отображается ссылка *Детали* для просмотра журнала.

Срок хранения

Данные о клиентах и их сессиях хранятся в течение года. Подробные журналы для истории запусков хранятся 3,5 месяца.

Приглашение пользователей в аккаунт

Agent Platform поддерживает совместную работу в аккаунте. Вы можете пригласить других пользователей для работы с платформой и назначить им подходящие роли. В зависимости от роли пользователи смогут просматривать или редактировать проекты внутри аккаунта.

ПОДСКАЗКА

Если над проектом работает одновременно несколько человек, обычного доступа к аккаунту недостаточно — есть риск случайно перезаписать чужие изменения. Подключите проект к [Git-репозиторию](#): так изменения будут проходить через pull и push, сохраняться в истории версий, а конфликты можно будет разрешать в специальном редакторе.

Доступ для новых пользователей

Вы можете приглашать пользователей в те аккаунты, которыми управляете. Для таких аккаунтов отображается надпись «👤 Управляю».

Чтобы пригласить пользователей:

1. Перейдите в раздел **Аккаунты** в [Conversational Cloud](#).
2. Выберите аккаунт, которым вы управляете.
3. В аккаунте нажмите *Пригласить пользователя*.
4. Выберите продукт Agent Platform.
5. Выберите **группы и роли** для пользователей:
6. Укажите email пользователей.
7. Нажмите *Пригласить*.

Подробно о приглашении пользователей читайте в [документации](#) Conversational Cloud.

Доступ для ранее приглашенных пользователей

Пользователям вашего аккаунта, у которых еще нет доступа к Agent Platform, вы можете предоставить его:

1. Перейдите в раздел **Аккаунты** в Conversational Cloud.
2. Выберите аккаунт, которым вы управляете.
3. В аккаунте нажмите ... напротив пользователя и выберите *Редактировать*.
4. Выберите группу *Agent Platform* и соответствующую роль.
5. Нажмите *Сохранить*.

Роли и доступы

Роли позволяют разграничить доступ пользователей к функциям Agent Platform.

Общие роли

- **USER_ADMIN** предназначена для администратора, который отвечает за организацию прав доступа. Он может управлять пользователями и их ролями, а также просматривать действия пользователей.

❗ К СВЕДЕНИЮ

Если пользователь имеет в аккаунте роль **USER_ADMIN**, на карточке этого аккаунта он увидит подпись «👑 Управляю».

- **USER_AUDITOR** предназначена для просмотра данных о пользователях. Аудитор может видеть список пользователей, их действия и приглашения, но не может ничего менять.
- **BILLING_ADMIN** предназначена для администратора, который отвечает за управление тарифами и лимитами.

Роли Agent Platform

- **AP_ADMIN** предназначена для администратора платформы. Она предоставляет полный доступ ко всем функциям Agent Platform: управлению проектами, их импорту и экспорту, базами данных, интеграциями и учетными данными, конструктору, аналитике и работе с Git-репозиториями.
- **AP_DEVELOPER** предназначена для разработчиков. Позволяет создавать проекты, экспортировать и импортировать их, редактировать сценарии в конструкторе, работать с функциями, использовать Git-репозитории и просматривать данные в базах, но не дает возможность создавать или изменять интеграции.
- **AP_VIEWER** предназначена для просмотра проектов и баз данных, в том числе в конструкторе, но не дает возможности редактировать их.

Доступы	AP ADMIN	AP DEVELOPER	AP VIEWER
Создание и удаление проекта	Полный доступ	Полный доступ	Недоступно
Экспорт и импорт проекта	Полный доступ	Полный доступ	Недоступно
Конструктор	Полный доступ	Полный доступ	Только просмотр
Создание, редактирование, удаление блоков	Полный доступ	Полный доступ	Только просмотр
Сохранение, публикация, тестирование процессов	Полный доступ	Полный доступ	Только просмотр
Git-репозиторий	Полный доступ	Полный доступ	Недоступно
Встроенные и пользовательские функции	Полный доступ	Полный доступ	Только просмотр
Интеграции, MCP, каналы, учетные данные	Полный доступ	Только использование готовых	Только просмотр
Базы данных	Полный доступ	Только просмотр и поиск по БД	Только просмотр и поиск по БД
Клиенты и сессии	Полный доступ	Полный доступ	Полный доступ

Управление тарифом и балансом

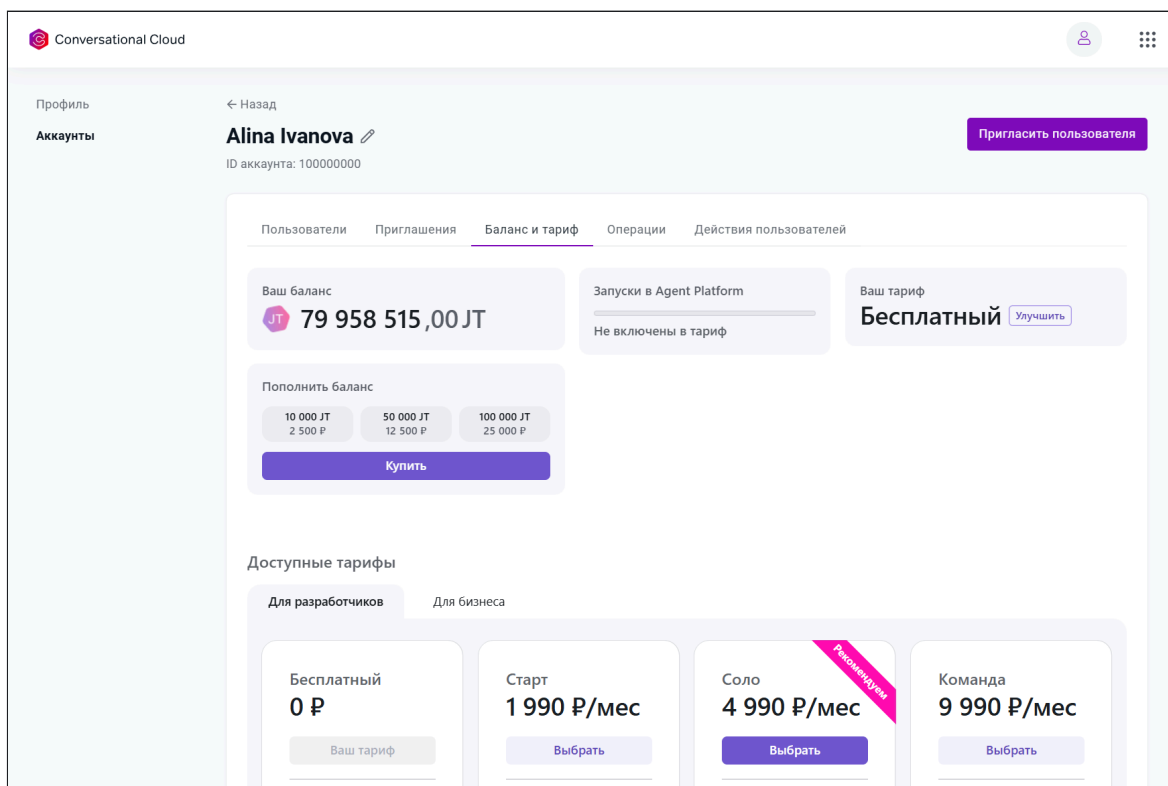
❗ К СВЕДЕНИЮ

Для просмотра и пополнения баланса, а также управления тарифом необходима роль `BILLING_ADMIN`. Описание роли смотрите в разделе [Общие роли](#) документации Conversational Cloud.

Управляйте тарифом Agent Platform и балансом J-токенов в личном кабинете [Conversational Cloud](#).

1. Перейдите в раздел [Аккаунты](#) в Conversational Cloud.
2. Выберите аккаунт, которым вы управляете.
3. Перейдите на вкладку *Баланс и тариф*.

На этой вкладке вы можете посмотреть баланс и текущий тариф, пополнить J-токены, изменить тарифный план и проверить лимиты.



❗ К СВЕДЕНИЮ

Подробнее о тарифах читайте в соответствующей [статье документации](#).

Как изменить тариф

1. Ознакомьтесь с тарифами в разделе *Доступные тарифы*.
2. Нажмите *Выбрать* для желаемого тарифа. Откроется окно с подтверждением покупки.
На этом этапе вы также сможете пополнить ваш баланс J-токенов.
Они нужны для оплаты запросов к LLM, работы с базами знаний, а также запусков сверх лимита, установленного тарифным планом.
3. Нажмите *Перейти к оплате* и завершите покупку. Новый тариф будет активирован.

❗ К СВЕДЕНИЮ

Если на старом тарифе остались неизрасходованные запуски, они автоматически добавятся к лимиту нового тарифа. Эти запуски расходуются в первую очередь и сгорают по прежнему сроку (а не по сроку нового тарифа).

Как пополнить баланс J-токенов

J-токены нужны для оплаты запросов к LLM, работы с базами знаний, а также запусков сверх лимита, установленного тарифным планом. В тарифные планы J-токены не входят — их нужно покупать отдельно:

1. На странице *Баланс и тариф* выберите подходящий пакет J-токенов.

Ваш баланс

JT

175 340,045 JT

Пополнить баланс

10 000 JT
2 475 Р

50 000 JT
12 375 Р

75 000 JT
18 562 Р

Другое количество JT

Купить

2. Нажмите *Купить* и завершите покупку на странице оплаты.

Как подключить автоплатеж

Автоплатеж автоматически продлевает ваш тариф каждый месяц. Если автоплатеж не подключен, тариф нужно продлевать вручную. Если оплата не поступит в срок, платформа переведет аккаунт на бесплатный тариф.

Подключить автоплатеж можно:

- При покупке тарифа
- На странице «Баланс и тариф»

Активируйте переключатель *Автопродление тарифа* при подтверждении покупки:

Подтверждение покупки

1. Тариф

Соло 4 990 Р/мес

Автопродление тарифа ⓘ

Преимущества тарифа Соло

✓ 50 000 запусков по тарифу

✓ 1 JT за каждый доп. запуск

✓ 1 JT за запрос в базу знаний RAG

2. Пакет токенов

Пакет токенов приобретается в дополнение к тарифу ⓘ

100 000 JT

25 000 Р

300 000 JT

75 000 Р

500 000 JT

125 000 Р

Другое количество

 JT

- Р

Не нужны токены

Всего

129 990 Р

Отмена

Перейти к оплате

Как отменить автоплатеж

Чтобы отменить автоплатеж, в разделе *Ваш тариф* отключите опцию *Автоплатеж*. Вы также можете удалить привязанную карту.

Техническая поддержка

Если вы не нашли ответа на свой вопрос в документации, обратитесь в техническую поддержку: напишите письмо на адрес support@just-ai.com или заведите заявку на [портале](#). Наши специалисты проконсультируют вас и помогут решить проблему.

Сообщество в Telegram

Также вступайте в сообщество Agent Platform в Telegram: https://t.me/Just_AI_Agent_Platform.

Здесь вы можете оставить обратную связь, предложить улучшения и поделиться опытом с другими пользователями.

